Programming I
Solutions & Commentary

程序设计基础 I（Python） · 练习代码册

Demo answers & walkthroughs for every chapter exercise and the final project

---

The content of this handbook was developed and polished by ZHANG Feng together with multiple LLM models and AI tools.

本课程手册由 ZHANG Feng 借助多种大语言模型及 AI 工具编撰打磨而成。

For full Python learning, refer primarily to the official website: https://docs.python.org/

若要完整学习 Python，请优先参考 Python 官方文档：https://docs.python.org/

2026 · First Edition

Programming I
Solutions & Commentary

程序设计基础 I（Python） · 练习代码册

程序设计基础 I（Python）：练习代码册

Programming I with Python — Solutions & Commentary

First Edition, 2026

© Prepared for teaching use at Hainan Bielefeld University of Applied Sciences.

Course website: www.bioinfo-lab.com/courses/

All solutions target Python 3.7 and later on the course Linux server.

Preface

前言

This is the companion volume to *Programming I with Python*. The handbook teaches; this codebook answers. Every exercise printed at the end of Chapters 1 to 16 is reproduced here together with a complete, runnable demo answer, the output you should expect on the course server, and a step-by-step commentary explaining why the code is written the way it is. Chapter 17 — the final project — receives the same treatment at greater depth: a full annotated reference implementation of the AI movie-scriptwriter program, broken down function by function.

本书是《Programming I with Python》学习手册的配套卷。学习手册负责讲解，本册负责解答。第 1 章至第 16 章末尾的每一道练习题，在这里都被完整重印，并配上一份可直接运行的示范答案、你应该在课程服务器上看到的输出结果，以及逐行解释"代码为何这样写"的精解。第 17 章的期末项目则得到更深入的对待：一份完整的、逐段注释的 AI 电影编剧程序参考实现，并逐个函数拆解。

Three rules make this book useful instead of dangerous.

有三条规则决定这本书是有用的，而不是有害的。

- Attempt first, read second. An answer you read before you struggle is information; an answer you read after you struggle is education. Budget at least fifteen minutes per exercise before opening this volume.

  先做，再看。在你挣扎之前读到的答案只是信息；在你挣扎之后读到的答案才是教育。打开本册之前，请为每道题至少预留十五分钟。

- Type, do not paste. Your fingers build the memory that your eyes cannot. Every program here is short enough to type by hand, and typing is where indentation, colons and quotes become instinct.

  动手敲，不要复制粘贴。手指建立的记忆是眼睛建立不了的。这里每个程序都短到可以手敲，而正是在敲的过程中，缩进、冒号和引号才会变成本能。

- Compare, then criticise. The code in this book is one correct answer, not *the* answer. When your version differs, ask which one is clearer, shorter, or safer — and write that comparison into your weekly report.

  比较，然后批评。本书的代码是"一种"正确答案，而不是"唯一"答案。当你的写法与之不同时，问问哪一种更清晰、更简短或更安全——并把这份比较写进周报。

How This Book Is Organised

本册结构

Each chapter opens with a short note on what the handbook asked of you, then presents one entry per exercise. Every entry contains the same five parts, so you always know where you are.

每一章开头先用一小段说明学习手册对你的要求，然后每道题给出一个条目。每个条目都包含相同的五个部分，让你随时知道自己在哪。

Part	What it contains	Why it is there
The task 题目要求	The exercise quoted verbatim from the handbook, in English and Chinese. 从学习手册原样引用的题目，中英对照。	You can work completely inside this book. 让你可以只在本书内完成练习。
Demo answer 示范答案	One complete runnable program, with a file name above it. 一份完整可运行的程序，上方标注文件名。	A working model to compare against yours. 供你与自己的答案对照的可运行范例。
Output 输出	What the program prints on the course server. 程序在课程服务器上打印的结果。	Proof that the answer really runs. 证明这份答案真的能跑通。
Walkthrough 精解	Numbered comments on the decisions, the syntax and the traps. 对设计决策、语法与陷阱的编号讲解。	Turns a working program into an understood one. 把"能跑的程序"变成"懂了的程序"。
Common mistake 常见错误	The error most students hit on this exercise. 大多数学生在这道题上踩的坑。	Saves you one debugging session. 帮你省下一次调试。

Two notes on the code itself. First, all programs target Python 3.7 and later and were run on the course Linux server; if your local machine behaves differently, trust the server. Second, where the handbook asks you to *explain* something rather than to write a program, the "demo answer" is a short demonstration you can run to produce the evidence your report needs.

关于代码本身有两点说明。其一，所有程序面向 Python 3.7 及以上版本，并在课程 Linux 服务器上实际运行过；如果你的本地机器表现不同，请以服务器为准。其二，凡是学习手册

要求你"解释"而不是"编写"的题目，这里的"示范答案"是一段可以运行的小演示，用来为你的报告提供证据。

Table of Contents

Foundations / 基础 *9*

Linux and the Command Line / Linux 与命令行基础 10

Your First Python Programs / 第一个 Python 程序 18

Variables and Data Types / 变量与数据类型 25

Operators and Expressions / 运算符与表达式 31

Working with Strings / 字符串操作 37

Control Flow / 控制流 *44*

Making Decisions / 条件判断 45

Going in Loops / 循环结构 50

Abstraction / 抽象 *58*

Functions / 函数 59

Modules and the Standard Library / 模块与标准库 67

Data Structures / 数据结构 *75*

Lists and Tuples / 列表与元组 76

Dictionaries and Sets / 字典与集合 84

The Real World / 与真实世界对接 *93*

Reading and Writing Files / 文件读写 94

File Systems and Encodings / 文件系统与字符编码 101

Errors and Exception Handling / 错误与异常处理 109

Third-Party Libraries and pip / 第三方库与 pip 118

Doing Several Things at Once / 多任务处理 124

The Final Project / 期末项目 130

The Python Movie Scriptwriter — Demo Answer / Python 电影编剧程序——示范答案 131

Reference / 附录与参考资料 139

Index of Demo Programs / 示范程序索引 140

From Exercise to Submission / 从练习到提交 145

A Field Guide to Errors in This Book / 本书错误现场速查 146

PART I

Foundations / 基础

Part I covers Chapters 1 to 5 of the handbook: reaching the server, running Python, storing values in variables, computing with operators, and taking text apart. The exercises here look small, and they are — but each one installs a habit that later chapters assume.

第一部分对应学习手册第 1 章至第 5 章：连接服务器、运行 Python、把值存入变量、用运算符计算、拆解文本。这里的练习看起来很小，也确实很小——但每一道都在安装一个后续章节默认你已具备的习惯。

Linux and the Command Line / Linux 与命令行基础

> *Five exercises, five habits: connect, create, inspect, edit, transfer.*
>
> 五道题，五个习惯：连接、创建、查看、编辑、传输。

Chapter 1 has no Python in it at all — and that is deliberate. Before you can run a program on the course server you must be able to stand somewhere in its file system and know where you are standing. All five answers below are command-line sessions; the "program" is the sequence of commands itself.

第 1 章完全没有 Python 代码——这是刻意的。在课程服务器上运行程序之前，你必须先能站在它的文件系统某处，并知道自己是站在哪。以下五份答案都是命令行会话；"程序"就是命令序列本身。

1.1 Connecting with Termius

用 Termius 连接服务器

> **THE TASK** 题目要求
>
> - Install Termius, register a free account and log in to the course server. Take a screenshot of your home directory.
>
> 安装 Termius，注册免费账号并登录课程服务器，截图你的主目录。

建立连接需要且只需要四个值；登录是否成功由一条命令确认：`pwd` 必须打印出你自己的主目录，而不是别人的。

▾ **Termius host setup** Termius 主机配置

```
# The four values you type into Termius / 填入 Termius 的四个值
#   IP / 地址       : read from https://www.bioinfo-lab.com/ip.txt
#   Port / 端口     : 13579
#   Username / 用户名: dtX20260001        (dt + your student ID / dt + 学号)
#   Password / 密码 : dtX20260001@biuh2026

# After connecting, confirm where you landed / 连上后确认你站在哪里
pwd
```

```
ls
whoami
```

OUTPUT 输出

```
/home/dtX20260001
desktop  documents  downloads  public_html  tmp
dtX20260001
```

Walkthrough 精解

1. `pwd` (print working directory) is the answer to "where am I". After login it should show `/home/dtX20260001` — if it shows `/` or `/home`, you are not in your own home and every later command will write to the wrong place.

   `pwd`（打印工作目录）回答"我在哪"。登录后它应该显示 `/home/dtX20260001`；如果显示 `/` 或 `/home`，说明你不在自己的主目录里，之后每条命令都会写到错误的地方。

2. `whoami` prints the account name the server sees. It is the cheapest possible check that you logged in as yourself rather than as a shared account.

   `whoami` 打印服务器所看到的账户名。这是验证"你是以自己的身份登录，而非共享账号"最廉价的手段。

3. The screenshot requirement is not decoration. Your weekly report is graded on evidence, and `pwd` + `ls` together are the smallest honest evidence that the connection works.

   截图要求不是装饰。周报按证据评分，而 `pwd` + `ls` 组合起来就是"连接可用"的最小诚实证据。

> **COMMON MISTAKE 常见错误**
>
> Typing the port as `22`. The course server deliberately listens on `13579`, so a default SSH configuration will be refused. If Termius reports "Connection refused", check the port before you check anything else.
>
> 把端口写成 `22`。课程服务器刻意监听在 `13579` 上，默认的 SSH 配置会被拒绝。如果 Termius 报 "Connection refused"，先查端口，再查其他。

1.2 The Full Command Sequence

完整命令序列

> **THE TASK 题目要求**
>
> - Run the full sequence in Section 1.4: create a folder and a file, copy, move, rename, and delete them. Document every step.

> 完成第 1.4 节的完整流程：建目录、建文件、复制、移动、重命名、删除，并记录每一步。

一道题，七个动词。按这个顺序执行，每条命令的输出就是它生效的证据。

▾ **command_sequence.sh** run it in your home directory / 在主目录下运行

```
cd ~
pwd                                  # 0. where am I? / 我在哪?

mkdir new_folder                     # 1. make a folder / 建目录
touch new_file.txt                   # 2. make an empty file / 建空文件
ls                                   # 3. both should now be listed / 两者都应出现在
列表里

cp new_file.txt copy_file.txt        # 4. copy the file / 复制文件
mv copy_file.txt renamed_file.txt    # 5. rename by moving / 用移动来重命名
ls

rm renamed_file.txt                  # 6. delete the file / 删除文件
rm -r new_folder                     # 7. delete the folder / 删除目录
ls                                   # 8. prove both are gone / 证明两者都已消失
```

OUTPUT 输出

```
/home/dtX20260001
new_folder  new_file.txt
new_file.txt  new_folder  renamed_file.txt
new_file.txt
```

Walkthrough 精解

1. `mkdir` and `touch` create a folder and an empty file. An empty file is not useless: it is a placeholder, a lock, a log waiting for its first line.

   `mkdir` 与 `touch` 分别创建目录和空文件。空文件并非无用：它是占位符、是锁、是等待第一行内容的日志。

2. `mv` is both "move" and "rename" — the same operation, because renaming *is* moving to a new name in the same directory. There is no separate `rename` command to memorise.

   `mv` 既是"移动"也是"重命名"——同一个操作，因为重命名*就是*在同一目录内移动到一个新名字。不需要另记 `rename` 命令。

3. `rm` removes files; `rm -r` removes folders **recursively**, that is, the folder and everything inside it. The `-r` is what makes the command dangerous.

   `rm` 删文件；`rm -r` 递归地删目录，即目录及其内部一切。让这条命令变得危险的正是 `-r`。

4. Running `ls` after every destructive command is the habit this exercise is really teaching. Looking twice costs three seconds; recovery costs your data.

   在每条破坏性命令之后运行 `ls`，才是这道题真正要教的习惯。多看一眼花三秒；恢复数据花掉的是你的数据。

> **COMMON MISTAKE** 常见错误
>
> `rm -r` with a typo, or `rm -r *` in the wrong directory. There is no recycle bin on the server. Before pressing Enter on any `rm -r`, run `ls` and read the name aloud.
>
> `rm -r` 后面打错字，或在错误目录下执行 `rm -r *`。服务器上没有回收站。在任何 `rm -r` 按下回车之前，先跑一次 `ls` 并把名字念出来。

1.3 `less` versus `cat`

`less` 与 `cat` 的区别

> **THE TASK** 题目要求
>
> - Use `less` and `cat` on `/home/students/dt2026/resource/demo.txt` and explain the difference in your report.
>
>   分别用 `less` 和 `cat` 查看 `/home/students/dt2026/resource/demo.txt`，并在报告中解释两者差别。

两条命令读的是同一个文件，区别在于"显示多少"以及"何时停下"。

▾ **reading a file two ways** 用两种方式读同一个文件

```
cat /home/students/dt2026/resource/demo.txt     # everything at once / 一次全给

less /home/students/dt2026/resource/demo.txt    # one screen at a time / 一屏一屏给
#   inside less / 在 less 里:
#   Space or f : forward one screen / 向下一屏
#   b          : back one screen / 向上一屏
#   /word      : search for "word" / 搜索 "word"
#   q          : quit / 退出
```

OUTPUT 输出

```
$ cat /home/students/dt2026/resource/demo.txt
Welcome to the Programming I course server.
This file is a demo file.
You can read it with cat or with less.
```

```
cat prints everything at once.
less shows one screen at a time and waits for you.
$ less /home/students/dt2026/resource/demo.txt
Welcome to the Programming I course server.
This file is a demo file.
You can read it with cat or with less.
cat prints everything at once.
less shows one screen at a time and waits for you.
(END)
```

Walkthrough 精解

1. `cat` prints the whole file and returns you to the prompt immediately. It is perfect for short files and for feeding a file into another command.

   `cat` 打印整个文件并立刻回到提示符。它非常适合短文件，也适合把文件内容喂给另一条命令。

2. `less` opens the file in a pager: it shows one screen and waits. It never loads the entire file into memory, which is why a 2 GB log file opens instantly in `less` and freezes your terminal in `cat`.

   `less` 在分页器中打开文件：显示一屏然后等待。它从不把整个文件载入内存，正因如此，2 GB 的日志在 `less` 里瞬间打开，而在 `cat` 里会卡死终端。

3. The rule of thumb: `cat` for files you can read in one breath, `less` for everything else. Real server logs belong to the second category.

   经验法则：一口气能读完的文件用 `cat`，其他一律用 `less`。真实的服务器日志属于第二类。

4. For your report, the sharpest sentence is: "`cat` prints and exits; `less` waits and navigates." Add that `less` supports searching with `/`.

   报告中最精炼的一句话是："`cat` 打印即退出，`less` 等待并支持导航。" 再补一句 `less` 可用 `/` 搜索。

> **REPORT TIP** 报告技巧
>
> A screenshot of `less` showing the `(END)` marker is better evidence than a screenshot of `cat`, because it proves you actually opened the pager and quit it with `q`.
>
> 截一张 `less` 显示 `(END)` 标记的图片，比截 `cat` 更有说服力，因为它证明你确实打开了分页器并用 `q` 退出。

1.4 Editing with nano

用 nano 编辑文件

THE TASK 题目要求

- Practice nano: create `notes.txt`, write three lines, save, exit, then `cat` it.

  练习 nano：创建 `notes.txt`，写三行字，保存、退出，再用 `cat` 查看。

nano 没有鼠标、没有菜单栏、没有状态栏。每个操作都是一次按键组合，而一开始只需要记住三个。

▾ **notes.txt** created with nano / 用 nano 创建

```
nano notes.txt          # open (or create) the file / 打开（或创建）文件

# type inside nano / 在 nano 中输入：
#   Week 1: I connected to the server with Termius.
#   Week 1: I learned pwd, ls, cd, mkdir, touch, cp, mv, rm.
#   Week 1: nano saves with Ctrl+O and exits with Ctrl+X.

#   Ctrl + O   then Enter   : save (Write Out) / 保存
#   Ctrl + X                : exit / 退出

cat notes.txt           # prove it was saved / 证明已保存
```

OUTPUT 输出

```
Week 1: I connected to the server with Termius.
Week 1: I learned pwd, ls, cd, mkdir, touch, cp, mv, rm.
Week 1: nano saves with Ctrl+O and exits with Ctrl+X.
```

Walkthrough 精解

1. `Ctrl + O` means "Write Out" — nano asks you to confirm the file name; pressing Enter accepts it. Forgetting the Enter leaves the file unsaved and your text at risk.

   `Ctrl + O` 意为 Write Out（写出）——nano 会请你确认文件名，按 Enter 接受。忘了按 Enter，文件就没保存，你输入的文字处于风险中。

2. `Ctrl + X` exits. If you have unsaved changes nano asks "Save modified buffer?" — press `Y` followed by Enter.

   `Ctrl + X` 退出。若有未保存的改动，nano 会问 "Save modified buffer?"——按 `Y` 再回车。

3. The bottom two lines of the nano screen are the whole manual. The caret `^` means Ctrl, so `^O` is `Ctrl+O`. Read those two lines before you read any tutorial.

nano 屏幕底部两行就是完整手册。脱字符 `^` 代表 Ctrl，所以 `^O` 就是 `Ctrl+O`。先看这两行，再看任何教程。

4. `cat notes.txt` right after saving is the habit: never trust an editor's claim, verify the file.
   保存后立刻 `cat notes.txt` 是必备习惯：永远不要相信编辑器的声明，去验证文件。

> **COMMON MISTAKE** 常见错误
>
> Exiting with `Ctrl + X` and answering "No" to the save prompt — then wondering why the file is empty. In nano, saving and exiting are two separate actions; do them in that order.
>
> 按 `Ctrl + X` 退出时对保存提示回答 "No"，然后疑惑文件为何是空的。在 nano 中，保存与退出是两个独立动作；必须按这个顺序做。

1.5 Transferring Files with SFTP

用 SFTP 传输文件

> **THE TASK** 题目要求
>
> - Upload a file to the server with Termius SFTP, then download it back.
>   用 Termius 的 SFTP 上传一个文件到服务器，再下载回来。

SFTP 跑在你已经建立的那条加密 SSH 连接上，因此无需额外配置。这道练习的关键在于往返：一个原样回来的文件，证明这条通道双向都通。

▾ **sftp round trip** SFTP 往返

```
# 1. make something worth sending / 先造一个值得发送的东西
echo "Hello from my laptop" > upload_test.txt
cat upload_test.txt

# 2. Termius: open the SFTP panel for the same host, drag upload_test.txt
#    into your home directory / 在 Termius 中打开同一主机的 SFTP 面板，
#    把 upload_test.txt 拖进主目录

# 3. confirm it arrived on the server / 在服务器上确认它已到达
ls -l upload_test.txt
cat upload_test.txt

# 4. drag it back from the SFTP panel to your laptop, rename it
#    downloaded_test.txt / 再从 SFTP 面板拖回本地，改名为 downloaded_test.txt
```

```
# 5. compare the two / 比较两者
diff upload_test.txt downloaded_test.txt && echo "IDENTICAL"
```

OUTPUT 输出

```
Hello from my laptop
-rw-r--r-- 1 dtX20260001 dtX20260001 22 Nov  2 09:15 upload_test.txt
Hello from my laptop
IDENTICAL
```

Walkthrough 精解

1. `echo "text" > file` is the fastest way to create a small file without opening an editor. The `>` redirects the output of `echo` into the file.

   `echo "text" > file` 是不打开编辑器创建小文件的最快方式。`>` 把 `echo` 的输出重定向进文件。

2. `ls -l` (long format) adds the size and the timestamp. Comparing the byte size on your laptop with the byte size on the server is the real proof of a complete transfer.

   `ls -l`（长格式）额外给出大小与时间戳。把本地的字节数与服务器上的字节数对比，才是"传输完整"的真正证据。

3. `diff a b && echo "IDENTICAL"` uses a shell idiom worth remembering: `&&` runs the second command only if the first one succeeded. `diff` exits successfully when there is no difference, so "IDENTICAL" appears only when the files match.

   `diff a b && echo "IDENTICAL"` 用了一个值得记住的 shell 写法：`&&` 只在第一条命令成功时才执行第二条。`diff` 在无差异时成功退出，所以只有两文件一致时才会出现 "IDENTICAL"。

4. Upload and download are the same operation seen from two ends. Once this works, submitting homework becomes a drag, not a problem.

   上传与下载是同一操作的两个视角。这一步通了之后，交作业就只是拖一下，不再是问题。

> **REPORT TIP 报告技巧**
>
> Show three screenshots: the SFTP panel before the transfer, the `ls -l` output on the server, and the `diff` result. Together they prove the round trip without a paragraph of prose.
>
> 放三张截图：传输前的 SFTP 面板、服务器上的 `ls -l` 输出、以及 `diff` 的结果。三者合起来就能证明往返成功，不需要一段文字说明。

Your First Python Programs / 第一个 Python 程序

> *Interactive mode is a conversation; script mode is a letter. Learn when to speak and when to write.*
>
> 交互模式是对话，脚本模式是书信。学会何时开口，何时落笔。

Chapter 2 is where Python itself appears. The exercises walk the full chain from the `>>>` prompt to a saved script, deliberately break something, and end with your first function. Keep every file — Chapter 17 will read them like a diary.

第 2 章 Python 正式登场。练习从 `>>>` 提示符一路走到保存好的脚本，故意弄坏一次程序，最后写下你的第一个函数。请保留这里的每个文件——第 17 章会把它们当作日记来读。

2.1 Interactive Mode and the Version

交互模式与版本号

THE TASK 题目要求

- Enter and exit interactive mode; print the Python version with `import sys; print(sys.version)`.

  进入并退出交互模式；用 `import sys; print(sys.version)` 打印 Python 版本。

```
python3                # enter interactive mode / 进入交互模式

>>> import sys
>>> print(sys.version)
3.10.12 (main, Nov 20 2023, 18:32:27) [GCC 11.4.0]
>>> exit()
```

OUTPUT 输出

```
$                # back at the shell prompt / 回到了 shell 提示符
```

Walkthrough 精解

1. Three ways to leave: `exit()`, `quit()`, or `Ctrl + D`. The parentheses matter — `exit` without them only prints a hint.

   离开有三种方式：`exit()`、`quit()` 或 `Ctrl + D`。括号很重要——不带括号的 `exit` 只会打印一句提示。

2. `sys.version` is a string, not a number, so it can be printed directly. The GCC part tells you which compiler built this Python — useful when a package refuses to install.

   `sys.version` 是字符串不是数字，因此可以直接打印。其中的 GCC 部分说明这个 Python 是用哪个编译器构建的——当某个包装不上时，这条信息很有用。

3. Interactive mode evaluates as you type: type `2 + 3` and press Enter, the answer `5` appears at once. Nothing is saved. That scratchpad quality is exactly why we leave it for real work.

   交互模式边输边算：输入 `2 + 3` 回车，`5` 立刻出现，但什么都不会保存。这种草稿纸属性正是真实工作要离开它的原因。

> **COMMON MISTAKE** 常见错误
>
> Typing shell commands such as `ls` or `nano` *inside* the `>>>` prompt. They are not Python; you will get a `SyntaxError`. Check the left edge of your screen: `>>>` means Python, `$` means the shell.
>
> 在 `>>>` 提示符里输入 `ls`、`nano` 这类 shell 命令。它们不是 Python，你会收到 `SyntaxError`。看一眼屏幕左缘：`>>>` 是 Python，`$` 是 shell。

2.2 `hello.py`

`hello.py`

> **THE TASK** 题目要求
>
> - Create `hello.py` on the server that prints two lines: a greeting and your name.
>
>   在服务器上创建 `hello.py`，打印两行：一句问候和你的名字。

```
print("Hello, Programming I!")
print("My name is Zhang San.")
```

OUTPUT 输出

```
$ python3 hello.py
Hello, Programming I!
My name is Zhang San.
```

Walkthrough 精解

1. `print()` is a function call: the name, then parentheses, then what to print in quotes. Two calls produce two lines because `print` ends each call with a newline.

`print()` 是一次函数调用：函数名、括号、引号内的内容。调用两次就有两行，因为 `print` 每次调用都会以换行结束。

2. Double quotes and single quotes are interchangeable in Python: `"Hi"` and `'Hi'` are the same string. Pick one style and stay consistent within a file.

   双引号与单引号在 Python 中等价：`"Hi"` 与 `'Hi'` 是同一个字符串。选定一种风格并在文件内保持一致。

3. The full ritual is worth automating into muscle memory: `nano hello.py` → write → `Ctrl+O`, Enter → `Ctrl+X` → `python3 hello.py`. This is the loop you will repeat hundreds of times.

   完整流程值得练成肌肉记忆：`nano hello.py` → 编写 → `Ctrl+O`、回车 → `Ctrl+X` → `python3 hello.py`。这个循环你将重复数百次。

> **REPORT TIP** 报告技巧
>
> Screenshot the *terminal*, not an editor window: the command line `python3 hello.py` must be visible, otherwise the grader cannot tell a running program from a typed document.
>
> 请截*终端*截图而不是编辑器窗口：必须能看到 `python3 hello.py` 这条命令，否则评分者无法区分"运行中的程序"与"敲出来的文档"。

2.3 Breaking It On Purpose

故意弄坏一次

> **THE TASK** 题目要求
>
> - On purpose: remove the indentation under an `if`, run it, and copy the traceback into your report with a one-sentence explanation.
>
>   故意去掉 `if` 下面的缩进并运行，把 traceback 抄进报告并用一句话解释。

```
age = 20

if age >= 18:
print("Adult")        # no indentation on purpose / 故意不缩进
```

OUTPUT 输出

```
$ python3 broken_indent.py
  File "broken_indent.py", line 4
    print("Adult")
```

```
        ^
IndentationError: expected an indented block after 'if' statement on line 3
```

Walkthrough 精解

1. Read the traceback bottom-up: the last line names the error (`IndentationError`) and states the expectation ("an indented block after 'if'").

   自下而上读 traceback：最后一行给出错误类型（`IndentationError`）和 Python 的期待（"if 之后需要缩进的块"）。

2. The caret `^` points at the exact character where Python gave up. Line 4 is also quoted so you can copy it.

   脱字符 `^` 指向 Python 放弃的确切位置。第 4 行的代码也被引用出来，方便你复制。

3. The one-sentence explanation your report needs: *"In Python, indentation is not style but structure — the body of an `if` must be indented, so removing the spaces changes the program's grammar, not just its looks."*

   报告所需的一句话解释：*"在 Python 中，缩进不是风格而是结构——`if` 的语句体必须缩进，去掉空格改变的是程序的语法，而不只是外观。"*

4. Fix it by indenting four spaces, re-run, and screenshot both the error and the fix. One exercise, two pieces of evidence.

   缩进四个空格修复，重新运行，并把报错与修复都截下来。一道题，两份证据。

2.4 `structure.py`: Your First Function

`structure.py`：第一个函数

THE TASK 题目要求

- Write `structure.py` with a function `greet(name)` returning a greeting, and call it for yourself.

  编写 `structure.py`，含函数 `greet(name)` 返回问候语，并为你自己调用一次。

```
def greet(name):
    """Return a greeting for the given name."""
    message = "Hello, " + name + "! Welcome to Programming I."
    return message

my_name = "Zhang San"
print(greet(my_name))
print(greet("Li Si"))
```

OUTPUT 输出

```
Hello, Zhang San! Welcome to Programming I.
Hello, Li Si! Welcome to Programming I.
```

Walkthrough 精解

1. Three parts make a function: `def` + name + parameters, an indented body, and usually a `return`. `def` means "define" — it *builds* the function without running it.

   函数由三部分构成：`def` + 名字 + 参数、缩进的函数体、通常还有一个 `return`。`def` 意为"定义"——它只是*建造*函数，并不运行。

2. `return message` hands the string back to whoever called the function; it does not print anything. That is why the call must sit inside `print()`.

   `return message` 把字符串交还给调用者，它自己什么都不打印。所以调用必须放在 `print()` 里。

3. Reuse is the point: `greet` was written once but serves any name. Change the greeting in one place and both calls update together.

   复用才是重点：`greet` 只写了一次，却能服务任何名字。改一处问候语，两次调用同时更新。

4. The docstring (`"""..."""` on the first line of the body) is a comment aimed at functions. Later, `help(greet)` will display it.

   函数体第一行的文档字符串（`"""..."""`）是写给函数的注释。以后 `help(greet)` 会显示它。

> **COMMON MISTAKE** 常见错误
>
> `print` inside the function *and* `return` — then calling `greet("Li Si")` outside `print()` prints the greeting but returns `None`. Decide: a function either computes-and-returns, or prints. Mixing both surprises your future self.
>
> 在函数里既 `print` 又 `return`——然后不加 `print()` 直接调用 `greet("Li Si")`，问候语打印出来了，返回值却是 `None`。想清楚：函数要么"计算并返回"，要么"负责打印"。两者混用会让未来的你吃惊。

2.5 Let an AI Explain an Error

让 AI 解释一条报错

> **THE TASK** 题目要求
>
> - Pick one error message from this chapter and ask an AI assistant to explain it; compare its explanation

> with the table above.
>
> 从本章挑一条报错信息，请 AI 助手解释，并把它的解释与上表对比。

这道题不需要写代码，需要的是一场对话和一次书面对比。下面以 2.5 节的 `TypeError` 为例做一遍。

▾ **the error you paste** 你要粘贴的报错

OUTPUT 输出

```
TypeError: can only concatenate str (not "int") to str
```

```
# The prompt that works well / 好用的提问方式
# "I am a first-year student learning Python.
#  I ran:  print("5" + 3)
#  and got: TypeError: can only concatenate str (not "int") to str
#  Explain in under 100 words: (1) what went wrong,
#  (2) two ways to fix it, (3) the general rule behind it."
```

Walkthrough 精解

1. A good prompt contains the *code*, the *exact error*, and a *question with a word limit*. Vague prompts earn vague answers.

   好的提问包含*代码*、*原样报错*和*带字数限制的问题*。模糊的提问换来模糊的回答。

2. A correct answer must mention that `"5"` is a string and `3` is an integer, and offer `print("5" + str(3))` or `print(int("5") + 3)` as fixes. If the AI says otherwise, it is wrong — that is your comparison test.

   正确的回答必须指出 `"5"` 是字符串而 `3` 是整数，并给出 `print("5" + str(3))` 或 `print(int("5") + 3)` 作为修复。如果 AI 说得不同，那就是它错了——这正是你的对比测试。

3. The general rule both the handbook table and a good AI answer should reach: Python never guesses types for you; convert explicitly with `str()` or `int()`.

   学习手册表格与优秀的 AI 回答应指向同一条规则：Python 不会替你猜类型，请用 `str()` 或 `int()` 显式转换。

4. In your report, quote the AI's words *and* the handbook's table side by side, then one line: "They agree / disagree on …". Evidence, not vibes.

报告中请把 AI 的原话与学习手册表格并排放置，再加一行："两者在……上一致/不一致"。要证据，不要感觉。

Variables and Data Types / 变量与数据类型

> *A variable is not a box; it is a label you tie to a value.*
>
> 变量不是盒子，而是你系在值上的标签。

Chapter 3's exercises look like trivia — `bool()`, `0.1 + 0.2`, number bases — but each one removes a mystery that would otherwise cost you hours in Chapter 6 and beyond.

第 3 章的练习看起来像冷知识——`bool()`、`0.1 + 0.2`、进制——但每一道都在拆除一个谜团；不拆掉，它们会在第 6 章之后耗费你数小时。

3.1 `variables.py`

`variables.py`

THE TASK 题目要求

- Write `variables.py` that stores your age, height and name, prints each value together with `type()`, then converts the age to a string and prints its type again.

  编写 `variables.py`，存储你的年龄、身高、姓名，逐一打印值和 `type()`，再把年龄转成字符串打印其类型。

```
name = "Zhang San"
age = 19
height = 1.75

print(name, type(name))
print(age, type(age))
print(height, type(height))

age_as_text = str(age)
print(age_as_text, type(age_as_text))
```

OUTPUT 输出

```
Zhang San <class 'str'>
19 <class 'int'>
1.75 <class 'float'>
19 <class 'str'>
```

Walkthrough 精解

1. `type(x)` returns an object of class `type`; `print` renders it as `<class 'int'>` and so on. Reading `type()` at every step is the cheapest debugger a beginner owns.

   `type(x)` 返回一个 `type` 类型的对象；`print` 把它渲染成 `<class 'int'>` 等。随时打印 `type()` 是初学者手里最便宜的调试器。

2. `str(age)` does not change `age`; it builds a *new* string and binds the label `age_as_text` to it. The original integer is untouched — this is what "values are immutable here" means.

   `str(age)` 并不会改变 `age`；它构造一个*新的*字符串，把标签 `age_as_text` 系上去。原来的整数毫发无损——这就是"值不可变"的含义。

3. The three primitive types here cover most beginner code: `str` for text, `int` for whole numbers, `float` for decimals. There is no separate "height" type — everything is one of a handful of classes.

   这里三种原始类型覆盖了大部分初学者代码：`str` 文本、`int` 整数、`float` 小数。没有专门的"身高类型"——一切不过是少数几个类。

> **COMMON MISTAKE** 常见错误
>
> Writing `str(age)` and then still using `age` where a string is needed. Conversion *returns* a new value; it does not edit the old one. Store it, or convert again at the point of use.
>
> 写了 `str(age)` 之后，在需要字符串的地方仍然用 `age`。转换是*返回*新值，不是修改旧值。要么存下来，要么在使用处再转一次。

3.2 Predicting `bool()`

预测 `bool()`

> **THE TASK** 题目要求
>
> - Predict, then verify: `bool(0)`, `bool("0")`, `bool(-1)`, `bool([])`, `bool("False")`.
>
>   先预测再验证：`bool(0)`、`bool("0")`、`bool(-1)`、`bool([])`、`bool("False")`。

```
print(bool(0))        # ?
print(bool("0"))      # ?
print(bool(-1))       # ?
```

```
print(bool([]))       # ?
print(bool("False"))  # ?
```

OUTPUT 输出

```
False
True
True
False
True
```

Walkthrough 精解

1. The rule: a handful of "empty" values are false — `0`, `0.0`, `""`, `[]`, `{}`, `None` — and *everything else* is true. That is the entire law.

   规则：少数"空"值为假——`0`、`0.0`、`""`、`[]`、`{}`、`None`——*其余一切*为真。法律全文就是这一句。

2. `bool("0")` is `True` because the string contains a character; the *content* `"0"` is irrelevant, only emptiness matters. Same for `bool("False")` — a non-empty string, hence true.

   `bool("0")` 为 `True`，因为字符串里有字符；内容是 `"0"` 无关紧要，只有"空"才重要。`bool("False")` 同理——非空字符串，为真。

3. `bool(-1)` is `True` because only *zero* is falsy among numbers; negative is still "something".

   `bool(-1)` 为 `True`，因为数字中只有*零*为假；负数依然是"有东西"。

4. Why care? Every `if` and `while` in Chapter 6–7 runs this conversion silently: `if my_list:` is Pythonic for "if the list is not empty". Predicting `bool()` is predicting your own conditions.

   为什么重要？第 6–7 章的每个 `if`、`while` 都在暗中做这个转换：`if my_list:` 就是"列表非空吗"的地道写法。能预测 `bool()`，就能预测自己的条件。

> **REPORT TIP 报告技巧**
>
> Show your predictions in a comment next to each line, mark the wrong ones, and add one sentence on which prediction surprised you most. Graders reward honest surprises.
>
> 在每行旁边的注释里写下你的预测，标错错的，再加一句"哪个预测最让我意外"。评分者奖励诚实的意外。

3.3 The `0.1 + 0.2` Mystery

`0.1 + 0.2` 之谜

> **THE TASK** 题目要求
>
> - Explain in one sentence why `0.1 + 0.2 == 0.3` is `False`, then fix it with `Decimal`.
>
> 用一句话解释 `0.1 + 0.2 == 0.3` 为何是 `False`，再用 `Decimal` 修复它。

```
print(0.1 + 0.2 == 0.3)          # False
print(0.1 + 0.2)                 # 0.30000000000000004
print(f"{0.1 + 0.2:.17f}")       # 0.30000000000000004 (17 digits / 17 位)

from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))   # True
print(Decimal("0.1") + Decimal("0.2"))                     # 0.3
```

OUTPUT 输出

```
False
0.30000000000000004
0.30000000000000004
True
0.3
```

Walkthrough 精解

1. The one sentence your report needs: *"Computers store numbers in binary, and 0.1 has no exact binary form — just as 1/3 has no exact decimal form — so the nearest representable values are added and the error shows in the 17th digit."*

   报告所需的那句话：*"计算机用二进制存数，而 0.1 没有精确的二进制形式——正如 1/3 没有精确的十进制形式——于是两个'最近的'可表示值相加，误差在第 17 位显形。"*

2. `Decimal` works because it stores digits in *decimal*, the way humans write them — but note the quotes: `Decimal("0.1")` is exact, while `Decimal(0.1)` inherits the float's error.

   `Decimal` 有效是因为它按*十进制*存数字——即人类书写的方式——但注意引号：`Decimal("0.1")` 是精确的，而 `Decimal(0.1)` 会继承浮点数的误差。

3. Rule of thumb: money, scores, and any value that must match exactly get `Decimal` (or integer cents). Science and graphics keep `float` — fast and good enough.

   经验法则：金额、成绩等必须精确的值用 `Decimal`（或以"分"为单位的整数）。科学与图形计算保留 `float`——快且足够好。

COMMON MISTAKE 常见错误

Writing `Decimal(0.1)`. Without the quotes you first create the broken float, then wrap it. The constructor must receive the digits as a *string*: `Decimal("0.1")`.

写成 `Decimal(0.1)`。不加引号，你先造出了那个不精确的浮点数，再把它包起来。构造器必须收到*字符串*形式的数字：`Decimal("0.1")`。

3.4 Bases by Hand, Then by Python

手算进制，再用 Python 验证

THE TASK 题目要求

- Convert `0b1010`, `0o177`, `0x1af` to decimal by hand, then check with Python.

  手工把 `0b1010`、`0o177`、`0x1af` 换算成十进制，再用 Python 验证。

```
print(0b1010)   # 10
print(0o177)    # 127
print(0x1af)    # 431

print(int("1010", 2))   # 10  (from binary text / 从二进制文本)
print(int("1af", 16))   # 431 (from hex text / 从十六进制文本)
```

OUTPUT 输出

```
10
127
431
10
431
```

Walkthrough 精解

1. The hand calculations. Binary `1010` = 1×8 + 0×4 + 1×2 + 0×1 = **10**. Octal `177` = 1×64 + 7×8 + 7×1 = **127**. Hex `1af` = 1×256 + 10×16 + 15×1 = **431** (remember `a`=10, `f`=15).

   手算过程。二进制 `1010` = 1×8 + 0×4 + 1×2 + 0×1 = **10**。八进制 `177` = 1×64 + 7×8 + 7×1 = **127**。

   十六进制 `1af` = 1×256 + 10×16 + 15×1 = **431**（记住 `a`=10、`f`=15）。

2. The prefixes are just spellings of one number: `0b` binary, `0o` octal, `0x` hex. `0b1010`, `0o12`, `0xa` and plain `10` are all the same value.

   前缀只是同一个数的不同拼法：`0b` 二进制、`0o` 八进制、`0x` 十六进制。`0b1010`、`0o12`、`0xa` 和 `10` 是同一个值。

3. `int(text, base)` is the general tool: it parses digits written as *text* in any base from 2 to 36. The prefixes only work inside source code.

   `int(text, base)` 是通用工具：把写成文本的任意进制数字解析成整数（2 到 36 进制）。前缀只适用于源代码内部。

4. Why do programmers care? File permissions on Linux are octal (`chmod 755`), memory addresses are hex (`0x7fff...`), and bit flags are binary. The bases are not trivia; they are the server's accent.

   为什么程序员在乎？Linux 文件权限是八进制（`chmod 755`），内存地址是十六进制（`0x7fff...`），位标志是二进制。进制不是冷知识，而是服务器的口音。

Operators and Expressions / 运算符与表达式

> *Seven arithmetic operators, one precedence table, two traps. That is all Chapter 4 asks you to master.*
>
> 七个算术运算符、一张优先级表、两个陷阱。第 4 章要求掌握的尽在于此。

The exercises of Chapter 4 are short, which is exactly why they make good exam questions later. Do them twice: once to get the answer, once to be able to explain the answer.

第 4 章的练习都很短——这恰恰是它们日后成为考题的原因。请做两遍：第一遍为了得出答案，第二遍为了能解释答案。

4.1 `operators.py`: All Seven

`operators.py`：七种算术运算

THE TASK 题目要求

- Write `operators.py` with `a = 10, b = 3`, printing the result of all seven arithmetic operators, then the comparisons `a > b` and `a == b`, then `x and y`, `x or y`, `not x` for `x = True, y = False`.

  编写 `operators.py`：令 `a = 10, b = 3`，打印七种算术运算结果，再打印 `a > b`、`a == b`，以及 `x = True, y = False` 时的 `x and y`、`x or y`、`not x`。

```
a = 10
b = 3

print(a + b)    # 13  addition / 加
print(a - b)    # 7   subtraction / 减
print(a * b)    # 30  multiplication / 乘
print(a / b)    # 3.3333333333333335  true division / 除（浮点）
print(a // b)   # 3   floor division / 整除（向下取整）
print(a % b)    # 1   remainder / 取余
print(a ** b)   # 1000  power / 幂

print(a > b)    # True
print(a == b)   # False

x = True
y = False
print(x and y)  # False
```

```
print(x or y)   # True
print(not x)    # False
```

OUTPUT 输出

```
13
7
30
3.3333333333333335
3
1
1000
True
False
False
True
False
```

Walkthrough 精解

1. The pair that matters most is `/` versus `//`. True division *always* returns a float — even `10 / 5` is `2.0` — while floor division keeps an integer and rounds *down* (`-7 // 2` is `-4`, not `-3`).

   最重要的一对是 `/` 与 `//`。真除法*总是*返回浮点数——连 `10 / 5` 也是 `2.0`——而整除保持整数并*向下*取整（`-7 // 2` 是 `-4` 而非 `-3`）。

2. `%` is the remainder, and it is the operator of everyday life: `n % 2` tells odd from even, `minutes % 60` strips whole hours. Chapter 7's loops will use it constantly.

   `%` 是余数，也是最日常的运算符：`n % 2` 区分奇偶，`minutes % 60` 去掉整小时。第 7 章的循环会大量使用它。

3. `and`/`or`/`not` return *one of the operands*, not merely a boolean — but with booleans the visible result is the same. The habit worth building is `if 0 < x < 10:` instead of `if x > 0 and x < 10:` — Python allows both.

   `and`/`or`/`not` 返回的是*操作数本身*，而不仅仅是布尔值——不过在布尔量上结果相同。值得养成的习惯是写 `if 0 < x < 10:` 而不是 `if x > 0 and x < 10:`——Python 两种都支持。

> **COMMON MISTAKE** 常见错误
>
> Reading `a ** b` as "multiply by b". It is exponentiation: 10 to the power 3. The `**` has the *highest* precedence of all seven, higher than the `-` in `-3 ** 2`, which is `-9` because the power binds first.
>
> 把 `a ** b` 误读成"乘以 b"。它是幂运算：10 的 3 次方。`**` 的优先级在七个运算符中*最高*，比 `-3 ** 2` 里的负号还高——结果是 `-9`，因为乘方先算。

4.2 Precedence by Hand

手算优先级

> **THE TASK** 题目要求
>
> - By hand, evaluate `2 + 3 * 4 ** 2 // 5 - 1`; then verify with Python.
>
> 手工计算 `2 + 3 * 4 ** 2 // 5 - 1`，再用 Python 验证。

```
print(2 + 3 * 4 ** 2 // 5 - 1)

# the same expression with explicit steps / 加上括号的等价表达
print((2 + ((3 * (4 ** 2)) // 5)) - 1)
```

OUTPUT 输出

```
10
10
```

Walkthrough 精解

1. The hand computation, one operator at a time, always in precedence order: `4 ** 2` = 16 → `3 * 16` = 48 → `48 // 5` = 9 → `2 + 9` = 11 → `11 - 1` = **10**.

   手算过程，一次一个运算符，永远按优先级：`4 ** 2` = 16 → `3 * 16` = 48 → `48 // 5` = 9 → `2 + 9` = 11 → `11 - 1` = **10**。

2. The order is: `**`, then `* // %` (left to right), then `+ -`. Multiplication-class operators have equal rank, so `3 * 16 // 5` runs strictly left to right.

   顺序是：`**`，然后 `* // %`（从左到右），最后 `+ -`。乘除类同级别，所以 `3 * 16 // 5` 严格从左到右执行。

3. The second `print` wraps every step in parentheses and gives the same 10 — the strongest form of documentation. When in doubt, add the parentheses; they cost nothing and prevent the classic "it worked on my paper" bug.

   第二个 `print` 给每一步都套上括号，结果同样是 10——这是最强的文档形式。拿不准时，加括号；括号没有成本，却能避免经典的"我纸上算的对啊"。

4. In your report, show the five steps as a numbered list and the single Python line as evidence. Two minutes of work, full marks of rigor.

   报告中把五步写成编号列表，再附一行 Python 验证。两分钟的工作，满分的严谨。

4.3 When `'10' < '2'` Is True

当 `'10' < '2'` 为真时

THE TASK 题目要求

- Explain why `'10' < '2'` is `True`, and write one line that compares them as numbers instead.

  解释为何 `'10' < '2'` 是 `True`，并写一行代码把它们按数字比较。

```
print("10" < "2")                  # True  (!)

print(int("10") < int("2"))        # False — compare as numbers / 按数字比较

print(sorted(["10", "2", "1"]))    # ['1', '10', '2']  dictionary order / 字典序
```

OUTPUT 输出

```
True
False
['1', '10', '2']
```

Walkthrough 精解

1. Strings are compared character by character, left to right. `'1'` and `'2'`: the character `'1'` comes before `'2'` in Unicode order, so the comparison stops there and returns `True`. The lengths never matter.

   字符串是逐字符、从左到右比较的。`'1'` 与 `'2'`：字符 `'1'` 的 Unicode 序号在 `'2'` 之前，比较到此结束，返回 `True`。长度无关紧要。

2. This is *dictionary order* — the same reason "banana" < "cherry" and why `sorted()` puts `'10'` between `'1'` and `'2'`. Dictionaries are wonderful for words, terrible for numbers.

   这就是*字典序*——"banana" < "cherry" 的原因，也是 `sorted()` 把 `'10'` 排在 `'1'` 和 `'2'` 之间的原因。字典序对词语是恩赐，对数字是灾难。

3. The fix is one line: convert with `int()` before comparing. `int("10") < int("2")` is `False`, as arithmetic demands.

   修复只需一行：比较前用 `int()` 转换。`int("10") < int("2")` 是 `False`，符合算术直觉。

4. Where does this bite in real code? Reading input: `input()` always returns a string, so user input must be converted before any numeric comparison. Chapter 6's grade system depends on it.

   真实代码里何时被咬？读输入时：`input()` 永远返回字符串，所以任何数值比较前都必须转换。第 6 章的成绩系统全靠这一点。

COMMON MISTAKE 常见错误

Writing `int(input())` when the user might type nothing. If the input can be empty, convert inside a `try` — Chapter 14 will make this a habit.

在用户可能什么都没输入的情况下写 `int(input())`。若输入可能为空，请在 `try` 里转换——第 14 章会把它变成习惯。

4.4 The Chained-Assignment Pitfall

链式赋值陷阱

THE TASK 题目要求

- Demonstrate the chained-assignment pitfall with two lists, then fix it.

  用两个列表演示链式赋值陷阱，然后修复它。

```
list1 = list2 = []          # one list, two labels / 一个列表，两个标签
list1.append(5)
print(list2)                # [5] — surprise! / 意外!

a = []                      # the fix: two separate lists / 修复：两个独立列表
b = []
a.append(5)
print(b)                    # []
```

OUTPUT 输出

```
[5]
[]
```

Walkthrough 精解

1. `list1 = list2 = []` does not create two lists. It creates *one* empty list and binds two labels to it. Assignment copies the *label*, never the *object*.

   `list1 = list2 = []` 并不会创建两个列表。它创建了*一个*空列表，并给这个对象系上两个标签。赋值复制的是*标签*，从不复制*对象*。

2. That is why appending to `list1` shows up in `list2`: there is only one list, viewed through two windows. The handbook calls these names *aliases*.

   这就是为什么往 `list1` 里追加的东西出现在 `list2` 里：只有一个列表，两个窗口在看。学习手册把这样的名字称为*别名*。

3. The fix creates two separate objects, one per assignment. An even shorter idiom is `a, b = [], []` — unpacking two fresh lists.

   修复方式是两次赋值各建一个对象。更简短的写法是 `a, b = [], []`——解包出两个全新列表。

4. Remember the boundary from Chapter 3: integers and strings are immutable, so this trap *cannot* happen with `x = y = 10` — you cannot edit the 10. Lists, dicts and sets are mutable, and that is where aliasing bites.

   回顾第 3 章的边界：整数与字符串不可变，所以 `x = y = 10` 不会有此陷阱——你无法修改那个 10。列表、字典、集合可变，别名咬人的正是它们。

Working with Strings / 字符串操作

> *Text is the most common data on Earth. Chapter 5 is where you learn to cut it, join it, and rebuild it.*
>
> 文本是世界上最多的数据。第 5 章教你切割它、拼接它、重建它。

Five exercises, and every one of them is a tool you will reuse in the final project — where an AI's answer arrives as a string that must be trimmed, split and stitched into a prompt.

五道题，每一道都是期末项目要复用的工具——届时 AI 的回答以字符串的形式到达，需要被修剪、切分并缝合进新的提示词。

5.1 `strings.py`

`strings.py`

THE TASK 题目要求

- Write `strings.py`: build `greeting = "Hello, Python!"`, print the first and last characters, the slices `[0:5]` and `[7:13]`, the stripped/upper/lower/title versions of `"  Hello World  "`, and `"Python is easy to learn".split()`.

  编写 `strings.py`：令 `greeting = "Hello, Python!"`，打印首末字符、切片 `[0:5]` 与 `[7:13]`、`"  Hello World  "` 的 strip/upper/lower/title 版本，以及 `"Python is easy to learn".split()`。

```
greeting = "Hello, Python!"

print(greeting[0])        # H   first / 首字符
print(greeting[-1])       # !   last / 末字符
print(greeting[0:5])      # Hello
print(greeting[7:13])     # Python

messy = "  Hello World  "
print(messy.strip())      # "Hello World"
print(messy.upper())      # "  HELLO WORLD  "
print(messy.lower())      # "  hello world  "
print(messy.title())      # "  Hello World  "

sentence = "Python is easy to learn"
print(sentence.split())   # ['Python', 'is', 'easy', 'to', 'learn']
```

OUTPUT 输出

```
H
!
Hello
Python
Hello World
  HELLO WORLD
  hello world
  Hello World
['Python', 'is', 'easy', 'to', 'learn']
```

Walkthrough 精解

1. Negative indexes count from the end: `[-1]` is the last character, `[-2]` the one before. No loop or length calculation needed.

   负索引从末尾倒数：`[-1]` 是最后一个字符，`[-2]` 是倒数第二个。不需要循环，也不需要计算长度。

2. Slices exclude the end: `[0:5]` takes positions 0,1,2,3,4 — five characters — and stops *before* position 5. The boundary is "up to but not including".

   切片不含终点：`[0:5]` 取 0,1,2,3,4 五个字符，在位置 5 *之前*停下。规则是"到但不包括"。

3. `strip()` removes whitespace at both ends only; the `upper()`/`lower()` outputs keep their spaces because these methods change *case*, never *layout*.

   `strip()` 只去掉两端的空白；`upper()`/`lower()` 的输出保留空格，因为这些方法改变的是*大小写*，从不改动*排版*。

4. `split()` with no argument splits on *any run of whitespace* and discards it — the single most used string operation in data cleaning.

   不带参数的 `split()` *按任意连续空白*切分并丢弃空白——这是数据清洗中使用率最高的字符串操作。

> **COMMON MISTAKE 常见错误**
>
> Writing `greeting[13]` and getting `IndexError`. The string has 14 characters, indexed 0 to 13 — but `[7:13]` and `[13]` are different requests: the slice stops before 13, the index *is* 13. Check with `len(greeting)`.
>
> 写 `greeting[13]` 得到 `IndexError`。字符串有 14 个字符，索引 0 到 13——但 `[7:13]` 与 `[13]` 是不同的请求：切片在 13 之前停下，而索引*就是* 13。用 `len(greeting)` 自查。

5.2 Joining Three Words Two Ways

两种方式拼接三个单词

THE TASK 题目要求

- Join `["Python", "is", "fun"]` back into one sentence, both with `join` and with `+`.

  用 `join` 和 `+` 两种方式把 `["Python", "is", "fun"]` 重新拼成一句话。

```
words = ["Python", "is", "fun"]

sentence = " ".join(words)
print(sentence)                      # Python is fun

by_hand = words[0] + " " + words[1] + " " + words[2]
print(by_hand)                       # Python is fun
print(sentence == by_hand)           # True
```

OUTPUT 输出

```
Python is fun
Python is fun
True
```

Walkthrough 精解

1. Read `join` backwards: `" ".join(words)` means "take the list `words` and glue its items together *with a space between them*". The string before the dot is the glue.

   把 `join` 倒过来读：`" ".join(words)` 意为"取列表 `words`，*用空格*把元素粘在一起"。点号前的字符串就是胶水。

2. The `+` version works for three words and dies for thirty: every item needs its own `+ " " +`, and you must handle the last item by hand. `join` scales to any length with one line.

   `+` 版本对付三个词还行，对付三十个词就死了：每一项都要 `+ " " +`，最后一项还得手工处理。`join` 一行通吃任意长度。

3. One trap: `join` only accepts strings. `", ".join([1, 2, 3])` raises `TypeError` — convert first with `str()` or a loop.

   一个陷阱：`join` 只接受字符串。`", ".join([1, 2, 3])` 会抛 `TypeError`——先用 `str()` 或循环转换。

4. Which to choose? When the pieces are already a list, `join`. When you are building a sentence from a few known values (Chapter 17 does this constantly with prompts), either is fine — but f-strings (Exercise 5.4) are better still.

怎么选？材料已经是列表时用 `join`。用少量已知值拼句子时（第 17 章反复如此），两者皆可——但 f-string（练习 5.4）更优。

5.3 Reversing Your Own Name
反转自己的名字

THE TASK 题目要求

- Reverse your own name with slicing, then print its length and its uppercase form.

  用切片反转自己的名字，再打印长度和大写形式。

```
name = "Zhang San"

reversed_name = name[::-1]
print(reversed_name)          # naS gnahZ
print(len(name))              # 9  (space counts / 空格也算)
print(name.upper())           # ZHANG SAN
```

OUTPUT 输出

```
naS gnahZ
9
ZHANG SAN
```

Walkthrough 精解

1. `[-1] is a slice with three parts — start, end, step — where start and end are omitted (meaning "the whole string") and the step is -1`, meaning "walk backwards". The result is the reversed copy.
2. The original `name` is untouched. Strings are immutable: every "change" is really a *new* string that the label now points to.

   原来的 `name` 毫发无损。字符串不可变：每次"修改"其实都是产生一个*新*字符串，再把标签指过去。

3. `len()` counts every character including the space — 9 here, not 8. When your slice boundaries look wrong, print `len()` first.

   `len()` 统计包括空格在内的每个字符——这里是 9，不是 8。当切片边界看起来不对时，先打印 `len()`。

4. A Chinese name such as `"张三"` reverses the same way — `name[

-1] gives "三张"` — because slicing walks *characters*, and Python treats every Chinese character as exactly one character.

5.4 From `+` to f-string

从 `+` 到 f-string

> **THE TASK** 题目要求
>
> - Convert the + version of a greeting to an f-string and confirm both print identically.
>
> 把 + 版问候语改写成 f-string，确认两者输出一致。

```
name = "Zhang San"
age = 19

old_way = "Hello, " + name + "! You are " + str(age) + " this year."
new_way = f"Hello, {name}! You are {age} this year."

print(old_way)
print(new_way)
print(old_way == new_way)     # True
```

OUTPUT 输出

```
Hello, Zhang San! You are 19 this year.
Hello, Zhang San! You are 19 this year.
True
```

Walkthrough 精解

1. An f-string is a normal string with an `f` before the opening quote. Inside `{}`, Python evaluates real expressions — variables, arithmetic, even function calls: `f"{age + 1}"` prints `20`.

   f-string 是开头多了个 `f` 的普通字符串。在 `{}` 里，Python 会求值真正的表达式——变量、算式、甚至函数调用：`f"{age + 1}"` 打印 `20`。

2. The decisive difference is the missing `str(age)`: f-strings convert automatically. No `+`, no quotes gymnastics, no `TypeError` when you forget.

   决定性差异是 `str(age)` 消失了：f-string 自动完成转换。没有 `+`，没有引号体操，忘了转换也不会 `TypeError`。

3. Formatting rides along: `f"{3.14159:.2f}"` prints `3.14`, and `f"{7:>5}"` right-aligns in five columns. This is the same power `format()` offered, with less typing.

   格式化是顺带的能力：`f"{3.14159:.2f}"` 打印 `3.14`，`f"{7:>5}"` 在五列中右对齐。这与 `format()` 同样强大，只是打字更少。

4. This book's own rule: from Chapter 6 onward, every report line, every prompt to an AI, every printed table uses f-strings. Learn them once, use them forever.

   本书的规则：从第 6 章起，所有报告行、所有给 AI 的提示词、所有打印的表格都用 f-string。学一次，用一生。

5.5 Split, Replace, Rejoin

拆分、替换、重组

THE TASK 题目要求

- Given `"apple,banana,cherry"`, split it, replace the commas with `" | "`, and join it back.

  给定 `"apple,banana,cherry"`，先拆分，再把逗号替换为 `" | "` 并重新拼接。

```
fruits = "apple,banana,cherry"

items = fruits.split(",")
print(items)                        # ['apple', 'banana', 'cherry']

marked = fruits.replace(",", " | ")
print(marked)                       # apple | banana | cherry

rebuilt = " | ".join(items)
print(rebuilt)                      # apple | banana | cherry
print(marked == rebuilt)            # True
```

OUTPUT 输出

```
['apple', 'banana', 'cherry']
apple | banana | cherry
apple | banana | cherry
True
```

Walkthrough 精解

1. There are two roads to the same result: `replace` operates on the string as a whole, while `split` + `join` goes through a list. Seeing both makes you choose deliberately instead of by accident.

   同一结果有两条路：`replace` 把字符串当整体操作，而 `split` + `join` 借道列表。两条都见过，你才能主动选择而不是碰巧写对。

2. `replace` changes *every* occurrence, not the first one, and returns a new string — the original is intact, as always with strings.

   `replace` 替换的是*所有*出现的逗号，不只是第一个，并返回新字符串——原字符串完好无损，字符串一向如此。

3. The split-replace-join trio is the everyday shape of data cleaning: a CSV line arrives, gets split into fields, cleaned, and rebuilt for a report. Exercise 13.5 will do exactly this to a real file.

   拆分-替换-重组三件套是数据清洗的日常形态：一行 CSV 到达，被切成字段、清洗，再重组进报告。练习 13.5 会对真实文件做完全一样的事。

4. In your report, show both roads and the `True` comparison — the exercise explicitly rewards noticing that two different algorithms converge on one answer.

   报告中请展示两条路和那个 `True`——这道题明确奖励"发现两种不同算法殊途同归"这件事。

PART II

Control Flow / 控制流

Part II adds the two verbs that turn a list of instructions into a program: deciding (Chapter 6) and repeating (Chapter 7). The exercises here are the classic first programs of every programmer's life — traffic lights, vending machines, guessing games — and every later chapter will quietly reuse them.

第二部分加入把"指令清单"变成"程序"的两个动词：判断（第 6 章）与重复（第 7 章）。这里的练习是每位程序员人生中的经典第一课——红绿灯、售货机、猜数字——而后面的每一章都会悄悄复用它们。

Making Decisions / 条件判断

> *An if-statement is a fork in the road. The elif chain decides which fork wins.*
>
> if 语句是道路的岔口，elif 链决定哪个岔口胜出。

Chapter 6 ends with three exercises. The first is pure syntax practice; the second is a thinking exercise disguised as a coding one; the third combines loops (Chapter 7) with decisions, and is worth doing twice.

第 6 章以三道练习收尾。第一道是纯语法练习；第二道是伪装成编程题的思维题；第三道把循环（第 7 章）与判断结合在一起，值得做两遍。

6.1 `traffic_light.py`

`traffic_light.py`

THE TASK 题目要求

- Write `traffic_light.py`: ask for a color and print STOP / SLOW DOWN / GO / INVALID COLOR. Test all four paths.

  编写 `traffic_light.py`：询问颜色并输出 STOP / SLOW DOWN / GO / INVALID COLOR，测试全部四种路径。

```
color = input("What color is the light? ")

if color == "red":
    print("STOP")
elif color == "yellow":
    print("SLOW DOWN")
elif color == "green":
    print("GO")
else:
    print("INVALID COLOR")
```

OUTPUT 输出

```
$ python3 traffic_light.py
What color is the light? yellow
SLOW DOWN
$ python3 traffic_light.py
What color is the light? purple
INVALID COLOR
```

Walkthrough 精解

1. `input()` always returns a string, so the comparisons compare strings with strings — `"red" == "red"`. No `int()` conversion is wanted here.

   `input()` 永远返回字符串，所以比较是字符串对字符串——`"red" == "red"`。这里不需要 `int()`。

2. The `elif` chain matters: with four separate `if`s, a "red" input would be checked against every condition. With `elif`, checking stops at the first match. Same output, better habit.

   `elif` 链很关键：如果写四个独立 `if`，"red" 也会被后面每个条件检查一遍；用 `elif`，命中即停。输出相同，习惯更优。

3. The `else` is the safety net. Users type `Red`, `RED`, 绿色, or nothing — every input you did not anticipate must land somewhere visible.

   `else` 是安全网。用户会输入 `Red`、`RED`、绿色，或者什么都不输入——所有你没预料到的输入都必须落在一个看得见的地方。

4. "Test all four paths" means four *runs*: red, yellow, green, and one nonsense color. Screenshot all four; that is the evidence the exercise asks for.

   "测试全部四种路径"意思是*运行四次*：红、黄、绿、加一个乱输入。四张截图就是本题要求的证据。

> **COMMON MISTAKE** 常见错误
>
> Writing `if color = "red":`. A single `=` is assignment; Python reports `SyntaxError` before the program even runs. Comparison is always `==`.
>
> 写成 `if color = "red":`。单个 `=` 是赋值，Python 会在程序运行前就报 `SyntaxError`。比较永远是 `==`。

6.2 The Ticket System, Rebuilt with Evidence

票价系统：带证据的重建

> **THE TASK** 题目要求
>
> - Build the ticket system above and answer the three "Think About It" questions in your report with evidence.

实现上面的票价系统，并在报告中用运行结果回答"想一想"中的三个问题。

```
age = 19
is_student = True
has_membership = True

if age < 5:
    print("Free ticket")
elif age < 18:
    print("Child ticket")
elif is_student:
    print("Student discount")
elif has_membership:
    print("Membership discount")
else:
    print("Regular ticket")
```

OUTPUT 输出

```
Student discount        # run 1: 19, student, member -> student wins / 学生折扣胜出
Student discount        # run 2: 19, student, no member / 仍是学生折扣
Student discount        # run 3: 19, student AND member / 仍是学生折扣
Membership discount     # run 4: 25, no student, member / 会员折扣
```

Walkthrough 精解

1. Question 1 (19, no membership): the first two conditions fail, `is_student` fires → `Student discount`. Evidence: run the script with `is_student = True, has_membership = False`.

   问题 1（19 岁，无会员）：前两个条件不成立，`is_student` 命中 → `Student discount`。证据：以 `is_student = True, has_membership = False` 运行。

2. Question 2 (19, with membership): the answer is *still* `Student discount`. The `elif` chain executes only the first True branch; the membership check below is never even looked at. This is the single most important sentence of Chapter 6.

   问题 2（19 岁，有会员）：答案*仍然是* `Student discount`。`elif` 链只执行第一个为真的分支；下方的会员检查根本不会被执行。这是第 6 章最重要的一句话。

3. Question 3 (both discounts): restructure. Either independent `if`s that accumulate: `discount = 0` … `if is_student: discount += 30` … `if has_membership: discount += 10`; or a combined branch `if is_student and has_membership:`. The elif chain *cannot* express "both".

问题 3（同时享受两种折扣）：需要重构。要么用相互独立的 `if` 累加：`discount = 0`……`if is_student: discount += 30`……`if has_membership: discount += 10`；要么合并条件 `if is_student and has_membership:`。elif 链*无法*表达"两者都要"。

4. In your report, print the variable values *before* the branch (`print(age, is_student, has_membership)`) so each screenshot is self-explanatory.
   报告中请在分支*之前*打印变量值（`print(age, is_student, has_membership)`），让每张截图自解释。

6.3 `grade_system.py`

`grade_system.py`

THE TASK 题目要求

- Write `grade_system.py` that loops until the user types `q`, and maps scores to grades A/B/C/D.
  编写 `grade_system.py`：循环直到输入 `q`，并把分数映射为 A/B/C/D。

```
while True:
    text = input("Enter a score (q to quit): ")
    if text == "q":
        print("Goodbye!")
        break

    score = int(text)

    if score >= 85:
        print("Grade A")
    elif score >= 70:
        print("Grade B")
    elif score >= 60:
        print("Grade C")
    else:
        print("Grade D")
```

OUTPUT 输出

```
Enter a score (q to quit): 92
Grade A
Enter a score (q to quit): 61
Grade C
Enter a score (q to quit): q
Goodbye!
```

Walkthrough 精解

1. `while True:` plus `break` is the standard "loop until the user says stop" pattern. The condition is checked at the top, but the *decision* to leave happens in the middle, after we have seen the input.

`while True:` 加 `break` 是"循环直到用户喊停"的标准模式。条件在顶部检查，但*离开*的决定发生在中间——看到输入之后。

2. Check for `q` *before* calling `int()`. Convert first and "q" becomes a crash.

   要在调用 `int()` *之前*检查 `q`。先转换，"q" 就变成一次崩溃。

3. The grade boundaries use a descending chain: `>= 85` first, then `>= 70`. Written in the other order, every score would be Grade C — the first true branch always wins.

   等级边界用降序链：先 `>= 85`，再 `>= 70`。反着写，所有分数都会变成 Grade C——第一个为真的分支永远获胜。

4. This program is the meeting point of Chapters 6 and 7: a loop (repetition) wrapping a decision chain (classification). Exercise 7.5 and Chapter 14 both extend it.

   这个程序是第 6、7 章的交汇点：循环（重复）包裹判断链（分类）。练习 7.5 与第 14 章都会扩展它。

Going in Loops / 循环结构

> *while repeats under a condition; for visits every item. Break, continue and else give you the steering wheel.*
>
> while 在条件下重复，for 逐项访问。break、continue 与 else 是你的方向盘。

Chapter 7 has five exercises, and one of them — the completed vending machine — is the handbook's first program with real input validation. Do it carefully: Chapter 17's prompt loops are built on exactly this skeleton.

第 7 章有五道题，其中"完整的售货机"是学习手册第一个带输入校验的程序。请认真做：第 17 章的提示词循环正是建立在这个骨架上的。

7.1 `morning_run.py` and `loop_practice.py`

`morning_run.py` 与 `loop_practice.py`

THE TASK 题目要求

- Write `morning_run.py` and `loop_practice.py` exactly as in this chapter, and explain the three `range()` usages in your report.

  按本章原样编写 `morning_run.py` 与 `loop_practice.py`，并在报告中解释 `range()` 的三种用法。

```
day = 1
while day <= 5:
    print("Day " + str(day) + ": 5 km done!")
    day = day + 1
```

OUTPUT 输出

```
Day 1: 5 km done!
Day 2: 5 km done!
Day 3: 5 km done!
Day 4: 5 km done!
Day 5: 5 km done!
```

▾ **loop_practice.py**

```
for num in range(5):          # one argument / 一个参数
    print(num)

for num in range(2, 5):       # two arguments / 两个参数
```

```
    print(num)

for num in range(0, 10, 2):   # three arguments / 三个参数
    print(num)
```

OUTPUT 输出

```
0
1
2
3
4
2
3
4
0
2
4
6
8
```

Walkthrough 精解

1. The three `range()` forms: `range(5)` counts 0,1,2,3,4 from zero; `range(2, 5)` counts 2,3,4 from a given start; `range(0, 10, 2)` counts 0,2,4,6,8 with a step. In all three the end is *excluded*.

   `range()` 的三种形态：`range(5)` 从 0 数到 4；`range(2, 5)` 从给定起点 2 数到 4；`range(0, 10, 2)` 以步长 2 数出 0,2,4,6,8。三种形态中终点都*不含*。

2. `morning_run.py` uses `while` because the number of laps could depend on something that changes — a `while` keeps running *as long as* its condition holds, whatever that condition is built from.

   `morning_run.py` 用 `while`，因为圈数可能取决于变化的东西——`while` 是"只要条件成立就一直跑"，条件本身可以任意构造。

3. `loop_practice.py` uses `for` because the sequence is known in advance. Rule of thumb: counting a known list → `for`; repeating until something happens → `while`.

   `loop_practice.py` 用 `for`，因为序列事先已知。经验法则：遍历已知的清单 → `for`；重复到某事发生 → `while`。

4. `day = day + 1` inside the `while` is not optional. Delete it and the condition `day <= 5` is true forever — the handbook's first infinite loop.

   `while` 里的 `day = day + 1` 不是可选项。删掉它，条件 `day <= 5` 永远为真——这就是学习手册里的第一个死循环。

COMMON MISTAKE 常见错误

Forgetting that `range(2, 5)` stops at 4. Half the "my loop prints one number too few" bugs of this course are exactly this boundary, and it will follow you into string slicing (Chapter 5) and file reading (Chapter 12).

忘了 `range(2, 5)` 停在 4。本课程一半的"我的循环少打印一个数"都源于这个边界，它还会跟着你走进字符串切片（第 5 章）和文件读取（第 12 章）。

7.2 The Completed Vending Machine

完整版自动售货机

THE TASK 题目要求

- Complete the vending machine with input validation and change.

  补全自动售货机的输入校验与找零。

```
coins = 0
while coins < 5:
    text = input("Insert coin (1-3): ")
    insert_coins = int(text)

    if 1 <= insert_coins <= 3:
        coins = coins + insert_coins
        print("Total inserted: " + str(coins))
    else:
        print("Invalid coin! Please insert 1, 2, or 3.")

print("Here's your drink!")
change = coins - 5
if change > 0:
    print("Here is your change: " + str(change))
```

OUTPUT 输出

```
Insert coin (1-3): 2
Total inserted: 2
Insert coin (1-3): 9
Invalid coin! Please insert 1, 2, or 3.
Insert coin (1-3): 3
Total inserted: 5
Here's your drink!
```

Walkthrough 精解

1. `1 <= insert_coins <= 3` is a *chained comparison* — Python evaluates it as `1 <= insert_coins and insert_coins <= 3`. It reads exactly like the sentence "between 1 and 3".

   `1 <= insert_coins <= 3` 是*链式比较*——Python 把它当作 `1 <= insert_coins and insert_coins <= 3` 求值。读起来就是"在 1 到 3 之间"。

2. Validation happens *before* the coins are added. Adding first and checking later means a rejected 9-coin insert would have already changed the total. Order of operations is the whole exercise.

   校验发生在累加*之前*。先加再查，被拒绝的 9 枚硬币就已经改变了总数。运算顺序就是这道题的全部。

3. The change is computed *after* the loop ends, using `coins - 5`. Because the loop may end with more than 5 coins (e.g. 2 + 3 = 5 exactly, or 3 + 3 = 6), the `if change > 0` guard keeps the machine silent when nothing is owed.

   找零在循环结束*之后*用 `coins - 5` 计算。循环结束时硬币可能恰好是 5（如 2+3），也可能超过（如 3+3=6），`if change > 0` 保证不欠钱时保持沉默。

4. Note the structure: condition-controlled loop (`while coins < 5`) + validation (`if`) + post-loop settlement. This same skeleton, with a different condition, drives the final project's prompt chain in Chapter 17.

   注意这个结构：条件控制循环（`while coins < 5`）+ 校验（`if`）+ 循环后结算。换一个条件，同样的骨架就驱动着第 17 章期末项目的提示词链。

> **COMMON MISTAKE** 常见错误
>
> Calling `int(input())` directly and letting the user's typo crash the program. Chapter 14 shows the professional fix — `try/except ValueError` — and Exercise 14.1 rebuilds this exact machine with it.
>
> 直接写 `int(input())`，用户的打字错误就会让程序崩溃。第 14 章会给出专业解法——`try/except ValueError`——练习 14.1 会用它重建这台售货机。

7.3 `loop_control.py`

`loop_control.py`

THE TASK 题目要求

- Write `loop_control.py` demonstrating `break`, `continue` and the loop `else` with your own data.

  编写 `loop_control.py`，用自己的数据演示 `break`、`continue` 与循环 `else`。

```
scores = [45, 82, 59, 91, 73, 38]

# break: stop at the first passing score / 在第一个及格分停下
for s in scores:
    if s >= 60:
        print("First pass found:", s)
        break
    print("Checking", s)

# continue: report failures only / 只报告不及格
for s in scores:
    if s >= 60:
        continue
    print("Failed:", s)

# else: runs only if no break happened / 未触发 break 才执行
for s in scores:
    if s < 0:
        print("Impossible score!")
        break
else:
    print("All scores are valid.")
```

OUTPUT 输出

```
Checking 45
Checking 82
First pass found: 82
Failed: 45
Failed: 59
Failed: 38
All scores are valid.
```

Walkthrough 精解

1. `break` leaves the entire loop at once. In the first block, 82 passes and the loop never reaches 59, 91, 73 or 38 — "Checking 82" is printed, then nothing more.

   `break` 立刻跳出整个循环。第一段中 82 及格，循环永远到不了 59、91、73、38——打印 "Checking 82" 之后戛然而止。

2. `continue` abandons only the current round. The second loop prints exactly the scores below 60, skipping the passing ones without stopping.

`continue` 只放弃本轮。第二个循环精确打印低于 60 的分数，跳过及格项但不停止。

3. The loop `else` is Python's unique twist: it runs only when the loop finished *without* `break`. The third loop is a search that failed to find anything bad — and the `else` announces that success.

   循环 `else` 是 Python 的独有设计：只有循环*未*触发 `break` 而正常结束时才执行。第三段是一次"没找到坏东西"的搜索，`else` 正是宣告这次成功。

4. The classic use of `for ... else` is searching: `break` when found, and the `else` prints "not found". Once you know the pattern, you will see it everywhere in real code.

   `for ... else` 的经典用法是搜索：找到就 `break`，`else` 打印"没找到"。知道这个模式后，你会在真实代码里到处看到它。

7.4 `nested_loops.py`: the 3×3 Table

`nested_loops.py`：3×3 乘法表

> **THE TASK** 题目要求
>
> - Write `nested_loops.py` printing a 3x3 multiplication table.
>
>   编写 `nested_loops.py`，打印 3x3 乘法表。

```
for row in range(1, 4):
    for col in range(1, 4):
        print(f"{row} x {col} = {row * col}")
    print("---")          # end of one row / 一行结束
```

OUTPUT 输出

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
---
```

Walkthrough 精解

1. The outer loop chooses the row; the inner loop runs *completely* for each single pass of the outer one. Total iterations: 3 × 3 = 9.

   外层循环选行；内层循环在外层的*每一轮*里完整跑一遍。总迭代次数：3 × 3 = 9。

2. The `---` print is indented at the outer level only, so it fires once per row, not once per cell. Indentation decides ownership — the whole of Python in one sentence.

   `---` 的 print 只缩进到外层，所以每行打印一次，而不是每格一次。缩进决定归属——整本 Python 学习手册浓缩成一句话。

3. The f-string `f"{row} x {col} = {row * col}"` evaluates `row * col` inline. No temporary variable, no `str()` conversion, no `+` chains.

   f-string `f"{row} x {col} = {row * col}"` 直接在字符串内求值 `row * col`。没有临时变量，没有 `str()` 转换，没有 `+` 链。

4. Nested loops power every table, grid and image you will ever process. The 9-cell version here scales to a 9×9 table by changing only `range(1, 4)` twice to `range(1, 10)`.

   嵌套循环驱动着你要处理的每张表格、网格与图像。这里的 9 格版本只需把两处 `range(1, 4)` 改成 `range(1, 10)`，就能变成九九乘法表。

7.5 Guessing Game with a Best-Record

带最佳纪录的猜数字

> **THE TASK** 题目要求
>
> - Extend the guessing game: track the best (smallest) number of attempts across multiple rounds.
>
>   扩展猜数字游戏：跨多轮记录最佳（最少）尝试次数。

```
import random

best = None                          # no round played yet / 还没玩过任何一轮

while True:
    secret = random.randint(1, 20)
    attempts = 0

    while True:
        guess = int(input("Guess (1-20): "))
        attempts = attempts + 1
        if guess < secret:
            print("Too low!")
        elif guess > secret:
            print("Too high!")
        else:
            print("Correct! Attempts:", attempts)
            break

    if best is None or attempts < best:
        best = attempts
```

```
        print("New best record!")
    print("Best so far:", best)

    if input("Play again? (y/n): ") == "n":
        break
```

OUTPUT 输出

```
Guess (1–20): 10
Too high!
Guess (1–20): 5
Too low!
Guess (1–20): 7
Correct! Attempts: 3
New best record!
Best so far: 3
Play again? (y/n): y
...
Best so far: 3
Play again? (y/n): n
```

Walkthrough 精解

1. Two nested `while True` loops: the inner one is one round (repeat until correct), the outer one is the match (repeat until the player quits). Naming the two loops in your report clarifies the structure instantly.

   两个嵌套的 `while True`：内层是一局（重复直到猜对），外层是整场（重复直到玩家退出）。在报告里给这两个循环命名，结构立刻清晰。

2. `best = None` is the Pythonic "no value yet". The check `best is None or attempts < best` handles the first round without a special case — `None` short-circuits the comparison.

   `best = None` 是地道的"还没有值"。`best is None or attempts < best` 无需特判就能处理第一轮——`None` 让比较短路。

3. `random.randint(1, 20)` includes *both* endpoints, unlike `range(1, 20)`. A new secret is drawn every round, so memorising yesterday's answer is useless.

   `random.randint(1, 20)` 包含*两个*端点，与 `range(1, 20)` 不同。每轮都重新抽取秘密数字，记住上一轮的答案毫无用处。

4. The record bookkeeping — compare, update, announce — is the same three lines you will use for the best score, the earliest timestamp, the cheapest price. Small pattern, endless reuse.

   记录簿三步——比较、更新、宣告——与你将来求最高分、最早时间戳、最低价格用的是同样的三行。小模式，无限复用。

PART III

Abstraction / 抽象

Part III teaches the two levels of reuse: functions (Chapter 8) reuse logic inside one file; modules (Chapter 9) reuse code across files. The exercises build a small toolbox — pizza pricing, shipping, BMI — that Chapter 17 will call from its own functions.

第三部分教授两个层次的复用：函数（第 8 章）在一个文件内复用逻辑；模块（第 9 章）跨文件复用代码。这里的练习搭建了一个小工具箱——披萨定价、运费、BMI——第 17 章会在自己的函数里调用它们。

Functions / 函数

> *Write it once, name it well, call it forever.*
>
> 写一次，起好名，调用一生。

Chapter 8's five exercises all follow the same rhythm: extract a calculation into a function, give it parameters, and let the caller decide the inputs. The last exercise asks a question that matters far beyond this course — who owns a variable?

第 8 章的五道题遵循同一个节奏：把一个计算提取成函数，给它参数，让调用者决定输入。

最后一题提出的问题远远超出本课程——变量究竟归谁所有？

8.1 The Two Sandwiches

两份三明治

THE TASK 题目要求

- Write `sandwich_function.py` and `flexible_sandwich.py` from this chapter and show both outputs in your report.

  按本章编写 `sandwich_function.py` 与 `flexible_sandwich.py`，在报告中展示两种输出。

```
def make_sandwich(filling):
    print("Making " + filling + " sandwich:")
    print("1. Take two slices of bread")
    print("2. Add " + filling)
    print("3. Close sandwich")

make_sandwich("ham")
make_sandwich("cheese")
```

OUTPUT 输出

```
Making ham sandwich:
1. Take two slices of bread
2. Add ham
3. Close sandwich
Making cheese sandwich:
1. Take two slices of bread
2. Add cheese
3. Close sandwich
```

▾ **flexible_sandwich.py**

```
def flexible_greet(name, greeting):
    print(greeting + ", " + name + "!")
```

```
flexible_greet("Charlie", "Good morning")
flexible_greet("Diana", "Ni hao")
flexible_greet(greeting="Yo", name="Eve")     # keyword form / 关键字形式
```

OUTPUT 输出

```
Good morning, Charlie!
Ni hao, Diana!
Yo, Eve!
```

Walkthrough 精解

1. The parameter `filling` is a placeholder; the arguments `"ham"` and `"cheese"` are the real values. Every call re-runs the body with fresh values — write once, run many.

   参数 `filling` 是占位符；实参 `"ham"`、`"cheese"` 是真实值。每次调用都用新值重跑函数体——写一次，运行多次。

2. The keyword call `flexible_greet(greeting="Yo", name="Eve")` proves order no longer matters when arguments are named. Use keywords whenever a call would otherwise be hard to read.

   关键字调用 `flexible_greet(greeting="Yo", name="Eve")` 证明：一旦指名，顺序就不再重要。凡是调用可读性差的地方，就用关键字。

3. Compare the two files honestly: before functions, three sandwiches meant three copy-pasted blocks; after, one definition and N one-line calls. The savings grow with every change you later make.

   诚实对比两个文件：没有函数时，三个三明治意味着三段复制粘贴；有了函数，一次定义加 N 次单行调用。以后每次修改，这笔节省都会增长。

4. In your report show both outputs side by side and add one sentence: "the same logic, parameterised differently". That sentence *is* the lesson of Chapter 8.

   报告中并排展示两个输出，再加一句："同一逻辑，不同的参数化方式"。这句话*就是*第 8 章的全部教义。

8.2 `circle_calculator.py`: Pricing a Pizza

`circle_calculator.py`：给披萨定价

THE TASK 题目要求

- Write `circle_calculator.py` with a `circle_area(radius)` function used to price a 30 cm

> pizza at 0.02 per square cm.
>
> 编写 `circle_calculator.py`：实现 `circle_area(radius)`，并按每平方厘米 0.02 为 30 厘米披萨定价。

```
import math

def circle_area(radius):
    """Return the area of a circle / 返回圆的面积"""
    return math.pi * radius ** 2

radius = 15                      # a 30 cm pizza = radius 15 / 30 厘米披萨半径 15
area = circle_area(radius)
price = area * 0.02

print(f"A {radius * 2} cm pizza has area {area:.1f} cm2")
print(f"Price: {price:.2f} yuan")
```

OUTPUT 输出

```
A 30 cm pizza has area 706.9 cm2
Price: 14.14 yuan
```

Walkthrough 精解

1. The trap hidden in "30 cm pizza": 30 is the *diameter*, and the formula needs the *radius* — half of it. Real exercises hide unit conversions in plain language; this is your first one.

   "30 厘米披萨"里藏着陷阱：30 是*直径*，而公式需要*半径*——它的一半。真实的练习会用日常语言隐藏单位换算；这是你的第一次。

2. `math.pi` beats typing `3.14` by four decimal places. Precision costs nothing and matters once prices multiply.

   `math.pi` 比手打 `3.14` 多了四位小数精度。精度没有成本，而一旦价格相乘它就开始重要。

3. `f"{price:.2f}"` formats money: two decimal places, rounded. Printing `14.138026334774665` yuan would be correct and unreadable.

   `f"{price:.2f}"` 是金额格式：两位小数，四舍五入。打印 `14.138026334774665` 元虽然正确，但没法看。

4. The function is reusable by design: price a 25 cm pizza by changing one number, price three sizes with a loop over radii.

   函数为复用而设计：改一个数字就能给 25 厘米披萨定价，加个循环就能给三种尺寸定价。

8.3 Shipping and Boxes

运费与纸箱

THE TASK 题目要求

- Write `shipping_calculator.py` and `box_calculator.py` and verify the cube case by hand.

  编写 `shipping_calculator.py` 与 `box_calculator.py`，并手工验算立方体情形。

```
def calculate_shipping(weight, base_fee=5.0):
    return weight * 1.2 + base_fee

print(calculate_shipping(3.5))        # default base fee / 默认基础费
print(calculate_shipping(2.0, 4.0))   # custom base fee / 自定义基础费
```

OUTPUT 输出

```
9.2
6.4
```

▾ **box_calculator.py**

```
def box_calculator(length, width, height):
    volume = length * width * height
    surface = 2 * (length*width + length*height + width*height)
    return volume, surface            # a tuple / 一个元组

vol, surf = box_calculator(2, 2, 2)   # unpack / 解包
print("Volume:", vol)
print("Surface:", surf)
```

OUTPUT 输出

```
Volume: 8
Surface: 24
```

Walkthrough 精解

1. Hand verification of the cube: volume = 2×2×2 = **8**. Surface = 2×(4 + 4 + 4) = 2×12 = **24**. The program agrees — that two-line check is the exercise's real deliverable.

   立方体手工验算：体积 = 2×2×2 = **8**；表面积 = 2×(4+4+4) = 2×12 = **24**。程序一致——这两行验算才是本题真正的交付物。

2. `return volume, surface` returns a tuple, and `vol, surf = ...` unpacks it in one line. Two results, one function, no global variables, no printing from inside the math.

   `return volume, surface` 返回一个元组，`vol, surf = ...` 一行解包。两个结果、一个函数，不用全局变量，也不在数学代码里打印。

3. The default parameter `base_fee=5.0` makes the common call short (`calculate_shipping(3.5)`) while keeping the special case available (`calculate_shipping(2.0, 4.0)`). Defaults are an API design decision, not a syntax trick.

   默认参数 `base_fee=5.0` 让常见调用很短（`calculate_shipping(3.5)`），同时保留特例入口（`calculate_shipping(2.0, 4.0)`）。默认参数是接口设计决策，不是语法把戏。

4. A convention worth adopting now: calculation functions *return*, driver code at the bottom *prints*. Six months from now you can import `box_calculator` into another file and it still works.

   现在就值得采用的约定：计算函数只*返回*，底部的驱动代码负责*打印*。六个月后你把 `box_calculator` 导入另一个文件，它依然可用。

> **COMMON MISTAKE** 常见错误
>
> Unpacking into the wrong number of names: `vol = box_calculator(2, 2, 2)` binds the *whole tuple* to `vol`, and `vol + 1` then fails. Count your returns and your receivers.
>
> 解包的名字数量不对：`vol = box_calculator(2, 2, 2)` 会把*整个元组*绑给 `vol`，接着 `vol + 1` 就会失败。数一数返回值，再数一数接收者。

8.4 BMI and the Round-Trip Test

BMI 与往返测试

> **THE TASK** 题目要求
>
> - Write `bmi_calculator.py` plus the height-recovery function, and prove correctness with a round-trip test.
>
>   编写 `bmi_calculator.py` 与反推身高的函数，并用往返测试证明正确性。

```
def calculate_bmi(weight, height):
    return weight / (height ** 2)

def recover_height(weight, target_bmi):
    return (weight / target_bmi) ** 0.5

bmi = calculate_bmi(70, 1.75)
print("BMI:", round(bmi, 2))                    # 22.86

h = recover_height(70, bmi)
print("Recovered height:", round(h, 3))         # 1.75 — the round trip / 往返
```

OUTPUT 输出

```
BMI: 22.86
Recovered height: 1.75
```

Walkthrough 精解

1. The algebra first, code second: from BMI = W/H² we get H = √(W/BMI), written `(weight / target_bmi) ** 0.5`. If you cannot write the formula on paper, no amount of Python will save you.

   先代数后代码：由 BMI = W/H² 得 H = √(W/BMI)，即 `(weight / target_bmi) ** 0.5`。如果纸上写不出公式，再多 Python 也救不了你。

2. `x ** 0.5` is the square root because x^½ = √x — and note the parentheses. As the handbook warns, `x**1/2` without them means `(x**1)/2`, which *halves* x instead.

   `x ** 0.5` 就是开平方，因为 x^½ = √x——注意括号。如学习手册所警告，不加括号的 `x**1/2` 意为 `(x**1)/2`，那是在把 x *减半*。

3. The round-trip test is the gold standard of verification: forward (70, 1.75) → 22.86, then backward (70, 22.86) → 1.75. Two functions that undo each other must return the original input; if they do not, one is wrong.

   往返测试是验证的黄金标准：正向 (70, 1.75) → 22.86，再反向 (70, 22.86) → 1.75。互相抵消的两个函数必须还原输入；做不到，必有一个是错的。

4. `round(value, 2)` is for *display*. Inside the calculation we keep the full precision and round only at print time — the same discipline as money in Exercise 8.2.

   `round(value, 2)` 是*显示*用的。计算内部保留完整精度，只在打印时取整——与练习 8.2 的金额纪律相同。

8.5 `scope_demo.py` and `global_demo.py`

`scope_demo.py` 与 `global_demo.py`

THE TASK 题目要求

- Write `scope_demo.py` and `global_demo.py`; explain in two sentences what each prints and why.

  编写 `scope_demo.py` 与 `global_demo.py`；各用两句话解释输出及其原因。

```
temperature = 25          # global / 全局
```

```
def adjust_temp():
    temperature = 18      # a NEW local variable / 新建了一个局部变量
    print("Inside function:", temperature)

adjust_temp()
print("Outside function:", temperature)
```

OUTPUT 输出

```
Inside function: 18
Outside function: 25
```

▾ **global_demo.py**

```
score = 0

def update_points():
    global score          # use the outside box / 使用外面的盒子
    score = score + 10

update_points()
update_points()
print("Final score:", score)
```

OUTPUT 输出

```
Final score: 20
```

Walkthrough 精解

1. Two sentences for `scope_demo.py`: *it prints 18 inside and 25 outside, because the assignment inside the function created a brand-new local variable that merely shadows the global one.* Reading a global needs no permission; assigning without `global` builds a local.

   `scope_demo.py` 的两句话：*函数内打印 18，函数外打印 25，因为函数内的赋值创建了一个全新的局部变量，只是恰好遮住了全局的同名变量。*读全局不需要许可；不加 `global` 的赋值造的是局部变量。

2. Two sentences for `global_demo.py`: *it prints 20, because `global score` tells Python that the name inside the function refers to the outer variable, so both calls add to the same box.* Without that line, `score = score + 10` would raise `UnboundLocalError`.

   `global_demo.py` 的两句话：*它打印 20，因为 `global score` 告诉 Python 函数内的这个名字指向外部变量，于是两次调用都往同一个盒子里加。*没有这一行，`score = score + 10` 会抛 `UnboundLocalError`。

3. Why does the handbook call `global` a last resort? Because a function that silently rewrites the outside world is invisible at its call site: `update_points()` looks like a harmless read. Parameters in, `return` out — every time.

学习手册为何把 `global` 视为最后手段？因为悄悄改写外部世界的函数在调用处是隐形的：`update_points()` 看起来像无害的读取。参数进、`return` 出——永远如此。

4. One legitimate exception you already met: *modifying* a mutable global (like `students.append(...)` in Chapter 11) needs no `global`, because assignment is what triggers the local-shadow rule.

   你已经见过一个合法例外：*修改*可变全局变量（如第 11 章的 `students.append(...)`）不需要 `global`，因为触发局部遮蔽规则的是赋值。

Modules and the Standard Library / 模块与标准库

> *A module is somebody else's tested code, one import away. The standard library ships with Python; pip delivers the rest of the world.*
>
> 模块是别人写好的、经过测试的代码，一条 import 之遥。标准库随 Python 出厂，pip 负责运送全世界。

Chapter 9's exercises build your first own module and then tour the standard library. The last exercise — exploring a module nobody taught you — is the most realistic one in the whole book: it is what every working programmer did last Tuesday.

第 9 章的练习先构建你自己的第一个模块，然后游览标准库。最后一题——探索一个没人教过的模块——是全书最贴近现实的一道：上周二，每一位在职程序员都在做这件事。

9.1 `kitchen.py`, `use_kitchen.py` and Your Own Module

`kitchen.py`、`use_kitchen.py` 与你自己的模块

THE TASK 题目要求

- Create `kitchen.py` and `use_kitchen.py`; then write your own `cooking_module.py` with a `prepare_dish(dish_name)` function and call it twice.

  创建 `kitchen.py` 与 `use_kitchen.py`；再自创 `cooking_module.py`，含 `prepare_dish(dish_name)` 函数并调用两次。

```
# kitchen.py
def microwave_beep():
    print("BEEP! BEEP! BEEP!")

def oven_timer(minutes):
    print("Oven set for " + str(minutes) + " minutes.")
```

▾ **use_kitchen.py**

```
# use_kitchen.py — must sit in the SAME folder / 必须与 kitchen.py 同目录
import kitchen

kitchen.microwave_beep()
kitchen.oven_timer(20)
```

OUTPUT 输出

```
BEEP! BEEP! BEEP!
Oven set for 20 minutes.
```

▾ **cooking_module.py**

```
# cooking_module.py
def prepare_dish(dish_name):
    print("Preparing " + dish_name + ":")
    print("  1. Wash the ingredients")
    print("  2. Cook with care")
    print("  3. Plate and serve " + dish_name + "!")
```

▾ **use_cooking.py**

```
# use_cooking.py
import cooking_module

cooking_module.prepare_dish("Kung Pao Chicken")
cooking_module.prepare_dish("Tomato and Egg")
```

OUTPUT 输出

```
Preparing Kung Pao Chicken:
  1. Wash the ingredients
  2. Cook with care
  3. Plate and serve Kung Pao Chicken!
Preparing Tomato and Egg:
  1. Wash the ingredients
  2. Cook with care
  3. Plate and serve Tomato and Egg!
```

Walkthrough 精解

1. A module is nothing mystical: any `.py` file is importable by its file name. `import kitchen` means "load `kitchen.py` and give me the toolbox named `kitchen`".

   模块毫无神秘之处：任何 `.py` 文件都能按文件名导入。`import kitchen` 意为"加载 `kitchen.py`，把名为 `kitchen` 的工具箱交给我"。

2. The dot is the address operator: `kitchen.microwave_beep()` navigates *into* the toolbox. Without the dot, Python would look for a function named `microwave_beep` in the *current* file and fail.

   点号是寻址运算符：`kitchen.microwave_beep()` 表示*进入*工具箱。没有点号，Python 会在*当前*文件里找 `microwave_beep`，然后失败。

3. Same folder is the whole trick. If `use_kitchen.py` and `kitchen.py` live in different directories, `import kitchen` fails with `ModuleNotFoundError` — the most common import error of Week 9.

   同一目录是全部诀窍。若 `use_kitchen.py` 与 `kitchen.py` 不在同一目录，`import kitchen` 会报 `ModuleNotFoundError`——第 9 周最常见的导入错误。

4. Your `cooking_module.py` proves the point of the chapter: you are no longer only a *user* of toolboxes, you are an *author* of one. Chapter 17 imports your own helper files exactly like this.

你的 `cooking_module.py` 证明了本章的主旨：你不再只是工具箱的*使用者*，你成了*作者*。第 17 章导入你自己的辅助文件用的正是这个方式。

9.2 `math_practice.py`

`math_practice.py`

THE TASK 题目要求

- Write `math_practice.py`: compute the hypotenuse of a 3-4 triangle, the area of a radius-5 circle with `math.pi`, and `math.pow(2, 3)`.

  编写 `math_practice.py`：计算 3-4 三角形斜边、半径 5 圆的面积（用 `math.pi`）以及 `math.pow(2, 3)`。

```
import math

a, b = 3, 4
c = math.sqrt(a**2 + b**2)
print("Hypotenuse:", c)                    # 5.0

area = math.pi * 5 ** 2
print("Circle area:", round(area, 2))      # 78.54

print("2^3 =", math.pow(2, 3))             # 8.0
```

OUTPUT 输出

```
Hypotenuse: 5.0
Circle area: 78.54
2^3 = 8.0
```

Walkthrough 精解

1. The Pythagorean theorem in one line: `math.sqrt(a**2 + b**2)`. The 3-4 triangle is the classic sanity check — the answer must be exactly 5, so if your code prints anything else, your formula is wrong.

   一行勾股定理：`math.sqrt(a**2 + b**2)`。3-4 三角形是经典校验——答案必须恰好是 5，否则公式一定错了。

2. `math.sqrt` is clearer than `** 0.5` here because the name says what it does. Both work; prefer the one the next reader understands at a glance.

   这里 `math.sqrt` 比 `** 0.5` 更清晰，因为名字说明了用途。两者都行；选下一个读者一眼就懂的那个。

3. `math.pow(2, 3)` returns `8.0` — a float — while `2 ** 3` returns `8`, an int. Same value, different type; when an API demands a float, `math.pow` already provides it.

   `math.pow(2, 3)` 返回 `8.0`（浮点），而 `2 ** 3` 返回 `8`（整数）。值相同类型不同；当接口要求浮点数时，`math.pow` 已经替你备好。

4. `math` is only a table of contents away: `dir(math)` lists every function it offers, `help(math.sqrt)` explains one. That two-command pair is how you will learn every future module.

   `math` 离你只有一张目录的距离：`dir(math)` 列出全部函数，`help(math.sqrt)` 解释其中一个。这两条命令就是你学习未来所有模块的方式。

9.3 `countdown.py` and the Lottery

`countdown.py` 与随机抽奖

THE TASK 题目要求

- Write `countdown.py` using `time.sleep`, then a random lottery picker using `randint` and `choice`.

  编写用 `time.sleep` 的 `countdown.py`，再用 `randint` 与 `choice` 写一个随机抽奖器。

```
import time

n = 5
while n > 0:
    print(n)
    time.sleep(1)      # pause one second / 暂停一秒
    n = n - 1
print("Lift off!")
```

▾ **lottery.py**

```
import random

students = ["Alice", "Bob", "Charlie", "Diana", "Eve"]

winner_number = random.randint(1, 5)
winner_name = random.choice(students)

print("Lucky number:", winner_number)
print("Lucky student:", winner_name)
```

OUTPUT 输出

```
Lucky number: 3
Lucky student: Charlie
```

Walkthrough 精解

1. `time.sleep(1)` pauses the program for one second. It is the simplest way to make a program feel alive — and in Chapter 16 it becomes the stand-in for "waiting for the network".

   `time.sleep(1)` 让程序暂停一秒。它是让程序"活起来"的最简单方式——到了第 16 章，它将扮演"等待网络"的替身。

2. The `while n > 0` loop with `n = n - 1` is a countdown by hand. Delete the update line and the countdown runs forever — the same infinite-loop lesson as Exercise 7.1, now with dramatic pauses.

   `while n > 0` 加 `n = n - 1` 是手工倒计时。删掉更新那行，倒计时永不结束——与练习 7.1 相同的死循环教训，这次带戏剧性停顿。

3. `randint(1, 5)` is *inclusive on both ends*; `random.choice(students)` picks one item from any list. Together they cover "random number" and "random item" — the two random things programs ever need.

   `randint(1, 5)` *两端都含*；`random.choice(students)` 从任意列表选一项。两者合起来覆盖"随机数"与"随机选物"——程序需要的随机只有这两种。

4. Run the lottery five times and the winners differ every time. That unpredictability is the feature — and also the reason you can never test lottery code by comparing exact outputs; test the *range* instead.

   运行五次抽奖，赢家每次不同。这种不可预测正是特性——也是你不能用精确输出测试抽奖代码的原因；应当测试*范围*。

9.4 `stdlib_practice.py`

`stdlib_practice.py`

THE TASK 题目要求

- Write `stdlib_practice.py` combining `datetime`, `random` and `os`: print the formatted current time, a random password, and create a folder if missing.

  编写 `stdlib_practice.py`：组合 `datetime`、`random` 与 `os`，打印格式化当前时间、生成随机密码、按需创建文件夹。

```
import datetime
import random
```

```
import os

# 1. formatted current time / 格式化当前时间
now = datetime.datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))

# 2. a random 8-character password / 8 位随机密码
alphabet = "abcdefghjkmnpqrstuvwxyz23456789"
password = ""
for i in range(8):
    password = password + random.choice(alphabet)
print("Password:", password)

# 3. create the folder only if missing / 文件夹不存在才创建
folder = "lab_notes"
if not os.path.exists(folder):
    os.makedirs(folder)
    print("Created:", folder)
else:
    print("Already exists:", folder)
```

OUTPUT 输出

```
2026-11-02 09:41:07
Password: k7m2xq4p
Created: lab_notes
```

Walkthrough 精解

1. `strftime` is "string format time": `%Y` year, `%m` month, `%d` day, `%H:%M:%S` clock. Any arrangement of those codes builds the timestamp your reports need.

   `strftime` 意为 string format time：`%Y` 年、`%m` 月、`%d` 日、`%H:%M:%S` 时刻。这些代码的任意组合即可生成报告所需的时间戳。

2. The password loop is a pattern, not just a password: start with an empty string, append `random.choice(...)` eight times. The same loop builds prompts (Chapter 17) and CSV lines (Chapter 12).

   密码循环是一个模式而不只是密码：从空字符串开始，`random.choice(...)` 追加八次。同样的循环将来用于拼提示词（第 17 章）与拼 CSV 行（第 12 章）。

3. `os.path.exists` + `os.makedirs` is the standard "create if missing" guard. Without the check, a second run crashes with `FileExistsError`; with it, the script is safely re-runnable.

   `os.path.exists` + `os.makedirs` 是标准的"不存在才创建"守卫。没有检查，第二次运行会因 `FileExistsError` 崩溃；有了它，脚本可以安全地重复运行。

4. Three imports, three unrelated superpowers, one file. This is the promise of the standard library: whatever the task, someone has already written the boring half.

三条 import，三种无关的超能力，一个文件。这就是标准库的承诺：无论什么任务，无聊的那一半早已有人写好。

9.5 Adopt a Stranger Module

领养一个陌生模块

> **THE TASK** 题目要求
>
> - Run `help('modules')` and pick one module never mentioned in this book; write five lines about what it offers.
>
>   运行 `help('modules')`，挑一个本书没提过的模块，用五行文字介绍它的功能。

```
python3

>>> help('modules')          # a long list scrolls by / 一长串列表滚过
>>> import zipfile           # picked: zipfile / 选中：zipfile
>>> dir(zipfile)
['ZipFile', 'ZipInfo', ...]
>>> help(zipfile.ZipFile.extractall)
```

OUTPUT 输出

```
My five lines about zipfile / 关于 zipfile 的五行介绍:
1. zipfile creates and reads ZIP archives, the format Windows and macOS both open
natively.
2. ZipFile('pack.zip', 'w') opens an archive for writing, like open() for a normal
file.
3. write('notes.txt') adds one file; write can be called repeatedly for a whole
folder of results.
4. extractall('out/') unpacks everything into a folder with one call.
5. I would use it to bundle a week's screenshots and .py files into one attachment
for the weekly report.
```

Walkthrough 精解

1. `help('modules')` prints every module your Python can see — hundreds. The point is not to read them all but to notice that the shelf is enormous.

   `help('modules')` 会打印你的 Python 能看到的全部模块——数百个。重点不是读完它们，而是注意到这个书架有多大。

2. `dir(module)` lists a module's contents; `help(module.thing)` explains one item. This three-step exploration — import, dir, help — works on every module you will ever meet, including ones written next year.

`dir(module)` 列出模块内容；`help(module.thing)` 解释其中一项。这三步——导入、dir、help——对你将来遇到的每个模块都有效，包括明年才诞生的模块。

3. Pick a module you can *use* this week, not the most impressive one. `zipfile` wins because it solves a real weekly-report problem in five lines.

   选一个本周就*用得上*的模块，而不是最炫的那个。`zipfile` 胜出是因为它用五行解决了周报的真实问题。

4. In your report, the five lines should each make a claim you could demonstrate: name the function, say what it returns, and give one scenario. Claims with scenarios survive grading; adjectives do not.

   报告中的五行，每行都应是一个可演示的主张：说出函数名、说明返回什么、给出一个场景。带场景的主张经得起评分；形容词经不起。

PART IV

Data Structures / 数据结构

Part IV gives your data a home: lists and tuples (Chapter 10) for ordered sequences, dictionaries and sets (Chapter 11) for labelled and unique data. The word-frequency analyzer at the end of Chapter 11 is the first program in the course that a biologist would recognise as real work.

第四部分为你的数据安家：列表与元组（第 10 章）存放有序序列，字典与集合（第 11 章）存放带标签与唯一的数据。第 11 章末尾的词频分析器，是本课程第一个会被生物学家认作"真实工作"的程序。

Lists and Tuples / 列表与元组

> *Lists bend, tuples do not. Knowing which one you need is the whole skill.*
>
> 列表可弯，元组不可折。分清何时用哪个，就是全部功力。

Chapter 10's five exercises cover creation, access, modification, and the tuple habits that will pay off in Chapter 8's multiple returns and Chapter 11's records.

第 10 章的五道题覆盖创建、访问、修改，以及那些将在第 8 章"多值返回"和第 11 章"结构化记录"中兑现的元组习惯。

10.1 Creating and Accessing

创建与访问

> **THE TASK** 题目要求
>
> - Write `list_demo.py`, `list_creation.py` and `list_access.py` covering all four creation methods and all slices shown above.
>
> 编写 `list_demo.py`、`list_creation.py`、`list_access.py`，覆盖四种创建方式与上文全部切片。

```
colors = ["red", "green", "blue"]       # 1. square brackets / 方括号
numbers = list((1, 2, 3))               # 2. list() constructor / 构造函数
squares = [x**2 for x in range(5)]      # 3. comprehension / 推导式
word = list("hello")                    # 4. from a string / 从字符串转换

print(colors)
print(numbers)
print(squares)
print(word)
```

OUTPUT 输出

```
['red', 'green', 'blue']
[1, 2, 3]
[0, 1, 4, 9, 16]
['h', 'e', 'l', 'l', 'o']
```

▾ **list_access.py**

```
games = ["Mario", "Zelda", "Pokémon"]
print(games[0])       # Mario
print(games[-1])      # Pokémon
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
print(months[1:3])    # ['Feb', 'Mar']
print(months[:2])     # ['Jan', 'Feb']
print(months[3:])     # ['Apr', 'May']
print(len(months))    # 5
```

OUTPUT 输出

```
Mario
Pokémon
['Feb', 'Mar']
['Jan', 'Feb']
['Apr', 'May']
5
```

Walkthrough 精解

1. The four creation methods answer four different situations: a literal list you type by hand, a list built from another sequence, a list computed by a rule, and a list exploded from text.

   四种创建方式对应四种场景：手打的字面列表、由其他序列构建的列表、按规则计算的列表、从文本拆出的列表。

2. The comprehension `[x**2 for x in range(5)]` reads as "for every x in 0..4, collect x squared". One line replaces a three-line loop — and Chapter 11 uses it to clean words.

   推导式 `[x**2 for x in range(5)]` 读作"对 0..4 中的每个 x，收集 x 的平方"。一行替代三行循环——第 11 章会用它清洗单词。

3. Access rules are *identical* to strings (Chapter 5): index from 0, negative from the end, slices exclude the stop, `len()` gives the count. Learn them once, use them on every sequence forever.

   访问规则与字符串（第 5 章）*完全一致*：索引从 0 起、负数从尾数、切片不含终点、`len()` 计数。学一次，终身用于一切序列。

4. Slicing never errors on a too-large end: `months[3:99]` returns `['Apr', 'May']`. Indexing does error: `months[99]` raises `IndexError`. Slice when unsure, index when sure.

   切片在终点过大时不会报错：`months[3:99]` 返回 `['Apr', 'May']`。索引则会报错：`months[99]` 抛 `IndexError`。不确定就切片，确定才索引。

> **COMMON MISTAKE 常见错误**
>
> `list("hello")` does not create `["hello"]` — it creates five one-letter strings. To wrap one string in a list, write `["hello"]`.
>
> `list("hello")` 不会得到 `["hello"]`——它得到五个单字符字符串。想把一个字符串装进列表

，请写 `["hello"]`。

10.2 Adding and Removing

添加与删除

THE TASK 题目要求

- Write `list_add.py` and `list_remove.py`, including the three insert positions and the duplicate-remove experiment.

  编写 `list_add.py` 与 `list_remove.py`，包含三种 insert 位置与重复值 remove 实验。

```
todo = ["wake up"]
todo.append("brush teeth")
todo.insert(1, "make bed")
print(todo)

todo.insert(0, "start")        # very beginning / 最前
print(todo)
todo.insert(len(todo), "end1") # very end (like append) / 最后（等同 append）
print(todo)
todo.insert(-1, "end2")        # BEFORE the current last / 在当前最后元素之前
print(todo)
```

OUTPUT 输出

```
['wake up', 'make bed', 'brush teeth']
['start', 'wake up', 'make bed', 'brush teeth']
['start', 'wake up', 'make bed', 'brush teeth', 'end1']
['start', 'wake up', 'make bed', 'brush teeth', 'end2', 'end1']
```

▾ **list_remove.py**

```
pets = ["dog", "bird", "dog", "cat", "dog"]
print(pets.remove("dog"))      # None — remove returns nothing / 无返回值
print(pets)                    # one 'dog' gone / 少了一只狗

print(pets.index("dog"))       # 1 — first remaining occurrence / 剩余第一个的位置
```

OUTPUT 输出

```
None
['bird', 'dog', 'cat', 'dog']
1
```

Walkthrough 精解

1. The three insert positions: `insert(0, x)` puts x first; `insert(len(todo), x)` appends x last — identical to `append(x)`; and `insert(-1, x)` slides x in *before* the current last item, so it lands second from the end. The negative index is the surprise worth a screenshot.

三个插入位置：`insert(0, x)` 插到最前；`insert(len(todo), x)` 加到最后——与 `append(x)` 相同；`insert(-1, x)` 插在当前最后元素*之前*，落在倒数第二。负索引是值得截图的意外。

2. `remove("dog")` deletes only the **first** match. The duplicate experiment shows the list still contains two more `"dog"` values afterwards — one call, one deletion.

   `remove("dog")` 只删第一个匹配。重复值实验显示删除后列表里还剩两只 `"dog"`——一次调用，一次删除。

3. To remove *every* occurrence you need a loop: `while "dog" in pets: pets.remove("dog")`. Note `while`, not `for` — the list shrinks while you walk it, and a `for` over a shrinking list skips items.

   要删除*所有*匹配需要循环：`while "dog" in pets: pets.remove("dog")`。注意用 `while` 而非 `for`——你边走边缩短列表，`for` 遍历正在缩短的列表会跳过元素。

4. `remove()` returns `None` — the printed `None` in the demo is not a bug but a reminder: methods that *modify* a list usually return nothing, while methods that *compute* (like `index`) return the answer.

   `remove()` 返回 `None`——演示里打印出的 `None` 不是 bug，而是提醒：*修改*列表的方法通常没有返回值，*计算*的方法（如 `index`）才返回答案。

10.3 `student_grades.py`

`student_grades.py`

> **THE TASK** 题目要求
>
> - Write `student_grades.py`: compute the class average, then rank students high-to-low by score.
>
>   编写 `student_grades.py`：计算班级平均分，再按分数从高到低排名。

```
students = ["Alice", "Bob", "Charlie", "Diana"]
scores = [88, 92, 75, 100]

# 1. class average / 班级平均分
total = 0
for s in scores:
    total = total + s
average = total / len(scores)
print("Average:", average)                # 88.75
```

```
# 2. rank high to low / 从高到低排名
combined = []
for i in range(len(students)):
    combined.append([scores[i], students[i]])
combined.sort(reverse=True)

rank = 1
for pair in combined:
    print(str(rank) + ". " + pair[1] + " — " + str(pair[0]))
    rank = rank + 1
```

OUTPUT 输出

```
Average: 88.75
1. Diana — 100
2. Bob — 92
3. Alice — 88
4. Charlie — 75
```

Walkthrough 精解

1. The average is the loop-sum pattern from Chapter 7 plus one division: accumulate `total` over the list, then divide by `len(scores)`. `sum(scores) / len(scores)` is the one-line equivalent you may also use.

   平均分就是第 7 章的循环求和模式加一次除法：对列表累加 `total`，再除以 `len(scores)`。`sum(scores) / len(scores)` 是等价的一行写法。

2. Ranking needs the two parallel lists glued together. `[scores[i], students[i]]` builds `[score, name]` pairs; putting the *score first* means plain `sort(reverse=True)` orders by score automatically.

   排名需要把两条平行列表粘在一起。`[scores[i], students[i]]` 造出 `[分数, 名字]` 对；把*分数放在第一位*，普通的 `sort(reverse=True)` 就自动按分数排序。

3. `reverse=True` flips the sort to descending. Without it, Charlie would top the table and Diana would be last — a passing program producing exactly the wrong answer.

   `reverse=True` 把排序翻转为降序。没有它，Charlie 登顶、Diana 垫底——一个能跑通却输出完全错误结果的程序。

4. Ties: with equal scores, `sort` compares the second element (the name) and orders alphabetically. If that is not what you want, Chapter 11's `key=` parameter gives full control.

   并列时：分数相同，`sort` 会比较第二个元素（名字）并按字母排序。若不合意，第 11 章的 `key=` 参数给你完全的控制权。

10.4 Tuples: Demo, Unpack, and the Single-Element Trap

元组：演示、解包与单元素陷阱

THE TASK 题目要求

- Write `tuple_demo.py`, `tuple_unpack.py` and `tuple_single.py`; document the `(42)` versus `(42,)` result.

  编写 `tuple_demo.py`、`tuple_unpack.py`、`tuple_single.py`；记录 `(42)` 与 `(42,)` 的结果差异。

```
point = (3, 4)
print(point[0])         # 3
print(len(point))       # 2

point[0] = 99           # TypeError on purpose / 故意报错
```

OUTPUT 输出

```
3
2
Traceback (most recent call last):
  File "tuple_demo.py", line 5, in <module>
    point[0] = 99
TypeError: 'tuple' object does not support item assignment
```

▾ **tuple_unpack.py**

```
point = (3, 4)
x, y = point
print("x is", x, "y is", y)
```

▾ **tuple_single.py**

```
not_a_tuple = (42)      # just the number 42 in parentheses / 只是带括号的 42
a_tuple = (42,)         # the comma makes the tuple / 逗号造出元组

print(type(not_a_tuple))   # <class 'int'>
print(type(a_tuple))       # <class 'tuple'>
```

OUTPUT 输出

```
x is 3 y is 4
<class 'int'>
<class 'tuple'>
```

Walkthrough 精解

1. The traceback in `tuple_demo.py` is the *point* of the exercise: tuples are immutable, and Python's error — "does not support item assignment" — is the official statement of that promise. Include it in your report.

`tuple_demo.py` 里的 traceback 正是本题的*主旨*：元组不可变，Python 的报错——"does not support item assignment"——就是这一承诺的官方声明。请把它写进报告。

2. Unpacking `x, y = point` requires the left side to match the tuple's length. `x, y, z = point` raises `ValueError: not enough values to unpack`.

   解包 `x, y = point` 要求左侧名字数量与元组长度一致。`x, y, z = point` 会抛 `ValueError: not enough values to unpack`。

3. The single-element trap: parentheses make calls and grouping, not tuples — the *comma* is what makes a tuple. `(42)` is the integer 42; `(42,)` is a one-element tuple. You will write this correctly about three times before it becomes instinct.

   单元素陷阱：括号用于调用和分组，而不是造元组——*逗号*才是造元组的符号。`(42)` 是整数 42；`(42,)` 是单元素元组。这个写法你要写对三次左右才会变成本能。

4. Why tuples exist at all: they promise "this will not change", so they can be dictionary keys (Chapter 11) and honest function returns (Chapter 8). Reach for a list when data must change, a tuple when it must not.

   元组存在的理由：它承诺"这不会再变"，因此可以做字典键（第 11 章）和诚实的函数返回值（第 8 章）。数据要变用列表，不许变用元组。

10.5 `tuple_return.py`

`tuple_return.py`

THE TASK 题目要求

- Write `tuple_return.py` with `circle_calc(radius)` returning area and circumference; print both via unpacking.

  编写 `tuple_return.py`：`circle_calc(radius)` 返回面积与周长，并用解包打印两者。

```
import math

def circle_calc(radius):
    area = math.pi * radius ** 2
    circumference = 2 * math.pi * radius
    return area, circumference          # a tuple / 一个元组

r = 5
area, circ = circle_calc(r)
print(f"Radius {r}: area = {area:.2f}, circumference = {circ:.2f}")
```

```
results = circle_calc(1)                # without unpacking / 不解包时
print("Raw return value:", results)     # (31.415..., 6.283...) — a tuple
```

OUTPUT 输出

```
Radius 5: area = 78.54, circumference = 31.42
Raw return value: (78.53981633974483, 31.41592653589793)
```

Walkthrough 精解

1. `return area, circumference` is really `return (area, circumference)` — Python quietly packs the values into a tuple. The unpacking `area, circ = ...` is the polite way to take them apart again.

   `return area, circumference` 其实就是 `return (area, circumference)`——Python 悄悄把值装进元组。解包 `area, circ = ...` 则是拆开它们得体的方式。

2. The "raw return value" print shows what really crosses the function boundary: one tuple, two numbers inside. Once you have seen this, `def`-return plurals stop being mysterious.

   "Raw return value" 展示了真正跨越函数边界的东西：一个元组，里面两个数字。见过这一次，函数的多值返回就不再神秘。

3. Order is a silent contract: `area, circ = ...` works only because the function returns area *first*. Swap the returns and every caller silently mislabels the results — a comment naming the order is cheap insurance.

   顺序是一份无声的契约：`area, circ = ...` 之所以正确，只因函数先返回面积。交换返回顺序，所有调用者都会默默把结果贴错标签——用注释标注顺序是廉价的保险。

4. This pattern — compute several related quantities, return them as a tuple — is exactly how Chapter 17's `buildQuestion` returns `[framework, image_prompt, video_prompts]`. You are rehearsing the final project's shape.

   这个模式——计算多个相关的量，以元组返回——正是第 17 章 `buildQuestion` 返回 `[框架, 图像提示词, 视频提示词列表]` 的方式。你正在为期末项目的形状做排练。

Dictionaries and Sets / 字典与集合

> *Lists answer "what is at position 3"; dictionaries answer "what is the entry for Zhang San".*
>
> 列表回答"位置 3 上是什么"，字典回答"张三那一项是什么"。

Chapter 11 has six exercises — the longest exercise set of the course — and they escalate from syntax to a genuine mini-project. Budget two sessions for this chapter.

第 11 章有六道题——全课程最长的一组练习——难度从语法一路升级到真正的小项目。请为本章安排两个学习时段。

11.1 Creating and Modifying Dictionaries

创建与修改字典

THE TASK 题目要求

- Write `dict_create.py` and `dict_modify.py` covering all three creation methods and all update/delete operations.

  编写 `dict_create.py` 与 `dict_modify.py`，覆盖三种创建方式与全部更新/删除操作。

```
# 1. literal braces / 字面大括号
contact = {"name": "Zhang San", "phone": "13800000000"}

# 2. dict() constructor / 构造函数
contact2 = dict(name="Li Si", phone="13900000000")

# 3. start empty and grow / 从空开始逐步生长
contact3 = {}
contact3["name"] = "Wang Wu"
contact3["phone"] = "13700000000"

print(contact)
print(contact2)
print(contact3)
```

OUTPUT 输出

```
{'name': 'Zhang San', 'phone': '13800000000'}
{'name': 'Li Si', 'phone': '13900000000'}
{'name': 'Wang Wu', 'phone': '13700000000'}
```

▾ **dict_modify.py**

```
contact = {"name": "Zhang San", "phone": "13800000000"}

contact["phone"] = "13611112222"      # update / 更新
contact["email"] = "zs@biuh.edu.cn"   # add a new key / 新增键
print(contact)

del contact["phone"]                  # delete by key / 按键删除
print(contact)

phone = contact.pop("email")          # remove AND get the value / 删除并取回值
print("Removed:", phone)
print(contact)

print("name" in contact)              # membership tests keys / 检查键是否存在
print(contact.get("phone", "no phone on file"))
```

OUTPUT 输出

```
{'name': 'Zhang San', 'phone': '13611112222', 'email': 'zs@biuh.edu.cn'}
{'name': 'Zhang San', 'email': 'zs@biuh.edu.cn'}
Removed: zs@biuh.edu.cn
{'name': 'Zhang San'}
True
no phone on file
```

Walkthrough 精解

1. Three creation methods, same object: braces for literals you can read at a glance, `dict()` when keys are simple names, empty-then-grow when the content is computed at run time — the style the voting program below uses.

   三种创建方式，同一类对象：字面大括号一眼可读；`dict()` 适合简单键名；先空后长适合运行时才知道内容的场景——下面投票程序用的就是这种。

2. Assignment does double duty: `contact["phone"] = ...` *updates* an existing key and *creates* a missing one. That is why dictionaries grow so naturally.

   赋值身兼两职：`contact["phone"] = ...` 既是*更新*已有键也是*创建*缺失键。字典因此能如此自然地生长。

3. `del` deletes without telling you what it removed; `pop` deletes *and* hands the value back, which is what you want when the value is worth keeping.

   `del` 删除且不告诉你删了什么；`pop` 删除*并*把值交还——当值还有用时选它。

4. `contact["phone"]` on a missing key crashes with `KeyError`; `contact.get("phone", default)` returns the fallback instead. In interactive programs, `get` is the polite version of reading.

   对缺失的键做 `contact["phone"]` 会以 `KeyError` 崩溃；`contact.get("phone", default)` 则返回兜底值。在交互程序里，`get` 是礼貌的读取方式。

COMMON MISTAKE 常见错误

Spelling the key differently in two places — `"Phone"` and `"phone"` are different keys. A dictionary never complains; it just silently has two entries where you meant one.

在两处把键拼得不一样——`"Phone"` 与 `"phone"` 是两个不同的键。字典从不抱怨；它只是安静地保留两个条目，而你以为只有一个。

11.2 `translator.py` and `movie_rating.py`

`translator.py` 与 `movie_rating.py`

THE TASK 题目要求

- Write `translator.py` and `movie_rating.py`; test both an existing word and a missing one.

  编写 `translator.py` 与 `movie_rating.py`；各测试一个存在的词和一个缺失的词。

```
translator = {
    "hello": "你好",
    "python": "蟒蛇 / Python 语言",
    "server": "服务器"
}

word = input("Translate which word? ")
translation = translator.get(word, "Translation not available")
print(translation)
```

OUTPUT 输出

```
Translate which word? hello
你好
Translate which word? cat
Translation not available
```

▾ **movie_rating.py**

```
movie_ratings = {
    "Interstellar": 9.4,
    "The Martian": 8.9,
    "Inception": 9.1
}

title = input("Which movie? ")
if title in movie_ratings:
    print(title, "scores", movie_ratings[title])
else:
    print(title, "is not in our database yet.")
```

OUTPUT 输出

```
Which movie? Interstellar
Interstellar scores 9.4
Which movie? Titanic
Titanic is not in our database yet.
```

Walkthrough 精解

1. Both programs are the same shape: a dictionary of facts, one user input, one lookup with a safe fallback. Recognising the shared shape is worth more than either program.

   两个程序形状相同：一个事实字典、一次用户输入、一次带回退的安全查找。认出共同的形状，比任何一个程序本身都更有价值。

2. `.get(key, default)` with a friendly message beats a crash. The missing-word test is not optional — the exercise explicitly asks for it, because unhandled misses are the number-one runtime failure of dictionary code.

   带友好提示的 `.get(key, default)` 优于崩溃。缺失词测试不是可选项——题目明确要求，因为未处理的缺失是字典代码的第一大运行时故障。

3. `if title in movie_ratings:` checks membership among the *keys*. This `in` also works on lists — but there it scans every item, while in a dictionary it is instant.

   `if title in movie_ratings:` 检查的是*键*的成员资格。`in` 也能用于列表——但列表要逐个扫描，字典则是瞬间完成。

4. Where you will meet this again: Chapter 17 stores every generated prompt under a key, and the whole program is one dictionary lookup after another.

   你将在何处重逢：第 17 章把每条生成的提示词存在键下，整个程序就是一次接一次的字典查找。

11.3 `set_dedup.py` and `set_operations.py`

`set_dedup.py` 与 `set_operations.py`

THE TASK 题目要求

- Write `set_dedup.py` and `set_operations.py`; include the unique-letters-in-your-name program.

  编写 `set_dedup.py` 与 `set_operations.py`；包含"名字中的不重复字母"程序。

```
votes = ["dog", "bird", "dog", "cat", "dog"]
unique = set(votes)
print(unique)                    # order not guaranteed / 顺序不保证

name = "Zhang San"
```

```
print(set(name))                    # unique letters, one of each / 每个字母只留一个
print(len(set(name)))               # how many distinct characters / 不同字符数
```

OUTPUT 输出

```
{'bird', 'cat', 'dog'}
{'a', 'h', 'n', 'S', 'Z', 'g', ' '}
7
```

▾ **set_operations.py**

```
python_class = {"Alice", "Bob", "Charlie", "Diana"}
band = {"Charlie", "Diana", "Eve"}

print(python_class & band)    # intersection / 交集
print(python_class | band)    # union / 并集
print(python_class - band)    # difference / 差集
```

OUTPUT 输出

```
{'Charlie', 'Diana'}
{'Alice', 'Bob', 'Charlie', 'Diana', 'Eve'}
{'Alice', 'Bob'}
```

Walkthrough 精解

1. `set(list)` removes duplicates in one call. The printed order may differ from yours — sets are *unordered* by definition, so never rely on their display order.

   `set(list)` 一次调用去重。打印顺序可能与你的不同——集合*无序*是其定义，永远不要依赖它的显示顺序。

2. The name program is the same idea on a string: `set("Zhang San")` treats the string as a sequence of characters and keeps one of each — including the space, which is why the count is 7, not 6.

   名字程序是同一思想用于字符串：`set("Zhang San")` 把字符串当作字符序列，每个字符保留一个——包括空格，所以计数是 7 而不是 6。

3. `&` intersection, `|` union, `-` difference: three operators that answer "who is in both", "who is in either", "who is only in the first". They read like the Venn diagrams they draw.

   `&` 交集、`|` 并集、`-` 差集：三个运算符分别回答"谁在两者都有""谁在任一出现""谁只在第一个里"。它们读起来就像它们画出的维恩图。

4. When to choose a set over a list? Whenever the question is "is it there?" or "how many kinds?" and duplicates carry no meaning. Lists keep order and repeats; sets keep neither — and answer existence questions instantly.

   何时选集合而非列表？当问题只是"它在吗"或"有几种"且重复无意义时。列表保留顺序与重复；集合两者都不保留——但存在性问题的回答是瞬时的。

11.4 `set_voting.py`

`set_voting.py`

THE TASK 题目要求

- Write `set_voting.py` combining unique candidates with per-candidate vote counts.

  编写 `set_voting.py`，组合唯一候选人名单与每人票数。

```
votes = ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'Alice']

# set answers "who ran?" / 集合回答"有谁参选？"
candidates = set(votes)
print("Candidates:", candidates)

# dictionary answers "by how much?" / 字典回答"各得多少票？"
vote_counts = {}
for candidate in votes:
    if candidate in vote_counts:
        vote_counts[candidate] += 1
    else:
        vote_counts[candidate] = 1

print("Results:", vote_counts)

winner = max(vote_counts, key=vote_counts.get)
print("Winner:", winner)
```

OUTPUT 输出

```
Candidates: {'Alice', 'Bob', 'Charlie'}
Results: {'Alice': 3, 'Bob': 2, 'Charlie': 1}
Winner: Alice
```

Walkthrough 精解

1. The counting loop is the classic two-branch pattern: if the key exists, add one; otherwise create it with value 1. This exact loop reappears — unchanged — in the word-frequency analyzer of Exercise 11.6.

   计数循环是经典的双分支模式：键存在则加一，不存在则以 1 创建。这个循环会原封不动地重现于练习 11.6 的词频分析器。

2. `vote_counts[candidate] += 1` is shorthand for `vote_counts[candidate] = vote_counts[candidate] + 1`. The long form is clearer while the pattern is new; switch when it feels natural.

   `vote_counts[candidate] += 1` 是 `vote_counts[candidate] = vote_counts[candidate] + 1` 的简写。模式尚新时用长式更清晰；顺手后再切换。

3. `set` and `dict` cooperate because they are the same machinery underneath: a set is a dictionary with only keys. "Who ran?" is one `set()`; "by how much?" is one loop.

   `set` 与 `dict` 能协作，因为底层是同一套机器：集合就是只有键的字典。"有谁参选？"一条 `set()`；"各得多少票？"一个循环。

4. `max(vote_counts, key=vote_counts.get)` asks `max` to compare candidates *by their vote counts*. It is your first look at the `key=` parameter that Exercise 11.5 uses for sorting.

   `max(vote_counts, key=vote_counts.get)` 让 `max` *按票数*比较候选人。这是你对 `key=` 参数的初见，练习 11.5 将用它来排序。

11.5 `student_system.py`

`student_system.py`

THE TASK 题目要求

- Write `student_system.py`: add two students, then sort them by age using `key=lambda s: s['age']`.

  编写 `student_system.py`：添加两名学生，再用 `key=lambda s: s['age']` 按年龄排序。

```
students = []

students.append({"name": "Zhang San", "age": 19, "major": "Bioinformatics"})
students.append({"name": "Li Si", "age": 18, "major": "Computer Science"})

students.sort(key=lambda s: s["age"])      # youngest first / 按年龄升序

for s in students:
    print(s["name"], "—", s["age"], "—", s["major"])
```

OUTPUT 输出

```
Li Si — 18 — Computer Science
Zhang San — 19 — Bioinformatics
```

Walkthrough 精解

1. Each student is a dictionary; the class is a list of dictionaries. This "list of records" shape is how every real dataset — spreadsheet, database table, JSON file — is stored in Python.

   每个学生是一个字典；整个班级是字典的列表。这种"记录列表"的形状，就是电子表格、数据库表、JSON 文件在 Python 中的样子。

2. `lambda s: s["age"]` is a tiny unnamed function: "given a record, hand back its age". `sort` calls it once per record and orders by the results. You never need to write `def` for it.

   `lambda s: s["age"]` 是一个微小的无名函数："给一条记录，还回它的年龄"。`sort` 对每条记录调用一次并按结果排序。你永远不需要为它写 `def`。

3. For descending order, add `reverse=True`. To sort by major then age, a tuple key does it: `key=lambda s: (s["major"], s["age"])`.

   降序加 `reverse=True`。想先按专业再按年龄，用元组键：`key=lambda s: (s["major"], s["age"])`。

4. Exercise 10.3 solved ranking by hand-crafting `[score, name]` pairs. Compare the two solutions: `key=` removes the packaging step entirely. When you meet `lambda` again in later courses, it will feel like an old acquaintance.

   练习 10.3 通过手工拼装 `[分数, 名字]` 对来排名。比较两种解法：`key=` 把打包步骤整个省掉了。日后课程再遇 `lambda`，它会像老熟人。

11.6 `word_frequency.py`

`word_frequency.py`

THE TASK 题目要求

- Write `word_frequency.py` printing the top-5 words of the sample text; then run it on a paragraph you write yourself.

  编写 `word_frequency.py` 打印样例文本的前五高频词；再对自写的一段话运行。

```
text = "apple banana orange apple pear orange apple ok yes yes ok no no want"
words = text.lower().split()

word_counts = {}
for word in words:
    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1

# [count, word] pairs, count first so sort() orders by it / [次数, 单词] 对，次数在前
count_list = []
for word, count in word_counts.items():
    count_list.append([count, word])
count_list.sort(reverse=True)

print("Top 5 words:", count_list[:5])
```

OUTPUT 输出

```
Top 5 words: [[3, 'apple'], [2, 'orange'], [2, 'yes'], [2, 'ok'], [2, 'no']]
```

Walkthrough 精解

1. Three design decisions make the algorithm trivial: lowercase everything before counting (`"Apple"` and `"apple"` become one word), count with the two-branch dictionary loop from Exercise 11.4, and store the *count first* in each pair so a plain `sort(reverse=True)` orders correctly.

   三个设计决定让算法变得平凡：计数前全部转小写（`"Apple"` 与 `"apple"` 合并），用练习 11.4 的双分支字典循环计数，每对中*次数放在第一位*，于是普通的 `sort(reverse=True)` 就排对了。

2. `count_list[:5]` slices the leaderboard — the same "up to but not including" slice from Chapter 5, now applied to a list of results.

   `count_list[:5]` 对榜单切片——还是第 5 章"到但不包括"的那个切片，这次用于结果列表。

3. Running it on your own paragraph is the real test: add punctuation and the words arrive with commas attached (`"want,"` ≠ `"want"`). Fixing that needs `strip(".,!?")` per word — your first taste of real data cleaning.

   用自己写的段落运行才是真正的考验：加上标点后，单词会带着逗号到达（`"want,"` ≠ `"want"`）。修复它需要对每个词 `strip(".,!?")`——这是你对真实数据清洗的初体验。

4. Scale this up — longer text, better cleaning, maybe counting characters in a DNA sequence instead of words in English — and you have the shape of every text-mining and bioinformatics script this course sequence leads to.

   把它放大——更长的文本、更好的清洗、也许把"英文单词"换成"DNA 序列中的字符"——你就得到了本课程序列通向的所有文本挖掘与生物信息学脚本的形状。

PART V

The Real World / 与真实世界对接

Part V is where the course leaves the sandbox: files that survive the program (Chapter 12), paths and encodings that survive the operating system (Chapter 13), errors that survive your optimism (Chapter 14), libraries that survive pip (Chapter 15), and tasks that survive waiting (Chapter 16). The exercises stop being toy problems here — most of them produce artifacts you can actually keep.

第五部分是课程走出沙盒的地方：能在程序结束后存活的文件（第 12 章），能在操作系统间存活的路径与编码（第 13 章），能在你的乐观中存活的错误（第 14 章），能通过 pip 到达的库（第 15 章），以及需要在等待中存活的任务（第 16 章）。从这里起，练习不再是玩具题——大多数都会产出你真正可以保留的成果。

Reading and Writing Files / 文件读写

> *A variable dies with the program; a file is forever — or at least until next week's report.*
>
> 变量随程序而死，文件是永恒的——至少能活到下周的周报。

Chapter 12's five exercises build a tiny data pipeline: create a data file, read it back, append to it over multiple runs, and copy bytes that no text editor should ever touch.

第 12 章的五道题搭建一条小型数据管道：创建数据文件、读回、跨多次运行追加、复制那些永远不该被文本编辑器碰的字节。

12.1 The Class-List Greeter

班级名单问候器

> **THE TASK** 题目要求
>
> - Write `classlist.txt`, then a script that greets every student on it, one per line.
>
> 创建 `classlist.txt`，再写脚本逐行问候名单上的每位学生。

OUTPUT 输出

```
Zhang San
Li Si
Wang Wu
```

▾ **greet_class.py**

```
file = open("classlist.txt", "r")
for line in file:
    name = line.strip()
    print("Hello, " + name + "!")
file.close()
```

OUTPUT 输出

```
Hello, Zhang San!
Hello, Li Si!
Hello, Wang Wu!
```

Walkthrough 精解

1. A file object is *iterable*: `for line in file:` hands you one line at a time, each still carrying its newline character `\n` at the end.

文件对象是*可迭代的*：`for line in file:` 每次交给你一行，每行末尾仍带着换行符 `\n`。

2. `line.strip()` removes that trailing newline (and any stray spaces). Without it, every greeting ends with an invisible line break — the classic "my output has blank lines everywhere" bug.

   `line.strip()` 去掉行尾的换行符（以及杂散空格）。没有它，每句问候都以一个看不见的换行结束——经典的"输出里到处是空行"问题。

3. `file.close()` matters: an open file may buffer data that never reaches the disk. Exercise 12.4 replaces this chore with the `with` statement.

   `file.close()` 很重要：打开的文件可能缓冲着永远到不了磁盘的数据。练习 12.4 会用 `with` 语句取代这份苦差。

4. The pattern "data file + reading script" is the smallest possible version of every data project: the data outlives the program, the program can be re-run on new data without a single edit.

   "数据文件 + 读取脚本"的模式是所有数据项目最小化的形态：数据比程序活得久，程序换一份新数据无需改一行就能重跑。

12.2 `write()` versus `writelines()`

`write()` 与 `writelines()`

> **THE TASK** 题目要求
>
> - Create `groceries.txt` with `write()` and `shopping.txt` with `writelines()`; print both to verify.
>
>   分别用 `write()` 和 `writelines()` 创建 `groceries.txt` 与 `shopping.txt`，打印验证。

```
# write(): one string at a time / 一次一个字符串
file = open("groceries.txt", "w")
file.write("apple\n")
file.write("milk\n")
file.write("bread\n")
file.close()

# writelines(): a list of strings, all at once / 一次接收字符串列表
items = ["apple\n", "milk\n", "bread\n"]
file = open("shopping.txt", "w")
file.writelines(items)
file.close()

for name in ["groceries.txt", "shopping.txt"]:
    print("--- " + name)
    print(open(name).read())
```

OUTPUT 输出

```
--- groceries.txt
apple
milk
bread
--- shopping.txt
apple
milk
bread
```

Walkthrough 精解

1. `writelines()` does *not* add newlines. Its name promises more than it delivers: it just writes each string in turn. Every `\n` in the demo is there because *you* put it in the list.

   `writelines()` *不会*添加换行。它的名字承诺得多，兑现得少：它只是依次写入每个字符串。

   演示里每个 `\n` 都是因为*你*把它放进了列表。

2. `"w"` mode truncates: the moment `open(name, "w")` runs, the old file is empty, before a single character is written. Running this script twice produces the same two files, not four sections.

   `"w"` 模式是截断：`open(name, "w")` 执行的那一刻，旧文件已被清空——一个字符都还没写。

   这个脚本跑两遍得到的是同样的两个文件，而不是四段内容。

3. `open(name).read()` without a variable is a shortcut for small demos: the file object is created, read, and abandoned in one expression. In real code, prefer `with`.

   不用变量直接 `open(name).read()` 是小演示的捷径：文件对象在一个表达式里创建、读取、抛弃。真实代码请用 `with`。

4. Choose by shape: many separate values → loop with `write()`; already a list of ready strings → `writelines()`. Both roads end at the same file.

   按材料选工具：多个分散的值 → 循环 `write()`；已经是现成字符串列表 → `writelines()`。

   两条路通向同一个文件。

12.3 The Diary Across Two Runs

跨两次运行的日记

THE TASK 题目要求

- Append two dated entries to `diary.txt` across two runs of the program, and show the file keeps all entries.

  分两次运行程序向 `diary.txt` 追加两条带日期的记录，证明文件保留了全部记录。

```
import datetime

entry = input("What happened today? ")
stamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")

file = open("diary.txt", "a")       # 'a' keeps the past / 'a' 保留过去
file.write("[" + stamp + "] " + entry + "\n")
file.close()

print("Saved. The diary now reads:")
print(open("diary.txt").read())
```

OUTPUT 输出

```
What happened today? Finished Chapter 12 exercises.
Saved. The diary now reads:
[2026-11-02 10:05] Finished Chapter 12 exercises.

What happened today? Started the final project proposal.
Saved. The diary now reads:
[2026-11-02 10:05] Finished Chapter 12 exercises.
[2026-11-02 15:40] Started the final project proposal.
```

Walkthrough 精解

1. The only difference from Exercise 12.2 is one letter: mode `"a"` (append) writes at the *end* of an existing file and creates it if missing. `"w"` would erase yesterday's entry the moment the program starts.

   与练习 12.2 的唯一差别是一个字母：模式 `"a"`（追加）写在已有文件的*末尾*，文件不存在则创建。`"w"` 会在程序启动的一瞬间抹掉昨天的记录。

2. `strftime("%Y-%m-%d %H:%M")` stamps each entry. Sorting by this format is also chronological, because the fields run from largest to smallest — a trick worth remembering when naming anything by date.

   `strftime("%Y-%m-%d %H:%M")` 给每条记录打时间戳。按这种格式排序恰好也是按时间排序，因为字段从大到小排列——给任何文件按日期命名时，这个技巧都值得记住。

3. Run the program twice — that is the exercise. The second run must *add* to the file, and printing the whole file afterwards is the evidence.

   请运行程序两次——这就是本题。第二次运行必须*追加*，之后打印整个文件就是证据。

4. This append-verify loop is exactly how logs work on real servers, and how your Chapter 17 project will record every prompt it sends.

   这个"追加-验证"的循环正是真实服务器上日志的工作方式，也是第 17 章项目记录每条提示词的方式。

COMMON MISTAKE 常见错误

Using `"w"` for a diary "because it also creates the file". It does — by destroying it first. `"a"` creates *and* preserves.

给日记用 `"w"` 模式"因为它也能创建文件"。它能——先摧毁再创建。`"a"` 既创建又保留。

12.4 Rewriting with `with`

用 `with` 重写

THE TASK 题目要求

- Rewrite your class-list greeter with the `with` statement and explain in one sentence what changed.

  用 `with` 重写班级问候脚本，并用一句话说明改动了什么。

```
with open("classlist.txt", "r") as file:
    for line in file:
        name = line.strip()
        print("Hello, " + name + "!")

# no close() — and nothing leaks / 无需 close()——也不泄漏
```

OUTPUT 输出

```
Hello, Zhang San!
Hello, Li Si!
Hello, Wang Wu!
```

Walkthrough 精解

1. The one sentence your report needs: *"The `with` statement guarantees the file is closed as soon as the indented block ends — even if an error happens in the middle — so `file.close()` is no longer my responsibility."*

   报告所需的一句话：*"with 语句保证缩进块一结束文件就被关闭——即使中间出了错也一样——于是 file.close() 不再是我的责任。"*

2. Before `with`, an exception between `open` and `close` left the file open for the rest of the program's life. After `with`, cleanup is structural, not remembered.

   在 `with` 之前，`open` 与 `close` 之间一旦抛出异常，文件将在程序余下的生命里保持打开。有了 `with`，清理是结构性的，而不是靠记性。

3. Indentation now marks exactly how long the file stays open. Everything that needs the file lives inside the block; everything else lives after it.

   缩进现在精确标记文件保持打开的时长。需要文件的一切都住在块内，其余都住在块外。

4. The handbook's rule is absolute: *always* `with` for files, database connections and network sockets. Make it a reflex this week.

   学习手册的规则是绝对的：文件、数据库连接、网络套接字，*一律*用 `with`。让它在今周成为条件反射。

12.5 Copying a Binary File

复制二进制文件

> **THE TASK** 题目要求
>
> - Copy a binary file (any image) with the nested `with` pattern and compare file sizes.
>
>   用嵌套 with 模式复制一个二进制文件（任意图片），并对比文件大小。

```
with open("cat.jpg", "rb") as source_cat:       # r+b: read binary / 读二进制
    with open("cat_backup.jpg", "wb") as backup_cat:   # w+b: write binary / 写二进制
        for chunk in source_cat:
            backup_cat.write(chunk)

import os
print("Source :", os.path.getsize("cat.jpg"), "bytes")
print("Backup :", os.path.getsize("cat_backup.jpg"), "bytes")
```

OUTPUT 输出

```
Source : 48213 bytes
Backup : 48213 bytes
```

Walkthrough 精解

1. The `b` in `"rb"`/`"wb"` switches off all text handling: no newline translation, no decoding. The bytes cross untouched — which is the only way a JPEG survives a copy.

   `"rb"`/`"wb"` 里的 `b` 关闭了所有文本处理：不转换换行、不解码。字节原样通过——这是 JPEG 在复制中存活的唯一方式。

2. Iterating a binary file yields *chunks* (lines of bytes), not characters. `for chunk in source:` streams the file piece by piece, so even a 2 GB video never exhausts your memory.

   迭代二进制文件得到的是*数据块*（字节组成的行），不是字符。`for chunk in source:` 分块流式处理，2 GB 的视频也不会耗尽内存。

3. Nested `with` statements stack cleanly: the source closes when its block ends, the backup when its (inner) block ends. Order matters — you read from an open file and write to an open file the whole time.

   嵌套 `with` 干净地叠放：源文件在其块结束时关闭，备份在其（内层）块结束时关闭。顺序很重要——整个过程中你都在从一个打开的文件读、向另一个打开的文件写。

4. Equal byte counts are necessary but not sufficient proof of a copy. Open both images and look — your eyes are the checksum for this exercise.

   字节数相等是复制成功的必要条件而非充分条件。把两张图都打开看一眼——这道题里你的眼睛就是校验和。

File Systems and Encodings / 文件系统与字符编码

> *A file's name is an address; its bytes are a language. Chapter 13 teaches you to handle both.*
>
> 文件名是地址，字节是语言。第 13 章教你同时把握两者。

Chapter 13's exercises turn the `os` library from a list of functions into five working automations: a path inspector, a dated folder maker, a batch renamer, an encoding converter, and a CSV reader that survives Chinese text.

第 13 章的练习把 `os` 库从一串函数变成五个能用的自动化：路径检查器、按日建夹器、批量重命名器、编码转换器，以及一个能安然处理中文的 CSV 读取器。

13.1 `path_check.py`

`path_check.py`

THE TASK 题目要求

- Write `path_check.py` that reports existence, type and byte size for three paths of your choice.

  编写 `path_check.py`，对你选定的三个路径报告存在性、类型与字节大小。

```
import os

paths = ["classlist.txt", "photos", "ghost.txt"]

for path in paths:
    print("---", path)
    if not os.path.exists(path):
        print("  does not exist / 不存在")
    elif os.path.isfile(path):
        print("  file,", os.path.getsize(path), "bytes")
    elif os.path.isdir(path):
        print("  folder with", len(os.listdir(path)), "items")
```

OUTPUT 输出

```
--- classlist.txt
  file, 24 bytes
--- photos
  folder with 3 items
--- ghost.txt
  does not exist
```

Walkthrough 精解

1. The three questions form a decision tree, and the order matters: `exists` first, then `isfile`, then `isdir`. Asking `getsize` on a missing path would crash with `FileNotFoundError`.

   三个问题构成决策树，顺序很重要：先 `exists`，再 `isfile`，再 `isdir`。对不存在的路径调用 `getsize` 会以 `FileNotFoundError` 崩溃。

2. A folder's "size" is misleading (it measures the directory entry, not the contents), so the solution reports the number of items instead — a small modeling decision that makes the output honest.

   文件夹的"大小"有误导性（它测的是目录项本身，不是内容），所以解法改为报告条目数——一个让输出变得诚实的小型建模决定。

3. The list at the top is the only thing you edit to reuse the script. Three paths, thirty paths — the loop does not care.

   顶部的列表是复用此脚本时唯一要改的东西。三条路径还是三十条，循环一视同仁。

4. `os.path` functions never throw on weird input — they answer questions. Building your programs out of questions instead of assumptions is what makes them survive contact with users.

   `os.path` 的函数对奇怪的输入从不抛异常——它们只回答问题。用"提问"而不是"假设"来构建程序，你的程序才能在与用户的接触中存活。

13.2 `daily_folder.py`

`daily_folder.py`

> **THE TASK** 题目要求
>
> - Write `daily_folder.py` that creates `lab_YYYYMMDD` for today and prints the result.
>
>   编写 `daily_folder.py`，为今天创建 `lab_YYYYMMDD` 文件夹并打印结果。

```
from datetime import datetime
import os

today = datetime.now().strftime("%Y%m%d")
folder = "lab_" + today

if not os.path.exists(folder):
    os.mkdir(folder)
    print("Created:", folder)
else:
    print("Already here:", folder)
```

OUTPUT 输出

```
Created: lab_20261102
$ python3 daily_folder.py        # second run / 第二次运行
Already here: lab_20261102
```

Walkthrough 精解

1. `strftime("%Y%m%d")` produces `20261102` — no separators, so the folder name sorts chronologically in every file manager and needs no interpretation.

   `strftime("%Y%m%d")` 产出 `20261102`——不带分隔符，因此这个文件夹名在任何文件管理器里都按时间排序，且无需解释。

2. `os.mkdir` fails loudly when the folder exists (`FileExistsError`); the `if not exists` guard converts that crash into a polite status line. The guard is Exercise 9.4's pattern, reused.

   `os.mkdir` 在文件夹已存在时会大声失败（`FileExistsError`）；`if not exists` 守卫把这次崩溃变成一行礼貌的状态。这个守卫就是练习 9.4 模式的复用。

3. Machine-generated names are the whole point: humans mistype dates constantly, and `lab_2026-11-2` and `lab_20261102` become two different folders. The computer should name things the computer will look for.

   机器生成的命名才是重点：人类经常打错日期，`lab_2026-11-2` 和 `lab_20261102` 会变成两个不同的文件夹。凡是计算机要找的东西，就该由计算机来命名。

4. Run it twice and show both outputs — the second proves the guard works. This two-run evidence pattern recurs in Exercises 12.3 and 13.4.

   运行两次并展示两个输出——第二个证明守卫生效。这种"两次运行"的证据模式还会出现在练习 12.3 与 13.4。

13.3 `batch_rename.py`

`batch_rename.py`

THE TASK 题目要求

- Write `batch_rename.py`: create `photos/photo_001.jpg` and `photo_002.jpg`, then rename both to `vacation_*`.

  编写 `batch_rename.py`：创建 `photos/photo_001.jpg` 与 `photo_002.jpg`，再重命名为 `vacation_*`。

```
import os

# 1. build a demo photo folder / 造一个演示用照片文件夹
os.makedirs("photos", exist_ok=True)
open("photos/photo_001.jpg", "w").close()
open("photos/photo_002.jpg", "w").close()
print("Before:", os.listdir("photos"))

# 2. rename every photo_* file / 重命名每个 photo_* 文件
for file in os.listdir("photos/"):
    if file.startswith("photo"):
        new_name = file.replace("photo", "vacation")
        os.rename("photos/" + file, "photos/" + new_name)
        print(file, "->", new_name)

print("After:", os.listdir("photos"))
```

OUTPUT 输出

```
Before: ['photo_001.jpg', 'photo_002.jpg']
photo_001.jpg -> vacation_001.jpg
photo_002.jpg -> vacation_002.jpg
After: ['vacation_001.jpg', 'vacation_002.jpg']
```

Walkthrough 精解

1. The demo files are created with `open(..., "w").close()` — an empty file is enough to rename. `os.makedirs(..., exist_ok=True)` is the one-argument version of the guard you wrote in Exercise 13.2.

   演示文件用 `open(..., "w").close()` 创建——空文件足以用来重命名。`os.makedirs(..., exist_ok=True)` 是你在练习 13.2 里手写守卫的单参数版本。

2. `os.rename(old, new)` needs *full paths* on both sides: `os.listdir` returns bare names, so `"photos/" + file` rebuilds the address before renaming. Forgetting the folder prefix is this exercise's classic failure.

   `os.rename(old, new)` 两边都需要*完整路径*：`os.listdir` 返回的是裸名字，所以要用 `"photos/" + file` 重建地址再重命名。忘加目录前缀是本题的经典失败方式。

3. The `startswith("photo")` filter protects files you did not mean to touch. Batch operations without a filter rename *everything* — including the script's own output files if you run it twice.

   `startswith("photo")` 过滤器保护了你不想动的文件。没有过滤器的批量操作会重命名*一切*——包括脚本自己上一次的产物。

4. Because `replace` runs on every matching name, running the script a second time is harmless: no `photo*` names remain, so the loop finds nothing to do. Idempotent scripts — safe to re-run — are a professional habit worth copying.

由于 `replace` 作用于每个匹配的名字，脚本重复运行是无害的：已没有 `photo*` 名字，循环无事可做。"幂等"的脚本——可以安全重跑——是值得学习的职业习惯。

13.4 GBK to UTF-8

GBK 转 UTF-8

> **THE TASK** 题目要求
>
> - Create a GBK file, convert it to UTF-8 with Python, and open both in a text editor to compare.
>
> 创建一个 GBK 文件，用 Python 转成 UTF-8，并在文本编辑器中打开对比。

```
# 1. write a GBK file on purpose / 故意写一个 GBK 文件
text = "姓名,年龄\n张三,25\n李四,30\n"
with open("data_gbk.csv", "w", encoding="gbk") as f:
    f.write(text)

# 2. read it back AS GBK, write it out AS UTF-8 / 以 GBK 读入，以 UTF-8 写出
with open("data_gbk.csv", "r", encoding="gbk") as src:
    content = src.read()
with open("data_utf8.csv", "w", encoding="utf-8") as dst:
    dst.write(content)

# 3. prove the round trip / 证明往返成功
print(open("data_utf8.csv", encoding="utf-8").read())
```

OUTPUT 输出

```
姓名,年龄
张三,25
李四,30
```

Walkthrough 精解

1. An encoding is a translation dictionary between text and bytes. The file's bytes never change meaning on their own — *you* declare which dictionary to use, both when writing (`encoding="gbk"`) and when reading.

   编码是文本与字节之间的翻译字典。文件的字节本身不会改变含义——是*你*在写入（`encoding="gbk"`）和读取时声明用哪本字典。

2. The conversion is a three-liner because it is just copy-with-translation: read with the old dictionary, write with the new one. Python 3's default is UTF-8, which is why the second `open` can omit the argument.

转换只需三行，因为它就是"带翻译的复制"：用旧字典读，用新字典写。Python 3 默认 UTF-8，所以第二个 `open` 可以省略参数。

3. In a text editor, the GBK file opened *as UTF-8* shows mojibake (乱码) — not because the data is damaged, but because the wrong dictionary was applied. Same bytes, wrong key.
   在文本编辑器里，以 UTF-8 方式打开的 GBK 文件会显示乱码——不是数据坏了，而是用错了字典。同样的字节，错误的钥匙。
4. Screenshot both files' properties (or `ls -l`) as well: GBK stores each Chinese character in 2 bytes, UTF-8 in 3, so the UTF-8 file is measurably larger. The size difference is the encoding made visible.
   顺便截下两个文件的属性（或 `ls -l`）：GBK 每个汉字 2 字节，UTF-8 是 3 字节，所以 UTF-8 文件可测地更大。这个大小差就是"编码"的可见形态。

> **COMMON MISTAKE** 常见错误
>
> Opening the GBK file without `encoding="gbk"`. Python 3 assumes UTF-8 and crashes with `UnicodeDecodeError` — an error that names the *right* suspect: the dictionary, not the data.
>
> 不加 `encoding="gbk"` 就打开 GBK 文件。Python 3 默认按 UTF-8 处理，会以 `UnicodeDecodeError` 崩溃——这个报错点名了*正确的嫌疑人*：字典，而不是数据。

13.5 `csv_reader.py`

`csv_reader.py`

> **THE TASK** 题目要求
>
> - Write `csv_reader.py` that reads the Chinese CSV above with the correct encoding, then deliberately uses the wrong one and records the error.
>
>   编写 `csv_reader.py`，用正确编码读取上面的中文 CSV；再故意用错编码并记录报错。

```
# data.csv contains / 文件内容:
#   姓名,年龄
#   张三,25
#   李四,30
# (saved in GBK, as in Exercise 13.4 / 以 GBK 保存，如练习 13.4)
```

```
print("--- correct encoding / 正确编码 ---")
with open("data.csv", "r", encoding="gbk") as file:
    for line in file:
        name, age = line.strip().split(",")
        print(name, "is", age, "years old")

print("--- wrong encoding / 错误编码 ---")
try:
    with open("data.csv", "r", encoding="utf-8") as file:
        content = file.read()
    print(content)
except UnicodeDecodeError as e:
    print("Failed as expected:", e)
```

OUTPUT 输出

```
--- correct encoding / 正确编码 ---
姓名 is 年龄 years old
张三 is 25 years old
李四 is 30 years old
--- wrong encoding / 错误编码 ---
Failed as expected: 'utf-8' codec can't decode byte 0xd0 in position 0: invalid
start byte
```

Walkthrough 精解

1. `line.strip().split(",")` is the Chapter 5 string toolkit applied to a real file: strip the newline, split on the comma, unpack the two fields. The header line `"姓名,年龄"` is handled by the same three steps — which is why it appears in the output as a "person".

   `line.strip().split(",")` 是第 5 章字符串工具在真实文件上的应用：去换行、按逗号切分、解包两个字段。表头行 `"姓名,年龄"` 被同样的三步处理——这也是它作为"一个人"出现在输出里的原因。

2. Note the honest detail: the header goes through the same pipeline as the data. Real code skips it (`next(file)` or `if line.startswith("姓名")`), and noticing that the demo does not is exactly the kind of observation that earns report marks.

   注意这个诚实的细节：表头和数据走了同一条管道。真实代码会跳过表头（用 `next(file)` 或 `if line.startswith("姓名")`），发现演示没跳过，正是能挣到报告分数的那种观察。

3. The wrong-encoding test is wrapped in `try/except UnicodeDecodeError` so the program can *report* the failure and continue — the Chapter 14 pattern, previewed here one chapter early.

   错误编码测试包在 `try/except UnicodeDecodeError` 里，让程序得以*报告*失败并继续——这是第 14 章的模式，提前一章预演。

4. In your report, paste the full error line: it names the codec (`utf-8`), the byte (`0xd0`), and the position. That triple is what you will grep for whenever encoding strikes in the wild.

报告中请粘贴完整的错误行：它点明了编码（`utf-8`）、字节（`0xd0`）和位置。今后在真实世界遇到编码问题，你要 grep 的就是这三元组。

Errors and Exception Handling / 错误与异常处理

> *A crash is honest but useless. An handled error is useful and still honest.*
>
> 崩溃诚实但无用；被处理的错误既有用又诚实。

Chapter 14 has six exercises — the densest set in the course — because error handling is not one skill but a family: catching, distinguishing, guaranteeing cleanup, and raising your own errors. Everything here reappears in Chapter 15's network code.

第 14 章有六道题——全课程最密集的一组——因为错误处理不是一门技能，而是一个家族：捕获、区分、保证清理、亲手抛错。这里的一切都会在第 15 章的网络代码中重现。

14.1

The Bulletproof BMI Calculator · 防弹 BMI 计算器

THE TASK 题目要求

- Write `bmi_calculator.py` wrapped in `try/except` handling both zero height and non-numeric input.

  编写 `bmi_calculator.py`，用 `try/except` 同时处理零身高与非数字输入。

```
try:
    weight = float(input("Weight in kg: "))
    height = float(input("Height in meters: "))
    bmi = weight / height ** 2
except ValueError:
    print("Please enter numbers only.")
except ZeroDivisionError:
    print("Height cannot be zero!")
else:
    print("Your BMI is:", round(bmi, 2))
```

OUTPUT 输出

```
Weight in kg: abc
Please enter numbers only.
Weight in kg: 70
Height in meters: 0
Height cannot be zero!
Weight in kg: 70
Height in meters: 1.75
Your BMI is: 22.86
```

Walkthrough 精解

1. Two different failures, two different `except` blocks. Typing `abc` raises `ValueError` inside `float()`; typing `0` passes the conversion and fails later, in the division. Same `try`, two destinations.

   两种不同的失败，两个不同的 `except` 块。输入 `abc` 让 `float()` 内部抛 `ValueError`；输入 `0` 顺利通过转换，失败发生在后面的除法里。同一个 `try`，两个终点。

2. The successful path lives in `else`, not in `try`. If the BMI print itself had a bug, putting it inside `try` would let an `except` swallow it and print a misleading apology instead.

   成功路径住在 `else` 里而不是 `try` 里。如果打印 BMI 的代码本身有 bug，放进 `try` 会让某个 `except` 把它吞掉，转而打印一句误导性的道歉。

3. `float()` instead of `int()` because height in meters is 1.75, not 1. Choosing the converter is part of designing the input.

   用 `float()` 而非 `int()`，因为身高单位是米，1.75 而非 1。选择转换器本身就是输入设计的一部分。

4. This exercise is Exercise 7.2's vending machine, upgraded. Compare the two in your report: validation by `if 1 <= x <= 3` checks *rules you know*; `try/except` catches *failures you cannot predict*.

   这道题就是练习 7.2 售货机的升级版。请在报告中比较两者：`if 1 <= x <= 3` 校验的是*你已知的规则*；`try/except` 捕获的是*你无法预知的失败*。

> **COMMON MISTAKE** 常见错误
>
> Writing a bare `except:` with no error type. It catches everything — including the `KeyboardInterrupt` you trigger with Ctrl+C while trying to stop a stuck program. Always name the errors you expect.
>
> 写不带错误类型的裸 `except:`。它捕获一切——包括你用 Ctrl+C 想停掉卡死程序时触发的 `KeyboardInterrupt`。永远写明你预期的错误。

14.2 `file_detective.py` and `catch_all.py`

`file_detective.py` 与 `catch_all.py`

> **THE TASK** 题目要求
>
> - Write `file_detective.py` and `catch_all.py` from this chapter and record their outputs.
>
> 按本章编写 `file_detective.py` 与 `catch_all.py`，并记录输出。

```
try:
    with open("ghost_file.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("Pssst... this file doesn't exist!")
```

▾ **`catch_all.py`**

```
try:
    magic_number = "100" + 5
except Exception as e:
    print(f"An error occurred: {e}")
```

OUTPUT 输出

```
Pssst... this file doesn't exist!
An error occurred: can only concatenate str (not "int") to str
```

Walkthrough 精解

1. The detective names one suspect (`FileNotFoundError`) and gives one tailored response. Named errors are documentation: the reader learns exactly which failure the author anticipated.

   侦探只点名一个嫌疑人（`FileNotFoundError`）并给出一个量身定制的回应。指名错误就是文档：读者由此知道作者预见了哪种失败。

2. The catch-all uses `except Exception as e`, binding the error object to `e` so its message can be printed with an f-string. The output line *is* the original error, minus the crash.

   兜底网用 `except Exception as e`，把错误对象绑到 `e`，再用 f-string 打印它的消息。输出的那一行就是原始报错本身——只是不崩溃了。

3. Which to choose in real code? Specific first, catch-all last — as a final line of defense that logs and moves on. The handbook's warning stands: `except (ValueError, Exception)` is redundant because `Exception` already includes `ValueError`.

   真实代码怎么选？具体错误在前，兜底网殿后——作为记录日志并继续前进的最后防线。学习手册的警告仍然成立：`except (ValueError, Exception)` 是冗余的，因为 `Exception` 本就包含 `ValueError`。

4. Both files together demonstrate the two poles of error handling: *respond precisely* when you know the failure, *report honestly* when you do not.

   两个文件合起来演示了错误处理的两个极端：知道会怎么败时*精确应对*，不知道时*诚实上报*。

14.3 `multiple_exceptions.py`

`multiple_exceptions.py`

THE TASK 题目要求

- Write `multiple_exceptions.py`; test inputs `0`, `string` and `4`, and explain each result.

  编写 `multiple_exceptions.py`；测试输入 `0`、`string`、`4`，并逐一解释结果。

```
try:
    age = int(input("Age: "))
    print(str(100 / age))
except ValueError:
    print("Numbers only!")
except ZeroDivisionError:
    print("Age can't be zero!")
```

OUTPUT 输出

```
Age: 0
Age can't be zero!
Age: string
Numbers only!
Age: 4
25.0
```

Walkthrough 精解

1. Input `0`: `int("0")` succeeds, so no `ValueError`; the division `100 / 0` raises `ZeroDivisionError`, which the second block catches. The error happened *later in the try*, which is why a different handler fires.

   输入 `0`：`int("0")` 成功，没有 `ValueError`；除法 `100 / 0` 抛出 `ZeroDivisionError`，由第二个块捕获。错误发生在 `try` 的*更后面*，所以触发的是另一个处理器。

2. Input `string`: the conversion itself explodes — `int("string")` raises `ValueError` — and the program never reaches the division. Python checks handlers top-to-bottom and runs only the first match.

   输入 `string`：转换本身爆炸——`int("string")` 抛 `ValueError`——程序根本到不了除法那行。Python 自上而下检查处理器，只执行第一个匹配。

3. Input `4`: no exception at all, so both `except` blocks are skipped and `25.0` prints. Note the output is a float: true division always is (Chapter 4).

   输入 `4`：完全没有异常，两个 `except` 都被跳过，打印 `25.0`。注意输出是浮点数：真除法永远是浮点（第 4 章）。

4. The order of the `except` blocks does not matter here because the two errors are unrelated siblings. It matters enormously when one is a parent of the other — see the handbook's ordering warning.

   这里 `except` 块的顺序无关紧要，因为两个错误是互不相干的兄弟。当一个是另一个的父类时，顺序就极其重要——见学习手册的顺序警告。

14.4

The Tuple-Style Grade Calculator · 元组式成绩计算器

> **THE TASK** 题目要求
>
> - Rewrite the grade calculator in tuple style, then argue whether the tuple version is better.
>
>   将成绩计算器改写为元组式捕获，再论证元组版本是否更优。

```
try:
    with open("grades.txt") as f:
        print(f.read())
except (FileNotFoundError, PermissionError):
    print("File problem! Check the file exists and you have access.")
```

OUTPUT 输出

```
File problem! Check the file exists and you have access.
```

Walkthrough 精解

1. The tuple version groups errors that deserve the *same* response: from the user's point of view, "file missing" and "no permission" are both "I can't read your file". One handler, one message.

   元组版本把需要*同样*回应的错误归为一组：从用户视角看，"文件缺失"和"没有权限"都是"我读不了你的文件"。一个处理器，一句话。

2. The argument *for* tuples: less duplication, and adding a third related error (`IsADirectoryError`) is a one-word edit. The argument *against*: the user can no longer tell *which* of the three things went wrong.

   支持元组的理由：消除重复，且增加第三个相关错误（`IsADirectoryError`）只需改一个词。

   反对的理由：用户再也无法分辨三者中*究竟*哪个出了问题。

3. The honest verdict for a report: tuple style fits when the recovery is identical; separate handlers fit when each failure needs different advice. The calculator example from the handbook — "divide by zero" vs "type letters" — needs different advice, so it must stay split.

   报告的诚实结论：恢复方式完全相同时用元组式；各失败需要不同指导时用分开的处理器。学习手册的计算器例子——"除以零"与"输入字母"——需要不同指导，所以必须分开。

4. Either way, the tuple syntax itself is just a list of error types in parentheses: `except (A, B, C):`. It composes with everything else in this chapter.

   无论如何，元组语法本身只是括号里的一列错误类型：`except (A, B, C):`。它与本章其他一切都能组合。

14.5 `safe_config.py` and `safe_divide.py`

`safe_config.py` 与 `safe_divide.py`

> **THE TASK** 题目要求
>
> - Write `safe_config.py` and `safe_divide.py`; annotate each block with one line of comment.
>
>   编写 `safe_config.py` 与 `safe_divide.py`；为每个块写一行注释。

```
def safe_divide(a, b):
    try:
        result = a / b            # the risky part / 风险所在
    except ZeroDivisionError:     # the one failure we expect / 唯一预期的失败
        print("Division by zero!")
    else:
        print("Result is " + str(result))   # only on success / 仅成功时
    finally:
        print("Operation complete")         # always / 永远执行

safe_divide(10, 2)
safe_divide(5, 0)
```

▾ **safe_config.py**

```
config_file = False
try:
    config_file = open("game_config.cfg")   # may fail / 可能失败
except FileNotFoundError:
    print("Config file missing!")           # complain if absent / 缺失时抱怨
else:
    settings = config_file.read()           # only if opened / 仅当打开成功
    print("Game configuration loaded!")
finally:
    if config_file:                         # close only what was opened / 打开过才有关可闭
        config_file.close()
    print("Resource cleanup completed")
```

OUTPUT 输出

```
Result is 5.0
Operation complete
Division by zero!
Operation complete
Config file missing!
Resource cleanup completed
```

Walkthrough 精解

1. Narrate the four rooms of `safe_divide`: try computes, except complains, else celebrates, finally signs off. `finally` runs in *both* invocations — that is the "always" in the handbook's table.

   复述 `safe_divide` 的四个房间：try 计算、except 抱怨、else 庆祝、finally 签退。两次调用中 `finally` 都执行了——这就是学习手册表格里的"永远"。

2. `safe_config.py` shows why `finally` needs a guard: if `open` failed, there is nothing to close, so `if config_file:` protects the cleanup from its own good intentions. (`False` doubles as "not opened yet".)

   `safe_config.py` 解释了 `finally` 为何需要守卫：如果 `open` 失败了，就无东西可关，`if config_file:` 保护清理逻辑不因好心而出错。（`False` 兼任"尚未打开"的标记。）

3. Note that modern Python would write `with open(...) as f:` and skip the manual close entirely. This exercise exists so you understand *what* `with` automates.

   注意现代 Python 会直接写 `with open(...) as f:`，完全免去手动关闭。这道题的存在是为了让你理解 `with` 自动化的*究竟是什么*。

4. The one-line comments the task asks for are the deliverable: each comment names the block's *job*, not its syntax. When your comments could not be wrong if the code changed, they are real comments.

   题目要求的单行注释就是交付物：每条注释说明该块的*职责*，而不是它的语法。当代码怎么改注释都不会错时，那才是真注释。

14.6

`retry_dynamic.py` and Raising Your Own · `retry_dynamic.py` 与亲手抛错

THE TASK 题目要求

- Write `retry_dynamic.py` with decreasing wait times, then username checking with `raise` inside a `while` loop.

> 编写等待时间递减的 `retry_dynamic.py`；再在 `while` 循环中用 `raise` 实现用户名查重。

```
import time
import requests

wait = 8
while wait > 0:
    try:
        response = requests.get("https://www.baidu.com", timeout=3)
        print("Success!")
        break
    except requests.exceptions.ConnectTimeout:
        print("Timeout. Retrying in", wait, "seconds...")
        time.sleep(wait)
        wait = wait - 2        # decreasing patience / 递减的耐心
```

▾ **username_check.py**

```
existing_users = ["alice", "bob", "charlie"]

while True:
    try:
        new_user = input("Choose username: ")
        if new_user in existing_users:
            raise ValueError("Username taken!")
        print("Username available!")
        break
    except ValueError as e:
        print(e)
```

OUTPUT 输出

```
Choose username: bob
Username taken!
Choose username: diana
Username available!
```

Walkthrough 精解

1. The retry loop's patience is a variable: each failure costs 2 seconds less of waiting (`wait = wait - 2`), so the program tries hard at first and gives up gracefully. Fixed delays punish every failure equally; dynamic ones know how to stop.

   重试循环的耐心是个变量：每次失败少等 2 秒（`wait = wait - 2`），程序先努力尝试，再体面放弃。固定延时对所有失败一视同仁；动态延时懂得适可而止。

2. `raise ValueError("Username taken!")` creates *and* throws the error in one line. The message you write is exactly what the `except` prints — you are the author of your own error messages.

   `raise ValueError("Username taken!")` 一行之内完成错误的*构造*与*抛出*。你写的消息就是 `except` 打印的内容——你成了自己错误信息的作者。

3. Why `raise` instead of just printing "taken"? Because the *rule* and the *reaction* now live in different places: any caller — this input loop, a web form, Chapter 17's pipeline — can catch the same `ValueError` and react its own way.

   为什么用 `raise` 而不是直接打印"已被占用"？因为*规则*与*反应*从此分居两地：任何调用者——这个输入循环、一个网页表单、第 17 章的流水线——都能捕获同一个 `ValueError`，并按自己的方式反应。

4. The `while True` + `try/except` + `break` shape repeats until the input is acceptable. It is the professional version of the input-validation loop from Exercise 7.2, and it closes the course's arc: from "check what you know" to "catch what you cannot".

   `while True` + `try/except` + `break` 的形状会一直循环，直到输入可接受。它是练习 7.2 输入校验循环的职业版，也为本课程收束了一条弧线：从"检查你已知的"到"接住你未知的"。

Third-Party Libraries and pip / 第三方库与 pip

> *The standard library is what Python ships with; PyPI is what humanity has added since.*
>
> 标准库是 Python 出厂自带的，PyPI 是全人类后来加上去的。

Chapter 15's exercises leave the isolated machine: you install, you download, you talk to a live weather service. Outputs here contain live data, so yours will differ from the book's — that difference is not an error, it is the point.

第 15 章的练习走出了孤立的机器：你要安装、要下载、要与在线天气服务对话。这里的输出包含实时数据，所以你的结果与本书不同——差异不是错误，正是要义所在。

15.1 The Full `pip` Sequence

完整的 pip 流程

> **THE TASK** 题目要求
>
> - Run the full pip sequence in Section 15.2 on the server and screenshot `pip3 list`.
>
> 在服务器上运行 15.2 节的完整 pip 流程，并截图 `pip3 list`。

```
pip3 install requests                 # install / 安装
pip3 show requests                    # details of one package / 单个包详情
pip3 list                             # everything installed / 已安装的一切
pip3 install --upgrade requests       # upgrade / 升级
pip3 uninstall -y requests            # remove / 卸载
pip3 install requests                 # ... and put it back / ...再装回来
```

OUTPUT 输出

```
$ pip3 show requests
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
$ pip3 list
Package    Version
---------- -------
pip        23.0
requests   2.31.0
urllib3    2.0.7
```

Walkthrough 精解

1. `pip3 list` is the evidence screenshot: it proves the package manager works and shows what else arrived with `requests` — `urllib3` is a *dependency*, installed automatically.

   `pip3 list` 就是要截的证据图：它证明包管理器可用，并显示随 `requests` 一起到来的其他包——`urllib3` 是*依赖*，被自动安装。

2. `pip3 show` answers "what is this, exactly": version, summary, and what it depends on. Before upgrading anything in production, this is the page to read.

   `pip3 show` 回答"它到底是什么"：版本、简介、依赖。在生产环境升级任何东西之前，先读这一页。

3. The uninstall-reinstall loop in the ritual is deliberate practice: packages are cheap and disposable. Fear of breaking an environment is what keeps beginners from experimenting — and pip removes the risk.

   卸载再安装的循环是刻意练习：包是廉价的一次品。对弄坏环境的恐惧让新手不敢实验——而 pip 移除了这种风险。

4. Mind the `3`: `pip` may point to an old Python 2 on some systems. On this server always type `pip3`, just as you always type `python3`.

   注意那个 `3`：某些系统上 `pip` 可能指向老旧的 Python 2。在这台服务器上永远输入 `pip3`，正如你永远输入 `python3`。

> **COMMON MISTAKE** 常见错误
>
> Running `pip3 install` *inside* the Python interactive prompt. pip is a shell command — it belongs at the `$` prompt. `>>>` is for Python only.
>
> 在 Python 交互提示符里运行 `pip3 install`。pip 是 shell 命令——它属于 `$` 提示符。`>>>` 只属于 Python。

15.2 `requests_demo.py`

`requests_demo.py`

> **THE TASK** 题目要求
>
> - Write `requests_demo.py` fetching baidu.com, printing the status code and the first 100 characters.

> 编写 `requests_demo.py` 抓取 baidu.com，打印状态码与前 100 个字符。

```
import requests

response = requests.get("https://www.baidu.com")
print("Status Code:", response.status_code)
print("First 100 characters:")
print(response.text[:100])
```

OUTPUT 输出

```
Status Code: 200
First 100 characters:
<!DOCTYPE html><!--STATUS OK--><html><head><meta http-equiv=Content-Type
content="text/html;cha
```

Walkthrough 精解

1. `requests.get(url)` performs the entire conversation — DNS lookup, TCP connection, HTTP request, response parsing — behind one function call. What took a chapter of a networking course is one line here.

   `requests.get(url)` 在一次函数调用背后完成了整场对话——DNS 查询、TCP 连接、HTTP 请求、响应解析。网络课程一整章的内容，这里只有一行。

2. `response.status_code` is the verdict: `200` means OK, `4xx` means you asked wrong, `5xx` means the server stumbled. Check it *before* touching the text — a `404` page also has text, just not the text you wanted.

   `response.status_code` 是判决：`200` 一切正常，`4xx` 是你问错了，`5xx` 是服务器摔了。先看状态码*再*碰文本——`404` 页面也有文本，只是不是你要的文本。

3. `response.text[:100]` is the Chapter 5 slice applied to a live download: characters 0 through 99. Web pages are megabytes; the first hundred characters tell you what arrived.

   `response.text[:100]` 是第 5 章的切片用在实时下载上：第 0 到 99 个字符。网页动辄数兆，前一百个字符足以告诉你到达的是什么。

4. Your first 100 characters will differ from the book's — the page changes daily. What must match is the `200`. When it does not, read the status code before anything else.

   你的前 100 个字符会与本书不同——页面每天在变。必须一致的是那个 `200`。不一致时，先读状态码，再谈其他。

15.3 `weather_reporter.py`

`weather_reporter.py`

THE TASK 题目要求

- Write `weather_reporter.py` and report for two cities; identify the timezone problem in your report.

  编写 `weather_reporter.py` 查询两个城市；在报告中指出时区问题。

```
import datetime
import requests

def get_weather(city):
    # %t = temperature, %C = condition / %t 温度，%C 天气状况
    url = "http://wttr.in/" + city + "?format=%t+%C"
    response = requests.get(url)
    return response.text

def weather_report(city):
    time_now = datetime.datetime.now().strftime("%H:%M")
    weather = get_weather(city)
    return "[" + time_now + "] Weather in " + city + ": " + weather

print(weather_report("London"))
print(weather_report("NewYork"))
```

OUTPUT 输出

```
[10:22] Weather in London: +8°C Light rain
[10:22] Weather in NewYork: +12°C Clear
```

Walkthrough 精解

1. Two small functions, one data pipeline: `get_weather` fetches, `weather_report` decorates with a timestamp and joins the strings. Splitting fetch from format is what lets you change either without touching the other.

   两个小函数，一条数据管道：`get_weather` 负责抓取，`weather_report` 负责加时间戳并拼接字符串。抓取与排版分离，改其一而不动另一个。

2. The `?format=%t+%C` is a *query string* — everything after `?` customises the request. The weather service documents these codes; reading API documentation is now a survival skill.

   `?format=%t+%C` 是*查询字符串*——`?` 之后的一切都在定制请求。这些代码由天气服务的文档说明；读 API 文档从现在起是生存技能。

3. The timezone problem, in one sentence for your report: *both lines show the same time because `datetime.now()` returns the server's clock, not the queried city's — a report about New York is*

stamped with Beijing time. The honest fix needs timezone-aware datetimes (`datetime.timezone` or `pytz`); the first step, which this exercise grades, is noticing.

时区问题，报告用一句话：*两行时间相同，因为 `datetime.now()` 返回的是服务器时钟而非被查询城市的时钟——关于纽约的报告盖的是北京的钟。*诚实的修复需要带时区的日期时间（`datetime.timezone` 或 `pytz`）；而本题考察的第一步，是察觉到它。

4. Live APIs also *fail* live: unplug the network (or mistype the city) and `requests.get` hangs or errors. Wrap the call in the retry pattern from Exercise 14.6 and this toy becomes robust.

   在线 API 也会*在线地*失败：断网（或打错城市名）时 `requests.get` 会挂起或报错。套上练习 14.6 的重试模式，这个玩具就变得健壮。

> **REPORT TIP** 报告技巧
>
> Run the program twice a few hours apart and screenshot both: the changing weather with the unchanging timestamp makes the timezone flaw visible to any reader.
>
> 隔几个小时运行两次并都截图：天气在变而时间戳纹丝不动，任何读者都能一眼看出时区缺陷。

15.4 `backup_helper.py`

`backup_helper.py`

> **THE TASK** 题目要求
>
> - Write `backup_helper.py`: create `notes/` with two .txt files, run the backup, then verify destination contents.
>
>   编写 `backup_helper.py`：在 notes/ 放两个 .txt，运行备份，并核对目标内容。

```
import os
import shutil

# 1. build the demo data / 造演示数据
os.makedirs("notes", exist_ok=True)
open("notes/idea.txt", "w").write("Final project: AI scriptwriter\n")
open("notes/todo.txt", "w").write("1. finish chapter 15\n")

def backup_files(src_folder, dst_folder):
    # 2. create the destination if missing / 目标不存在则创建
    if not os.path.exists(dst_folder):
        os.makedirs(dst_folder)
        print("Created backup directory: " + dst_folder)
    # 3. copy every .txt file / 复制每个 .txt 文件
```

```
    for file in os.listdir(src_folder):
        if file.endswith(".txt"):
            src_path = os.path.join(src_folder, file)
            dst_path = os.path.join(dst_folder, file)
            shutil.copy2(src_path, dst_path)
            print("Copied " + file + "!")

backup_files("notes", "backup")

# 4. verify / 核对
print("Backup contents:", os.listdir("backup"))
print(open("backup/idea.txt").read())
```

OUTPUT 输出

```
Created backup directory: backup
Copied idea.txt!
Copied todo.txt!
Backup contents: ['idea.txt', 'todo.txt']
Final project: AI scriptwriter
```

Walkthrough 精解

1. `os.path.join(src_folder, file)` builds paths the portable way: on Linux it inserts `/`, on Windows `\`. The string-concatenation habit from Exercise 13.3 works but is platform-bound; `join` is the professional form.

   `os.path.join(src_folder, file)` 用可移植的方式拼路径：Linux 上插 `/`，Windows 上插 `\`。练习 13.3 的字符串拼接能用但绑定平台；`join` 是职业写法。

2. `shutil.copy2` copies the file *and* its metadata (timestamps). Plain `copy` would leave every backup looking freshly written today — useless when you later ask "which backup is newest?".

   `shutil.copy2` 复制文件*及其元数据*（时间戳）。普通 `copy` 会让每个备份看起来都是今天新写的——等你日后问"哪个备份最新"时就抓瞎了。

3. The `endswith(".txt")` filter makes the tool *specific*: images and code files in `notes/` stay behind. Every useful automation draws a line around what it touches.

   `endswith(".txt")` 过滤器让工具*有边界*：notes/ 里的图片和代码文件安然不动。每个有用的自动化都会为"自己碰什么"画出一条线。

4. Verification closes the loop: `os.listdir("backup")` proves the names arrived, printing one file proves the *contents* did. An unverified backup is a hope, not a backup.

   核对让闭环完成：`os.listdir("backup")` 证明名字到达了，打印一个文件证明*内容*也到了。未经核实的备份不是备份，是愿望。

Doing Several Things at Once / 多任务处理

> *Waiting is the programmer's oldest tax. Threads refund part of it.*
>
> 等待是程序员最古老的税，多线程退还其中一部分。

Chapter 16's four exercises climb from a single background task to a timed comparison of sequential versus concurrent downloads. The queue pattern in Exercise 16.3 is the direct ancestor of the final project's prompt pipeline.

第 16 章的四道题从单个后台任务爬升到"顺序 vs 并发下载"的计时对比。练习 16.3 的队列模式正是期末项目提示词流水线的直系祖先。

16.1 `thread_demo.py`

`thread_demo.py`

THE TASK 题目要求

- Write `thread_demo.py` with a 5-second background task; print "main continues" before it finishes.

  编写 `thread_demo.py`：后台任务 5 秒，在其结束前打印"主线程继续"。

```
from threading import Thread
import time

def blocking_task():
    print("Background task started...")
    time.sleep(5)
    print("Background task finished!")

this_thread = Thread(target=blocking_task)
this_thread.start()                      # runs in the background / 后台运行
print("Main thread continues working immediately!")
this_thread.join()                       # wait for it / 等它完成
print("Is thread alive?", this_thread.is_alive())
```

OUTPUT 输出

```
Background task started...
Main thread continues working immediately!
Background task finished!        (5 seconds later / 5 秒之后)
Is thread alive? False
```

Walkthrough 精解

1. `Thread(target=blocking_task)` packages the function but does not run it; `.start()` launches it on a second thread and returns *immediately*. That instant return is the entire trick.

   `Thread(target=blocking_task)` 只是打包了函数并不运行它；`.start()` 在第二条线程上启动它并*立刻*返回。这个"立刻返回"就是全部戏法。

2. Note the output order: the main thread's message appears *between* the background task's two lines. Without threads this interleaving is impossible — the program would print all three lines in order.

   注意输出的顺序：主线程的消息出现在后台任务两行*之间*。没有线程这种交错是不可能的——程序只会按顺序打印三行。

3. `blocking_task` is passed by *name without parentheses*: `target=blocking_task` hands over the function itself, while `target=blocking_task()` would run it first and hand over its return value (`None`).

   `blocking_task` 以*不带括号的名字*传入：`target=blocking_task` 交出的是函数本身；`target=blocking_task()` 会先运行函数，交出它的返回值（`None`）。

4. `.join()` re-threads the two paths: the main thread waits at that line until the background task ends. Delete it and the program may exit before "Background task finished!" ever prints.

   `.join()` 把两条路径重新汇合：主线程停在那行等待后台任务结束。删掉它，程序可能在"Background task finished!"打印之前就退出了。

16.2 `thread_args.py`

`thread_args.py`

> **THE TASK** 题目要求
>
> - Write `thread_args.py` passing different sleep times to two threads.
>
>   编写 `thread_args.py`，向两个线程传入不同的睡眠时长。

```
from threading import Thread
import time

def sleep_task(sleep_time=5):
    print("Sleeping for", sleep_time, "seconds...")
    time.sleep(sleep_time)
    print("Done after", sleep_time, "seconds.")
```

```
fast = Thread(target=sleep_task, args=(2,))
slow = Thread(target=sleep_task, args=(6,))

fast.start()
slow.start()
fast.join()
slow.join()
print("All threading finished.")
```

OUTPUT 输出

```
Sleeping for 2 seconds...
Sleeping for 6 seconds...
Done after 2 seconds.       (at t=2 / 第 2 秒)
Done after 6 seconds.       (at t=6 / 第 6 秒)
All threading finished.
```

Walkthrough 精解

1. Arguments travel in the `args` tuple — and `(2,)` needs its trailing comma. `(2)` is just the number 2; `(2,)` is a one-element tuple, the same trap as Exercise 10.4, back in a new costume.

   参数装在 `args` 元组里传递——`(2,)` 的尾逗号不可省。`(2)` 只是数字 2；`(2,)` 才是单元素元组——练习 10.4 的同一个陷阱，换了身戏服。

2. The output interleaves because the two threads genuinely run at the same time: both "Sleeping" lines print immediately, then each "Done" appears when *its* timer expires.

   输出交错是因为两条线程真的在同时运行：两行 "Sleeping" 立刻打印，随后每当*各自*的计时器到期，对应的 "Done" 才出现。

3. Total runtime is 6 seconds, not 8: the two sleeps overlap. That overlap is the whole economic argument for concurrency in one observation.

   总耗时是 6 秒而非 8 秒：两个睡眠重叠了。这一条观察就是并发全部经济学论证的浓缩。

4. To see the difference, change `args=(6,)` to `args=(3,)` — with equal lengths the ordering of the "Done" lines becomes a race. Races are not bugs *in* your code; they are missing decisions *about* your code.

   把 `args=(6,)` 改成 `args=(3,)` 再看——时长相同后，两行 "Done" 的先后就成了一场竞赛。竞争不是*代码里的* bug，而是*对代码*缺少决定。

16.3 `thread_queue.py`

`thread_queue.py`

THE TASK 题目要求

- Write `thread_queue.py`: two threads compute squares of different numbers; the main thread collects both results.

  编写 `thread_queue.py`：两个线程分别计算不同数字的平方，主线程收集两个结果。

```
from threading import Thread
import queue
import time

def calculate_task(q, value):
    print("Calculating square of", value)
    time.sleep(2)                    # pretend this is slow work / 假装这是个慢活
    q.put(value * value)             # ship the result / 送出结果

q = queue.Queue()
threads = []
for n in [5, 12]:
    t = Thread(target=calculate_task, args=(q, n))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("Result 1:", q.get())          # 25
print("Result 2:", q.get())          # 144
```

OUTPUT 输出

```
Calculating square of 5
Calculating square of 12
Result 1: 25
Result 2: 144
```

Walkthrough 精解

1. A function running in a thread cannot `return` to its caller — nobody is waiting at the call site. The queue is the mailbox: `q.put(...)` posts a result, `q.get()` collects one.

   线程里运行的函数无法向调用者 `return`——调用点没有人在等。队列就是邮箱：`q.put(...)` 投递结果，`q.get()` 收取结果。

2. `queue.Queue()` is *thread-safe*: two threads may `put` at the same instant and nothing is lost. A plain list would risk the race conditions the handbook warns about.

   `queue.Queue()` 是*线程安全的*：两个线程可以同一瞬间 `put` 而不丢数据。普通列表则有学习手册警告的竞争条件风险。

3. The loop builds, starts and joins a *list* of threads — the pattern scales from 2 threads to 200 without changing a line. Note results come out in the order they were put, and each `get()` removes its item.

循环构建、启动并汇合一个线程*列表*——这个模式从 2 线程扩到 200 线程不需要改一行。注意结果按投递顺序取出，且每个 `get()` 会取走它的元素。

4. In the final project, each AI call replaces `value * value` here and the same queue pattern collects the generated prompts. You have just rehearsed Chapter 17's skeleton.

   期末项目里，这里的 `value * value` 将换成一次 AI 调用，同一队列模式负责收集生成的提示词。你刚刚排练了第 17 章的骨架。

16.4 Sequential versus Threaded, Timed

顺序与并发，计时对比

THE TASK 题目要求

- Compare: run the same three downloads sequentially and in three threads, and time both versions.

  对比：用顺序与三线程两种方式下载同样三个任务，并分别计时。

```
import time
import requests
from threading import Thread
import queue

SITES = ["https://www.baidu.com",
         "https://www.python.org",
         "https://www.wikipedia.org"]

def download(url, q=None):
    response = requests.get(url, timeout=10)
    size = len(response.text)
    if q is not None:
        q.put((url, size))
    return size

# --- sequential / 顺序 ---
start = time.time()
for url in SITES:
    download(url)
print("Sequential: %.2f s" % (time.time() - start))

# --- threaded / 多线程 ---
q = queue.Queue()
start = time.time()
threads = [Thread(target=download, args=(url, q)) for url in SITES]
for t in threads: t.start()
for t in threads: t.join()
print("Threaded:   %.2f s" % (time.time() - start))
while not q.empty():
    print("  got", q.get()[0])
```

OUTPUT 输出

```
Sequential: 3.41 s
Threaded:   1.28 s
  got https://www.wikipedia.org
  got https://www.baidu.com
  got https://www.python.org
```

Walkthrough 精解

1. `time.time()` before and after, and the difference is the measurement. Put the timer *tight around the work* — including the loop, excluding the setup — or you will be timing your own prints.

   `time.time()` 前后各取一次，差值就是测量值。计时器要*紧贴工作*——含循环、不含准备——否则你测的是自己打印的速度。

2. The threaded version wins because downloads are *I/O-bound*: the CPU spends almost all its time waiting for the network, and threads are exactly the tool for waiting in parallel. Three waits of ~1 second happen at once, so ~1 second total.

   多线程版本获胜，因为下载是 *I/O 密集*的：CPU 几乎所有时间都在等网络，而线程正是"并行地等"的工具。三次约 1 秒的等待同时发生，总共约 1 秒。

3. The queue stores `(url, size)` tuples so results stay labelled — a thread that returns a bare number leaves you guessing which site it belongs to.

   队列存的是 `(url, size)` 元组，结果自带标签——只返回裸数字的线程会让你猜数字属于哪个网站。

4. What the timing does *not* show: CPU-bound work (compressing, encrypting, crunching numbers) gains nothing from threads in Python, because of the Global Interpreter Lock. Rule to remember: threads for waiting, processes for computing.

   计时没有展示的另一面：CPU 密集的工作（压缩、加密、算数）在 Python 里靠线程得不到收益，因为全局解释器锁的存在。记住这条规则：等待用线程，计算用进程。

PART VI

The Final Project / 期末项目

The handbook's Chapter 17 contains no exercise box, because the whole chapter *is* the exercise. This part of the codebook provides what the other chapters could not: a complete demo answer for the final project — a modified reference implementation with every function explained, every prompt dissected, and the five hard requirements checked off one by one.

学习手册第 17 章没有练习框，因为整章*就是*那道练习。本册的这一部分提供了其他章节无法提供的东西：期末项目的完整示范答案——一份修改版参考实现，逐函数讲解、逐条提示词拆解，并把五条硬要求逐一核对。

The Python Movie Scriptwriter — Demo Answer / Python 电影编剧程序——示范答案

> *The answer to the final project is a program that writes answers.*
>
> 期末项目的答案，是一个会写答案的程序。

The reference program at `/home/toolkit/tools/ai/ai.py` works, but its four builders are near-copies of each other. The demo answer below keeps the architecture — one conversation per function, iterative clip generation — while removing the duplication with one generic helper. It also adds two touches of its own: it asks the *user* for the movie idea (Requirement 5), and it keeps a log file of every prompt (Chapter 12, meeting the project).

`/home/toolkit/tools/ai/ai.py` 里的参考程序可以运行，但它的四个构建函数几乎是彼此的复制品。下面的示范答案保留了原架构——每个函数一次对话、迭代生成片段——同时用一个通用辅助函数消除了重复。它还加入了两处自己的改动：向*用户*询问电影创意（要求 5），并把每条提示词记入日志文件（第 12 章与项目正式会师）。

17.1 The Architecture at a Glance

架构一览

Function	Role	Book concept
`ask(content)`	One conversation turn with the AI 与 AI 的一次对话	Functions (Ch 8)
`build_clips(...)`	The generic iterative clip loop 通用的迭代片段循环	Loops + lists (Ch 7, 10)
`build_open(idea)`	Opening image + first 5 s clip 开场画面 + 首个 5 秒片段	Strings (Ch 5)
`build_question(idea)`	20 s framework ending on a cliffhanger 留悬念的 20 秒框架	Composition (Ch 8)
`build_value(idea)`	The same pattern for a 15 s ad	Reuse (Ch 9)

	同一模式用于 15 秒广告	
`build_answer(framework)`	Resolves the cliffhanger 消除悬念	Return values (Ch 8)
`run()`	Conductor: input, calls, logging 指挥：输入、调用、记录	Files (Ch 12)

Read the table once more after studying the code — it is also the outline for your presentation's code-walkthrough segment.

研读代码后再读一遍这张表——它同时就是你展示环节"代码讲解"部分的提纲。

17.2 The Demo Answer, Annotated

带注释的示范答案

▾ **movie_writer.py — part 1: the helpers** 第一部分：辅助函数

```
# movie_writer.py — demo answer, modified from the reference
# 示范答案：基于参考程序修改
from .common_imports import *
from src.util import *

def ask(content):
    """One conversation with the AI assistant. / 与 AI 助手的一次对话。"""
    message = [{'role': 'user', 'content': content}]
    return getResult(message)

def build_clips(kind, framework, first_frame, n_clips):
    """Iteratively generate n_clips 5-second clip prompts.
    迭代生成 n_clips 个 5 秒片段提示词。
    This ONE function replaces three near-identical while loops
    in the reference. / 这一个函数取代了参考程序里三个几乎相同的 while 循环。"""
    prompts_list = []

    # clip 1 is anchored to the opening frame / 第一段锚定在开场画面上
    prompt_1 = ask('{' + first_frame + '}'
                   + '以上为' + kind + '开头画面的提示词，现在根据以下' + kind
                   + '框架设计第一个 5 秒钟的片段的提示词，{' + framework
                   + '}，用于输入 WAN 生成' + kind + '片段，用中文生成')
    prompts_list.append(prompt_1)
    print('以下是第 1 个片段的提示词:')

    # clips 2..n each see the framework AND all previous clips
    # 第 2..n 段同时看到框架与此前全部片段
    i = 1
    while i < n_clips:
        history = ';'.join(prompts_list)
        next_prompt = ask('{' + framework + '},以上为' + kind + '故事框架，'
                          '以下是已经生成的片段的提示词：{' + history
                          + '}，生成下一个 5 秒的片段的提示词，'
```

```
                              '用于输入 WAN 生成' + kind + '片段，用中文生成')
        prompts_list.append(next_prompt)
        print('以下是第' + str(i + 1) + '个片段的提示词:')
        i = i + 1
    return prompts_list
```

movie_writer.py — part 2: the three builders　第二部分：三个构建函数

```
def build_open(start_content):
    """Opening image prompt + one 5-second clip. / 开场画面提示词 + 一段 5 秒片段。"""
    image_prompt = ask('{' + start_content + '}'
                       + '以上内容为一个电影的整体要求，现在设计一个开场画面，'
                       '开场画面需要吸引人，生成用于绘制该开场画面的提示词，'
                       '用于输入 Stable Diffusion 生成图像，用英文生成。')
    clips = build_clips('电影', start_content, image_prompt, 1)
    return [image_prompt, clips[0]]

def build_question(start_content):
    """A 20-second story framework that ends on a cliffhanger.
    一个结尾留悬念的 20 秒故事框架。"""
    framework = ask('{' + start_content + '}'
                    + '以上内容为一个电影的整体要求，现在设计一个 20 秒的电影故事框架，'
                    '该电影结尾要留有悬念，吸引观众期待结果，但是电影不给出结果。')
    print('以下是 20s 电影整体框架:')
    first_frame = ask('{' + framework + '}'
                      + '以上内容为一个 20 秒电影的故事框架，现在设计一个开场画面，'
                      '生成用于绘制该开场画面的提示词，'
                      '用于输入 Stable Diffusion 生成图像，用英文生成。')
    clips = build_clips('电影', framework, first_frame, 4)   # 4 x 5s = 20s
    return [framework, first_frame, clips]

def build_answer(question_content):
    """Sequel: the same chain, but the ending resolves the suspense.
    续篇：同一条链，但结尾消除悬念。"""
    framework = ask('{' + question_content + '}'
                    + '以上内容为一个悬疑电影的框架，现在设计一个 20 秒的接下来的框架，'
                    '该电影结尾要消除悬念，帮助观众给出结果。')
    first_frame = ask('{' + framework + '}'
                      + '以上内容为一个 20 秒电影的故事框架，现在设计一个开场画面，'
                      '生成用于绘制该开场画面的提示词，'
                      '用于输入 Stable Diffusion 生成图像，用英文生成。')
    clips = build_clips('电影', framework, first_frame, 4)
    return [framework, first_frame, clips]
```

movie_writer.py — part 3: the ad builder and the conductor　第三部分：广告构建函数与指挥

```
def build_value(value_content):
    """A 15-second advertisement: same machinery, different brief.
    一条 15 秒广告：同样的机器，不同的任务书。"""
    framework = ask('{' + value_content + '}'
                    + '以上内容为一个广告的整体要求，现在设计一个 15 秒的广告故事框架，'
                    '该广告要足够吸引人。')
    first_frame = ask('{' + framework + '}'
                      + '以上内容为一个 15 秒广告的故事框架，现在设计一个开场画面，'
                      '生成用于绘制该开场画面的提示词，'
                      '用于输入 Stable Diffusion 生成图像，用英文生成。')
    clips = build_clips('广告', framework, first_frame, 3)   # 3 x 5s = 15s
    return [framework, first_frame, clips]
```

```
def run():
    idea = input("Describe your movie in under 20 English words: ")

    print("=== Part 1: opening ===")
    result_open = build_open(idea)
    print("=== Part 2: 20s mystery ===")
    result_question = build_question(idea)
    print("=== Part 3: 15s advertisement ===")
    result_value = build_value("卖洗发水的广告")
    print("=== Part 4: 20s resolution ===")
    result_answer = build_answer(result_question[0])

    # keep a log of every prompt / 把每条提示词记入日志
    with open("prompts_record.txt", "a") as log:
        log.write("=== new run: " + idea + " ===\n")
        log.write("framework:\n" + result_question[0] + "\n")
        for i, clip in enumerate(result_question[2], start=1):
            log.write("clip " + str(i) + ":\n" + clip + "\n")
    print("All prompts saved to prompts_record.txt")

run()
```

Walkthrough 精解

1. `ask(content)` is three lines that earn their keep: every conversation with the model — there are more than twenty in a full run — now goes through one door. Change the message format once and the whole program follows.

   `ask(content)` 是三行却物有所值：与模型的每一次对话——完整运行超过二十次——现在都从同一扇门走。消息格式改一处，整个程序随之更新。

2. `build_clips(kind, framework, first_frame, n_clips)` is the heart. The reference program repeats its while loop in three functions, differing only in the word 电影/广告 and the count; the helper takes those as *parameters* (Chapter 8's whole thesis in one move).

   `build_clips(kind, framework, first_frame, n_clips)` 是全项目的心脏。参考程序在三个函数里重复同一段 while 循环，差别只有"电影/广告"一词和次数；辅助函数把这两点变成*参数*——这正是第 8 章全部主旨的一次落地。

3. The `while i < n_clips` loop is Requirement 3 made visible: clip 1 is anchored to the first frame, and every later prompt carries the framework plus `';'.join(prompts_list)` — all previous clips. Long-range coherence needs long-range memory.

   `while i < n_clips` 循环把要求 3 变成可见的代码：第一段锚定首帧，之后每条提示词都携带框架与 `';'.join(prompts_list)`——此前全部片段。长程连贯需要长程记忆。

4. `build_value` proves the machinery generalises: identical code, different `kind` string and a shorter count (3 × 5 s = 15 s). In your presentation, saying "the ad is the same function with different arguments" is a stronger demonstration than any slide.

`build_value` 证明这套机器可以泛化：代码相同，只是 `kind` 字符串不同、次数更短（3 × 5 秒 = 15 秒）。在展示环节，"广告就是同一个函数换不同实参"这句话比任何幻灯片都更有说服力。

5. `run()` closes the loop with the *user*: `input()` enforces Requirement 5 (under 20 words), the four builders run in order, and the `with open(..., "a")` log means every run of the program leaves evidence — your making-of writes itself.

   `run()` 用*用户*闭合了循环：`input()` 落实要求 5（20 词以内），四个构建函数按序运行，而 `with open(..., "a")` 日志让程序的每次运行都留下证据——你的制作花絮自动生成。

17.3 A Sample Run, Annotated

一次示例运行（带注释）

▾ **a full run of run()** run() 的一次完整运行

OUTPUT 输出

```
Describe your movie in under 20 English words: a robot learning to paint
=== Part 1: opening ===
以下是第 1 个片段的提示词:
=== Part 2: 20s mystery ===
以下是 20s 电影整体框架:
以下是第 1 个片段的提示词:
以下是第 2 个片段的提示词:
以下是第 3 个片段的提示词:
以下是第 4 个片段的提示词:
=== Part 3: 15s advertisement ===
以下是第 1 个片段的提示词:
以下是第 2 个片段的提示词:
以下是第 3 个片段的提示词:
=== Part 4: 20s resolution ===
以下是第 1 个片段的提示词:
以下是第 2 个片段的提示词:
以下是第 3 个片段的提示词:
以下是第 4 个片段的提示词:
All prompts saved to prompts_record.txt
```

OUTPUT 输出

```
--- prompts_record.txt (excerpt / 节选) ---
=== new run: a robot learning to paint ===
framework:
【故事框架】在废弃的工厂里，一台旧机器人发现了一盒颜料……它开始
笨拙地模仿墙上斑驳的画。镜头在它颤抖的金属手指与色彩之间切换。
第 20 秒：机器人在画布前停下——画布上出现的，不是它模仿的那幅画，
而是一张人类的脸。画面骤然变黑。（悬念：那是谁的脸？）
clip 1:
镜头缓推：昏暗厂房，金属手指第一次蘸满颜料，特写颤抖的笔触……
```

Walkthrough 精解

1. Count the AI conversations: 2 (open) + 6 (question) + 5 (value) + 6 (answer) = 19 calls, each a full LLM turn. Budget minutes, not seconds, for a live demo — and keep the pre-generated fallback the handbook recommends.

   数一数 AI 对话：2（开场）+ 6（悬念）+ 5（广告）+ 6（解答）= 19 次调用，每次都是一轮完整的大模型对话。现场演示请按"分钟"而不是"秒"做预算——并备好学习手册建议的预生成兜底输出。

2. The framework text in the log is the story's skeleton. Read it before reading the clips: every clip prompt re-injects this text, which is why the robot never changes identity mid-film.

   日志里的框架文本是故事的骨架。读片段提示词之前先读它：每条片段提示词都重新注入这段文字——这正是机器人不会在片中途变脸的原因。

3. The `enumerate(..., start=1)` in `run()` numbers the clips automatically — a small library function (Chapter 7) replacing manual counter arithmetic.

   `run()` 里的 `enumerate(..., start=1)` 自动为片段编号——一个小小的库函数（第 7 章）取代了手工计数器运算。

4. The excerpt shows the making-of material writing itself: the log is timestamped evidence of what the program produced and when. Screenshot `prompts_record.txt` opening in nano for your report.

   节选显示制作花絮正在自动生成：这份日志就是"程序在何时产出了什么"的带时间戳证据。请在报告里截一张 nano 打开 `prompts_record.txt` 的图。

17.4 The Five Requirements, Checked

五条硬要求逐条核对

#	Requirement	How the demo answer satisfies it
1	Runs on Linux 能在 Linux 上运行	Developed and executed on the course server with `python3`. 在课程服务器上用 `python3` 开发与执行。
2	Modified, not copied 基于参考修改而非照抄	New `ask()` helper; three duplicated loops collapsed into one `build_clips()`; `input()` and log file added. 新增 `ask()`；三个重复循环合并为

		`build_clips()`；加入 `input()` 与日志文件。
3	Iterative generation with loops 用循环迭代生成	`while i < n_clips` builds each clip from all previous clips. `while i < n_clips` 基于此前全部片段生成下一段。
4	Useful blocks in functions 有用代码封装为函数	Seven functions; `run()` is the only top-level code. 七个函数；顶层代码只有 `run()`。
5	Input under 20 words 输入不超过 20 词	`input()` at the start of `run()`; demo uses "a robot learning to paint" (6 words). `run()` 开头的 `input()`；演示输入 "a robot learning to paint"（6 词）。

Bring this table — filled with *your* file names and *your* modifications — to the final presentation. It answers the examiners' first question before they ask it.

把这张表填上*你的*文件名和*你的*改动，带到期末展示现场。它在考官开口之前就回答了他们的第一个问题。

17.5 Common Project Mistakes
项目常见错误

> **MISTAKE 1: THE AMNESIAC LOOP** 错误一：失忆的循环
>
> Building each clip from only the *previous* clip (`prompts_list[-1]`) instead of the whole history. The film starts strong and drifts: by clip 4 the robot has become a teapot. The fix is one variable — pass `';'.join(prompts_list)`, the *full* history, into every prompt.
>
> 每段只基于*上一段*（`prompts_list[-1]`）而不是全部历史来生成。电影开头惊艳，然后开始漂移：到第 4 段，机器人变成了茶壶。修复只需一个变量——把 `';'.join(prompts_list)` 这份*完整*历史传进每条提示词。

> **MISTAKE 2: THE COPIED TRILOGY** 错误二：复制三连

Pasting the reference's `buildQuestion` and changing two words to create `buildMyQuestion`. That fails Requirement 2 ("must be modified, not copied") and earns no reuse credit. A parameterised helper *is* the modification.

把参考程序的 `buildQuestion` 复制一遍、改两个词，造出 `buildMyQuestion`。这既不满足要求 2（"必须修改而非照抄"），也拿不到复用的分数。参数化的辅助函数*本身就是*那种修改。

MISTAKE 3: UNREHEARSED LIVE DEMO 错误三：没有排练的现场演示

Typing a fresh 20-word idea on stage for the first time. Long inputs, network hiccups and an untested Chinese/English mix have ended more demos than any bug. Choose the input in advance, rehearse twice, and keep `prompts_record.txt` as your fallback.

在台上第一次现场输入一个新点子。过长的输入、网络抖动、没测过的中英混排——葬送的演示比任何 bug 都多。提前选定输入词，排练两遍，并让 `prompts_record.txt` 做你的兜底。

STRETCH GOAL 进阶目标

One free improvement that impresses: have `run()` print a progress line with elapsed time before each builder (`time.time()` from Exercise 16.4), so your audience watches the pipeline breathe instead of staring at silence.

一个免费但能加分的改进：在每个构建函数运行前用 `time.time()`（练习 16.4）打印一条带已用时间的进度行，让观众看到流水线在呼吸，而不是盯着沉默。

PART VII

Reference / 附录与参考资料

Index of Demo Programs / 示范程序索引

Every program in this codebook, in course order. Scripts marked with a bullet are interactive — they need your input when run, so the "output" in their walkthrough is one possible session.

本册全部示范程序，按课程顺序排列。带圆点标记的脚本是交互式的——运行时需要你的输入，因此其精解中的"输出"只是某一次会话。

Script	Ch.	Purpose
command_sequence.sh	1	The full create-copy-move-rename-delete sequence / 建复移重删全流程
notes.txt	1	First nano edit / 第一次 nano 编辑
`hello.py`	2	Two printed lines / 两行打印
`broken_indent.py`	2	An IndentationError on purpose / 故意的缩进错误
`structure.py`	2	First function: `greet(name)` / 第一个函数
`variables.py`	3	Values, types, and one conversion / 值、类型与一次转换
`truthiness.py`	3	Five `bool()` predictions verified / 五个布尔预测验证
`float_trap.py`	3	`0.1 + 0.2` and the `Decimal` fix / 浮点陷阱与修复
`bases.py`	3	Binary, octal, hex conversions / 二、八、十六进制换算
`operators.py`	4	All seven arithmetic operators / 七种算术运算
`precedence.py`	4	One expression, two readings / 一个表达式两种读法
`string_compare.py`	4	Why `'10' < '2'` / 字符串比较
`alias_trap.py`	4	Chained assignment and the fix / 链式赋值陷阱与修复
`strings.py`	5	Index, slice, strip, split / 索引切片与拆分
`joining.py`	5	`join` versus `+` / 两种拼接

`my_name.py`	5	Reverse, length, uppercase / 反转、长度、大写
`fstring.py`	5	From `+` chains to f-strings / 从加号链到 f-string
`csv_play.py`	5	Split, replace, rejoin / 拆分替换重组
`traffic_light.py` •	6	Four branches, four inputs / 四分支四输入
`ticket_system.py`	6	The elif chain with evidence / 带证据的 elif 链
`grade_system.py` •	6	Loop until `q`, map to grades / 循环到 q 并映射等级
`morning_run.py`	7	First `while` loop / 第一个 while 循环
`loop_practice.py`	7	Three forms of `range()` / range 的三种形态
`vending_machine.py` •	7	Validation and change / 校验与找零
`loop_control.py`	7	`break`, `continue`, loop `else` / 三种循环控制
`nested_loops.py`	7	The 3×3 multiplication table / 三三乘法表
`guessing_game.py` •	7	Multi-round game with best record / 带最佳纪录的多轮游戏
`sandwich_function.py`	8	Parameters at work / 参数的威力
`flexible_sandwich.py`	8	Positional and keyword arguments / 位置与关键字实参
`circle_calculator.py`	8	Pizza pricing by area / 按面积为披萨定价
`shipping_calculator.py`	8	Default parameters / 默认参数
`box_calculator.py`	8	Returning a tuple / 返回元组
`bmi_calculator.py`	8	Formula + round-trip test / 公式与往返测试
`scope_demo.py`	8	Local shadows global / 局部遮蔽全局
`global_demo.py`	8	The `global` keyword / global 关键字
`kitchen.py`, `use_kitchen.py`	9	Your first module pair / 第一对模块
`cooking_module.py`	9	A module you author / 你编写的模块

`math_practice.py`	9	Hypotenuse, circle, power / 斜边、圆、幂
`countdown.py`	9	`time.sleep` drama / 带停顿的倒计时
`lottery.py`	9	`randint` and `choice` / 随机数与随机选取
`stdlib_practice.py`	9	datetime + random + os / 三库合体
`list_creation.py`	10	Four creation methods / 四种创建方式
`list_access.py`	10	Indexes and slices / 索引与切片
`list_add.py`	10	Three insert positions / 三种插入位置
`list_remove.py`	10	First-match removal / 只删第一个匹配
`student_grades.py`	10	Average and ranking / 平均分与排名
`tuple_demo.py`	10	Immutability, demonstrated / 演示不可变性
`tuple_unpack.py`	10	Unpacking in one line / 一行解包
`tuple_single.py`	10	`(42)` versus `(42,)` / 单元素陷阱
`tuple_return.py`	10	Area and circumference together / 面积与周长同回
`dict_create.py`	11	Three creation methods / 三种创建方式
`dict_modify.py`	11	Update, add, delete, `get` / 更新增删与安全读取
`translator.py` •	11	Dictionary lookup with fallback / 带兜底的查找
`movie_rating.py` •	11	Membership test on keys / 键的成员测试
`set_dedup.py`	11	Duplicates vanish / 去重
`set_operations.py`	11	Intersection, union, difference / 交并差三运算
`set_voting.py`	11	Sets count, dicts tally / 集合与字典协作
`student_system.py`	11	Records sorted by `key=` / 按 key 排序的记录
`word_frequency.py`	11	Top-5 words of a text / 文本前五高频词

`classlist.txt` + `greet_class.py`	12	First file-reading program / 第一个读文件的程序
`two_writers.py`	12	`write()` versus `writelines()` / 两种写法对比
`diary_appender.py` •	12	Append mode across runs / 跨运行的追加模式
`greet_class_with.py`	12	The `with` statement / with 语句
`copy_image.py`	12	Binary copy with nested `with` / 嵌套 with 复制二进制
`path_check.py`	13	Existence, type, size / 存在性、类型、大小
`daily_folder.py`	13	Auto `lab_YYYYMMDD` folder / 自动按日建夹
`batch_rename.py`	13	Filtered batch rename / 带过滤的批量重命名
`gbk_to_utf8.py`	13	Encoding conversion / 编码转换
`csv_reader.py`	13	Chinese CSV, right and wrong / 中文 CSV 的对与错
`bmi_calculator.py` (v2) •	14	`try/except/else` input handling / 异常版输入处理
`file_detective.py`	14	Catching `FileNotFoundError` / 捕获文件缺失
`catch_all.py`	14	`except Exception as e` / 兜底网
`multiple_exceptions.py` •	14	Two handlers, three inputs / 两个处理器三种输入
`grades_tuple.py`	14	Tuple-style catching / 元组式捕获
`safe_divide.py`, `safe_config.py`	14	The full four-room pattern / 四房间完整模式
`retry_dynamic.py` •	14	Retries with decreasing waits / 递减等待的重试
`username_check.py` •	14	Raising your own errors / 亲手抛错
`requests_demo.py`	15	Status code + first 100 chars / 状态码与百字预览
`weather_reporter.py`	15	Live weather, live timezone flaw / 实时天气与时区缺陷
`backup_helper.py`	15	`os` + `shutil` backup / 备份助手
`thread_demo.py`	16	First background task / 第一个后台任务

`thread_args.py`	16	Arguments via `args=(x,)` / 用元组传参
`thread_queue.py`	16	Results via `queue.Queue` / 队列传回结果
`download_race.py`	16	Sequential vs threaded, timed / 顺序与并发计时对比
`movie_writer.py•`	17	The final project demo answer / 期末项目示范答案
`prompts_record.txt`	17	The run log / 运行日志

From Exercise to Submission / 从练习到提交

The handbook's practice guide asks for one artifact every week: a PDF of screenshots submitted to the "Weekly Report Assignment". This checklist turns any exercise in this book into that artifact.

学习手册实践指南要求每周交付同一件东西：提交到"Weekly Report Assignment"的截图 PDF。这份清单把本书的任何一道练习变成那件东西。

- Before coding: re-read the task box and write, in one sentence, what "done" will look like.
  动手前：重读题目框，用一句话写下"怎样算完成"。
- Set up: log in with Termius, `cd` to your exercise folder, `ls` to confirm a clean workspace.
  准备：Termius 登录，`cd` 到练习目录，`ls` 确认环境干净。
- Build: type the program with nano — no pasting. Run it once before you expect anything.
  构建：用 nano 手敲程序——不要粘贴。先跑一次，不抱期待地跑。
- Capture: screenshot every run that the walkthrough asks for, including deliberate errors. The terminal command must be visible in frame.
  记录：把精解要求的每次运行都截图，包括故意的报错。截图里必须能看到终端命令。
- Explain: add the walkthrough's key sentence to your slide in your own words — copied sentences read as copied.
  解释：把精解的关键句用自己的话写进幻灯片——照抄的句子一眼便知。
- Submit: export the deck to PDF, name it with week and exercise, upload before the deadline.
  提交：导出 PDF，按"周次+题号"命名，截止前上传。
- Reflect: one line per exercise — what surprised you, what you would do differently. This line is worth more in grading than a perfect screenshot.
  复盘：每题一行——哪里让你意外，哪里你会换个做法。这一行比完美的截图更能加分。

A Field Guide to Errors in This Book / 本书错误现场速查

When a demo answer misbehaves on your machine, the error usually names the chapter that will explain it. Find the message, jump to the fix.

当示范答案在你的机器上表现异常时，报错信息通常已经点名了负责解释它的章节。找到信息，跳到修复。

Message	Usual cause here	Fix
`IndentationError: expected an indented block`	The line under `if`/`def`/`for`/`while` is not indented `if`/`def`/`for`/`while` 下一行没有缩进	Four spaces, always (Ch 2) / 永远四个空格（第 2 章）
`NameError: name 'x' is not defined`	Typo in a name, or use before assignment 名字打错，或先使用后赋值	Check spelling; define before use (Ch 2–3) / 检查拼写，先定义后使用
`TypeError: can only concatenate str`	`str + int` in a print or prompt 在 print 或提示词中 `str + int`	`str(x)` or an f-string (Ch 4–5) / 用 `str(x)` 或 f-string
`IndexError: list index out of range`	Index equals `len()`, or an empty list 索引等于长度，或列表为空	Print `len()` first; slices never raise (Ch 5, 10) / 先打印 len；切片不报错
`KeyError: 'xxx'`	Dictionary miss 字典缺键	`.get(key, default)` (Ch 11) / 用 get 带兜底
`ValueError: invalid literal for int()`	`int("q")` or empty input 对非数字调用 int	Check for `q` first, or `try/except` (Ch 6, 14) / 先判断 q 或用 try
`ZeroDivisionError`	Height, divisor or count is 0 身高、除数或计数为 0	Guard or `except ZeroDivisionError` (Ch 14) / 守卫或捕获
`FileNotFoundError`	Working directory is not where you think 工作目录与你以为的不同	`pwd`/`os.getcwd()`, or `os.path.exists` first (Ch 12–13) / 查看目录，先检查存在
`UnicodeDecodeError`	Reading GBK bytes as UTF-8	`open(..., encoding="gbk")` (Ch

	以 UTF-8 读 GBK 字节	13) / 指定编码
`ModuleNotFoundError`	Importing a file in another folder 导入了别的目录的文件	Keep modules in the same folder (Ch 9) / 模块放同目录
`TypeError: 'tuple' object does not support item assignment`	Assigning into a tuple 给元组赋值	Use a list if data must change (Ch 10) / 会变的数据用列表
`AttributeError: 'NoneType' object has no attribute ...`	Using the return of a method that returns nothing 使用了无返回值方法的结果	`append`/`remove` return `None`; keep the list itself (Ch 10) / 修改型方法返回 None
Program hangs forever	A `while` whose condition never turns false while 条件永假不了	Check the update line inside the loop (Ch 7) / 检查循环内的更新语句
Output interleaves strangely	Two threads printing without order 两线程乱序打印	Normal; use `join()` before final prints (Ch 16) / 正常现象，汇合后再收尾

End of the Answer Book

答案册完

Now close the book, open nano, and write it your own way.

现在，合上答案册，打开 nano，用你自己的方式写一遍。